Software Solution

INSY7315 - Portfolio of Evidence

Prepared for: Quloobul Moe'Mieneen



DRPS Solutions Ltd.

Project Team

Name	Student Number	Role
Uzair Essa	ST10440109	Project Manager
Dillon Rinkwest	ST10439147	Platform Engineer
Patrick Shutte	ST10440987	Software Engineer (Backend/QA)
Liam Swile	ST10438679	Frontend Engineer

Table of Contents

Table of Figures vii
Introduction 1
The Client 1
The Solution 1
List of stakeholders 1
Problem-Vs-Solution 2
Risk Matrix 4
Requirements 5
User Roles 5
User Stories 6
UX Journey Map 6
Navigation Diagrams 6
Journey by Roles: 8
Use Cases 9
Use Case Briefs 11
Project Plan 20
Work Breakdown Structure 21
Non-Functional Requirements 26
Availability and Reliability 26
Performance and Scalability 27
Security 28
Data Protection and Regulatory Compliance (POPIA) 31
Data Integrity and Consistency 31
Usability and Support 33
Auditability and Traceability 33
Maintainability and Deployability 34
Portability and Multi-Client Support 35
Domain Class Diagram 36
Explanations 37
Design Class Diagram 39

Implementation 43
Sequence Diagrams 43
State Diagrams 47
Deployment Diagram 50
Data Schema 51
Choice of Technology 51
Why relational database 51
Why Azure SQL Database 52
Support for the project requirements 53
Future scalability 54
Entity Relationship Diagram (ERD) 55
Notation and conventions 56
Subject areas 57
Identity and access 57
Projects and tasks 58
Meetings and events 58
Resource requests 58
Files and reporting 59
Notifications and audit 59
Relationships 60
Identity and access 60
Projects and tasks 61
Meetings and events 62
Resource requests 63
Files and reporting 63
Notifications and audit 64
Design Decisions 65
Referential Integrity 66
Indexing 66
Forward engineering 66
Architecture Plans 67
QULOOBUL MOE'MIENEEN Management Platform 67

Architecture Artifacts 67
Design Patterns 68
Architecture Patterns 72
Pattern chosen 73
Reasoning 73
Known limits 74
Cloud Architecture 75
Services used 76
Network segregation 76
Protocols between services 77
Technology and their trade-offs 78
Known limits of this cloud architecture 79
Diagram index 79
DevOps 80
Version control 80
Branching model and development flow 80
The GitHub Actions pipeline 82
Stages in detail 83
Automated testing in the pipeline 84
Quality gates 85
Secrets and configuration 85
How it meets the non-functional requirements 86
API Research 87
Internal - Azure Platform Built-In APIs 87
Azure SQL Database API (via Entity Framework Core) 87
ASP.NET Core Web API 88
ASP.NET Core Identity / Azure Entra External ID (AD B2C) 88
Azure Blob Storage 89
Azure SignalR Service 90
Azure Key Vault 90
Azure Notification Hubs 92
External - Email and Notifications 93

Resend 93
Azure Communication Services (ACS) 93
SendGrid (Twilio) 94
External - AI & Intelligence Features 95
Anthropic Claude API 95
OpenAI API 96
Google Gemini API 97
External - Authentication Extensions 98
Azure Entra ID (Active Directory) SSO 98
Clerk 98
External - Invoicing & Accounting APIs without confirmation 100
Xero API 100
Sage Business Cloud Accounting API 101
QuickBooks Online API (Intuit) 102
Invoice Ninja API 102
Stripe Invoicing / Billing API 103
Consolidated Cost Summary 105
Recommendation Summary 107
System Security 109
Network and Infrastructure Security 109
Transport Layer Security (TLS/HTTPS) 109
Firewall and Network Access Control 109
Secrets and Credential Management 110
Security Threat Analysis and Mitigation 110
Security Design Principles 112
POPIA Compliance 114
Data Minimisation 114
Data Residency 114
Encryption 114
Consent and Documentation 114
Breach Response 114
Risk Matrix 115

Summary 118
Technical & Financial Feasibility 119
2. Technical Feasibility 119
2.1 Hardware 120
2.2 Software 121
2.3 Network Requirements 124
2.4 Database Resources 125
2.5 Compliance & Security 126
2.6 Mobile-Specific Technical Requirements 126
3. Financial Feasibility 127
3.1 Phase 1 Infrastructure Costs (24/7 Uptime) 127
3.2 Microsoft for Nonprofits Azure Credit 129
3.3 Ongoing and Additional Costs 130
3.4 Cost Scenarios 131
3.5 Optional: Invoicing Module Costs 131
4. Feasibility Summary 132
Members CV Documents 134
Prototypes 139
Web application 139
GitHub 139
Video demonstration 139
Mobile application 139
GitHub 139
Video demonstration 139
References 140
AI Declaration of Usage 146
Sections where AI was used: 146
Sections where AI was used: 146
Sections where AI was used: 147
Sections where AI was used: 147
Sections where AI was used 147
Sections where AI was used 148

Sections where AI was used 148
Sections where AI was used 149

Table of Figures

Figure 1: Risk Matrix with legend for the system 4
Figure 2: Navigation Diagram of Mobile Application 6
Figure 3: Navigation Diagram of Web Application 7
Figure 4: Use case diagram - Staff Member interactions with the system. 9
Figure 5: Use case diagram - Admin Staff interactions with the system, including generalization from the Staff Member actor. 10
Figure 6: Project Plan 20
Figure 7: Work Breakdown Structure 21
Figure 8: QULOOBUL MOE'MIENEEN Domain Class Diagram 36
Figure 9: Design Class Diagram (Identity and Access) 39
Figure 10: Design Class Diagram (Collaboration- Events & Meetings) 39
Figure 11: Design Class Diagram (Resource Request, Notifications & Audit) 40
Figure 12: Design Class Diagram (Projects & Tasks) 41
Figure 13: Full Design Class Diagram for QULOOBUL MOE'MIENEEN 42
Figure 14: SD-01 Login/Authentication (UC-01) 43
Figure 15: SD-02 Assign Task and Notify Assignee (UC-04, UC-08) 44
Figure 16: SD-03 Submit and Review Resource Request (UC-05) 45
Figure 17: SD-04 Upload and Attach File to a Task (UC-07) 46
Figure 18: SM-01 ProjectTask (UC-02, UC-04) 47
Figure 19: SM Resource Request (UC-05) 48
Figure 20: SM-03 User (UC-01, UC-06) 49
Figure 21: DEP-01 Deployment Diagram 50
Figure 22: Support for the project requirements table 53
Figure 23: QULOOBUL _MOE'MIENEEN ERD 55
Figure 24: Subject Areas 57
Figure 25: Identity and access relationships 60
Figure 26: Projects and tasks relationships 61
Figure 27: Meetings and events relationships 62
Figure 28: Resource requests relationships 63
Figure 29: Files and reporting relationships 63
Figure 30: Notifications and audit relationships 64
Figure 31: Referential Integrity table 66
Figure 32: OBJ-01 Object Diagram 68
Figure 33: SA-01 System Architecture Diagram 72
Figure 34: CA-01 Cloud Architecture Model 75
Figure 35: CI/CD Pipeline and Branching Model (DO-01) 82

Introduction

DRPS Solution Ltd. is the virtual company used to present the group's persona and professional interface to QULOOBUL MOE'MIENEENs/NGOs that this group is here to tackle issues and develop digital technical solutions. The Company's goal is to tackle issues (by understanding them through research), create meaningful steps to solve problems (via proper planning and documentation), and develop industry-standard coding solutions to these issues. To ensure the company met a project that was both challenging and within the team members' skill level, the company sought the Non-Profit Organization (QULOOBUL MOE'MIENEEN).

The Client

QULOOBUL MOE'MIENEEN is an NPO whose vision statement is "To be a world-class Islamic Institution striving to achieve the satisfaction of Allah (SWT) by promoting unity, spiritual growth, empowerment, well-being and self-sustainability". The QULOOBUL MOE'MIENEEN detailed their issues and struggles with their system of project planning and organising of events, as well as possible occurrences such as construction, for which the QULOOBUL MOE'MIENEEN requires management of communication, resources and timelines. This created an opportunity for DRPS Solution Ltd. to propose an IT Solution, a new system accessed via a mobile app and a website that can host their sensitive data, offer project planning systems, and create communication capabilities for users.

The Solution

The solution that was decided upon takes the form of web-based project management software as well as a mobile application that provides further access to the system. This system encompasses the means to create and track tasks, events, files, and the different users in the organization. The project will make use of real-time notifications through SignalR, data storage in databases such as Azure, and other functions like voice recordings. All of which QULOOBUL MOE'MIENEEN is happy with.

List of stakeholders

- **13 Exco members, which include the chairman, secretary & treasurer**
- **Facility manager**
- **2 imaamat members**
- **2 administrators**

- **4 general staff**
- **2 caretakers**

24 members in total

Problem-Vs-Solution

Problems	Solutions
The organization required an IT solution that could allow them to organize the daily tasks and manage their events.	The solution that seemed best was a web application so that there could be wide support.
The staff suffers from daily objectives being misinterpreted, miscommunicated, and unplanned.	A digital objective board similar to a Kanban board could help properly plan tasks and communicate them with higher efficiency.
During organization meetings, minutes recorded are stored on separate devices and shared across WhatsApp, which could lead to fragmentation and non-organized data collection.	A recording feature within an application can reduce the problem with proper organization of data recording and automatically store metadata (dates, etc.), which can allow for easier findings.
The staff also suffers from inefficient communication of tasks performed and other vital information pertaining to these tasks.	With an application task view, there can be simple actions to add extra details to each task for both admin-style staff and other staff, such as staff assignees and an extra data set that shows who completed the task.
Staff, when requiring additional resources for tasks or daily maintenance, have issues bringing this to admin staff, such as these requests being sent via WhatsApp, which leads to unrecorded data or loss of the request. Also, requests could be brought to the wrong people, causing issues.	Creating a request page setup within an application that allows tracking and holding of request data can easily alleviate frustration and create an easy and thorough line of communication when it comes to staff requests.

Problems	Solutions
The admin staff requires an overview of all staff and their roles so that events and major changes can properly be expressed and tracked, as well as generally leading to better planning.	This can easily be solved with project overview dedicated pages within the application, as well as a dedicated staff overview page for proper assignment and communication.
The staff also have a problem with sharing vital files. This is done through emails and, as such, leads to possible mismanagement or loss of precious files.	The solution can be the database storage of these files and, through the application, proper recording and categorization of these files, which can lead to easier identification.
The staff requires privacy and levels of access specific to individuals to prevent miscommunication and issues.	The solution to this is login functionality within the application to prevent such an issue from arising.
Staff and admin have no reliable way of being alerted about new tasks, approaching deadlines, or outstanding resource requests, which results in missed deadlines and delayed responses.	An automated in-app notification system that alerts staff and admin in real time when a task, deadline or request is created, updated or due, ensuring important information is not overlooked.
Admin staff struggle to track overall staff performance and project progress over time, which makes it difficult to identify bottlenecks, workload imbalances, or areas needing improvement.	A reporting and analytics dashboard that visualizes task completion rates, staff workload, and project progress over time, supporting data-driven planning and decision-making.

Risk Matrix

Risk ID	Risk Description	Risk Category	Risk Impact	Risk Probability	Risk Rating	Mitigation Strategy
R1	The transition from the previous system to this new system can lead to friction amongst users casuing delays miscommunication and misunderstandings	Usability	4	3	12	To mitigate confusion and create a smooth onboarding experience a User Manual will be created and used to guide the users throughout the system.
R2	Data corruption or data loss via server failure or system error which can cause loss of vital information and records	Data	4	2	8	Backups of the data through automated timelimes via cloud services to prevent data loss or vital information being misplaced
R3	Staff or volunteers could miss updates or not receive notifications like with their previous system	Communication	4	4	16	To combat this a automated email system will be created to ensure email notifications to staff to prevent this miscommunications
R4	Data could be accessible to non-authorized staff and cause vital information to be unprotected	Security	5	2	10	To prevent this proper security practices will be implemented such as HTTPS, hashing authentication, role-base access and protected database access
R5	System downtime or unexpected shutdowns during events or vital moments could be disastrous	Availability	4	2	8	Hosting with reliable and tested services like Azure minimises the risk, stuff like monitoring and setting notifications also help but another step will be planning downtimes or setting out proper needs to appropriate services
R6	The system handling to double booking or schedule conflicts could be inadequate to the company	Functional	3	3	9	Ensuring server-side validation and proper date and time datasets to the system can prevent such inadequacy
R7	The Protection of Personal Information Act (POPIA) compliancy could be violated	Compliance	4	2	8	Only collecting data that is needed, documenting exchanges and ensuring consent as well as proper understanding of the POPIA act are steps to prevent compliancy errors
R8	Insufficient testing and quality control could lead to bugs and errors to the system	Quality	3	3	9	Proper testing practices such as unit testing, automated testing and DevOps testing
R9	Balloning cost or mismanaged resources	Financial	3	2	6	Proper tracking of resources and well formulated resource plans help to ensure that nothing goes overbudget
R10	Balloning requirements and solution not meeting basic requirement for the client	Technical	4	4	16	To mitigate this risk proper measures are taken to achieve full documentation and planning to ensure that the system is tracked, well thoughtout and goals are structured to give the project the best chance to achieve success
	Legend					
	Impact (1-5)	1 = Low	2 = Minor	3 = Moderate	4 = Major	5 = Severe
	Probability (1-5)	1 = Rare	2 = Unlikely	3 = Possible	4 = Likely	5 = Almost Certain
	Risk Rating (1-25)	1 - 6 = Low	7 - 14 = Medium	15 - 25 = High		

Figure 1: Risk Matrix with legend for the system

Requirements

After meetings with QULOOBUL MOE'MIENEEN, DRPS Solution Ltd. has identified these initial solution requirements:

- Task feature
- Recording feature
- Report summary for tasks
- Reminder system
- Request feature
- Project management section
- Login hierarchy
- Import file feature

User Roles

Dillon Rinkwest Roles:

- DevOps engineer
- Backend Engineer

Patrick Shutte Roles:

- Backend Engineer
- Quality Assurance Tester

Liam Swile Roles:

- Frontend Engineer
- Integration Developer

Uzair Essa Roles:

- Project Manager
- Frontend Engineer

User Stories

UX Journey Map

Available Screens/Pages:

- Login Screen/Page - The username and password sign-in display.
- Home Screen/Page - The landing page that holds the task for the day when starting the application.
- Daily Task Screen/Page - The calendar view of all tasks that would be either daily tasks or tasks linked to events or projects.
- Resource Request Screen/Page - This is the web page where users can submit requests for additional resources that would link with tasks or events, and these requests would go to the appropriate superiors.
- Meetings Screen/Page - This is the web page where recording of meetings can be recorded and stored, with extra storage of data such as meeting minutes, notes, and transcripts of the meeting.
- Event Planning Screen/Page – This page is for users to create events/projects to incorporate staff and link tasks to ensure proper project management for the NPO.
- User Management Screen/Page - This page is for administrative personnel and creates a record of employees' progress and task completion.
- Notifications Screen/Page - This page notifies users in the application of task assignments, resource requests, and event linking.

Navigation Diagrams

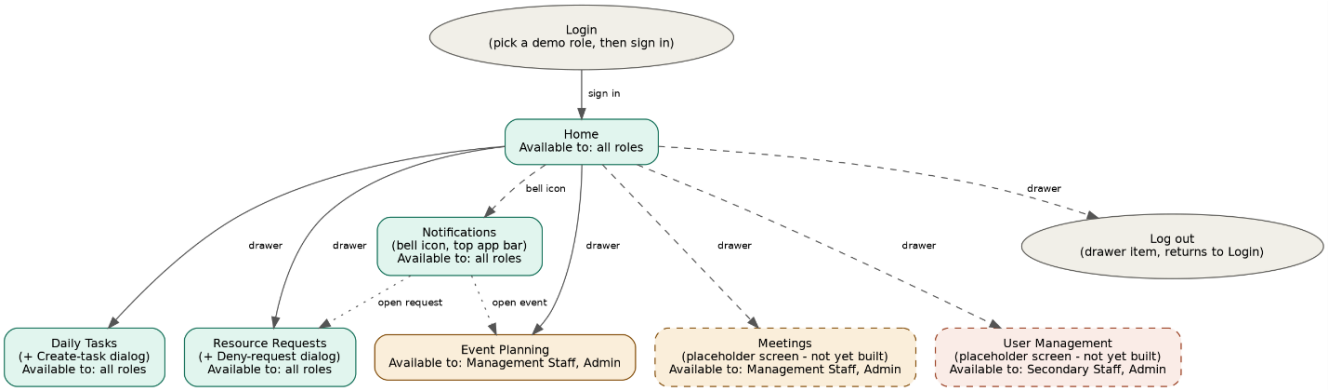


Figure 2: Navigation Diagram of Mobile Application

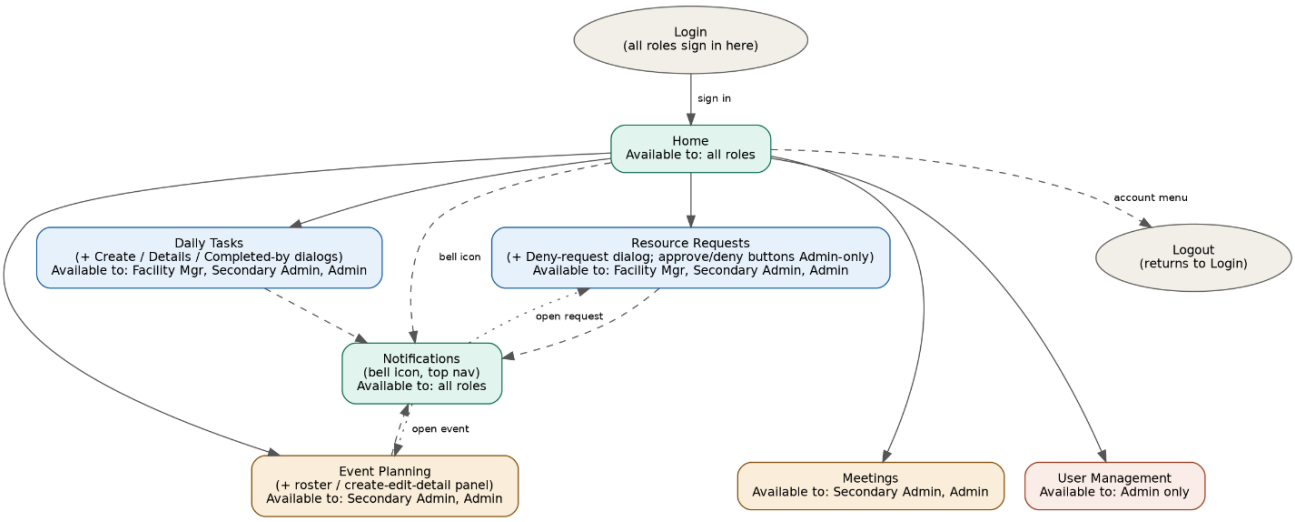


Figure 3: Navigation Diagram of Web Application

Within the application, there will be roles that receive different levels of functionality in order to meet client needs.

Screen/Page	General Staff	Facility Manager	Secondary Admin	Admin
Login Screen/Page	Yes	Yes	Yes	Yes
Home Screen/Page	Yes	Yes	Yes	Yes
Daily Task Screen/Page	No	Yes	Yes	Yes
Resource Request Screen/Page	No	Yes	Yes	Yes
Meetings Screen/Page	No	No	Yes	Yes
Event Planning Screen/Page	No	No	Yes	Yes
User Management Screen/Page	No	No	No	Yes
Notifications Screen/Page	Yes	Yes	Yes	Yes

Journey by Roles:

General Staff:

Signs in and is presented with a home screen displaying the tasks for today and the notification bell for them to see any of the daily tasks assigned to them.

Facility Manager:

Signs in, can see the task set out for today, can see notifications like tasks and events linked to them, and also has access to the resource request system where they can make requests to their supervisor for more resources about tasks or events.

Secondary Admin:

Signs in and can see all screens/pages as the facility manager, but within the resource manager they can approve or deny these resource requests. Additionally, they have access to the event planning screen/page where they can view these events and create events to plan them out.

Admin:

Signs in and has the same access as the secondary admin, except they have access to the user management screen/page where they can see the employee details and tasks performed, as well as tasks still to do.

Use Cases

The case diagrams below identify the two primary actors of the system - the Staff Member and the Admin Staff - and the functions each actor performs. The Admin Staff actor inherits all Staff Member use cases, in addition to a set of administrative use cases used to manage and oversee the organization.

Staff Use Case Diagram

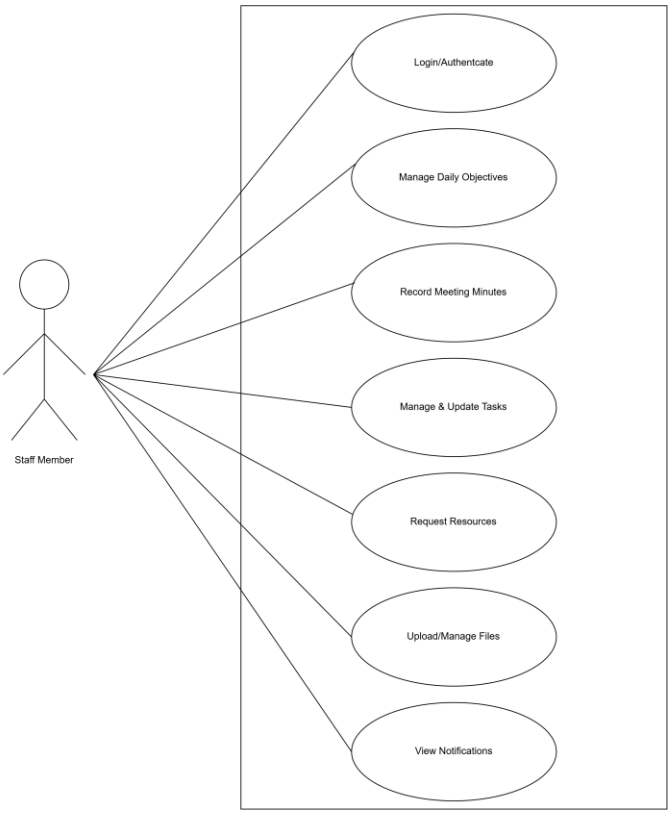


Figure 4: Use case diagram - Staff Member interactions with the system.

Admin Use Case Diagram

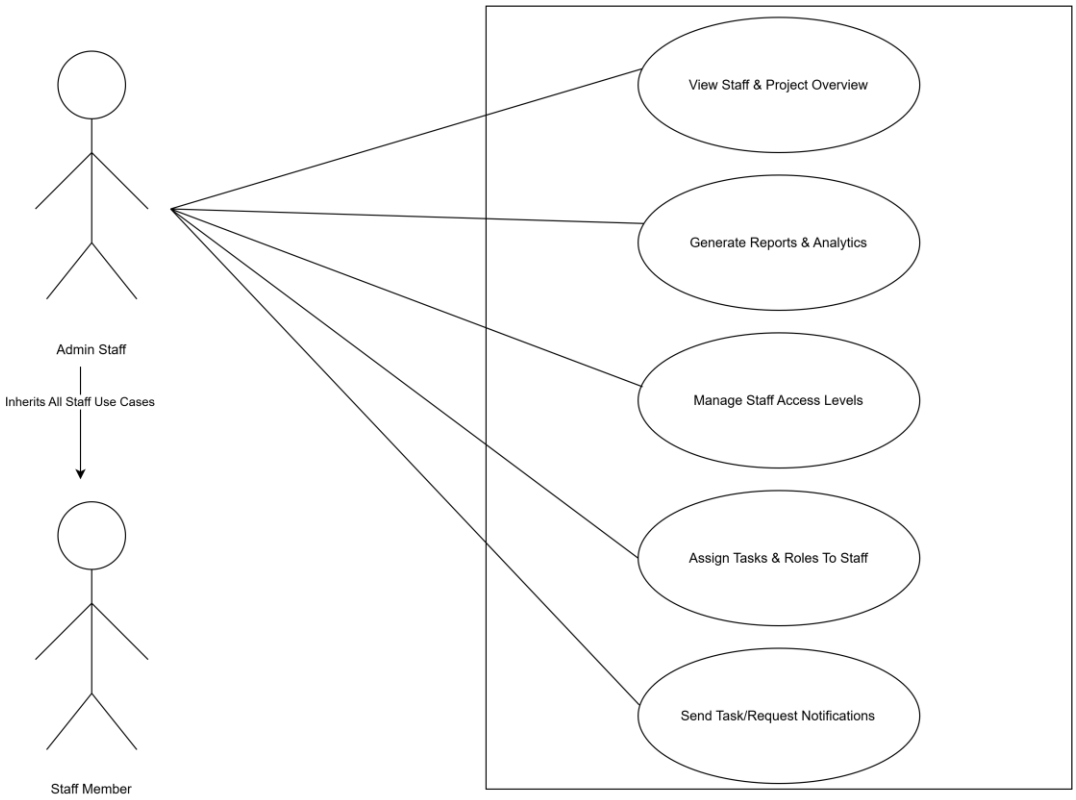


Figure 5: Use case diagram - Admin Staff interactions with the system, including generalization from the Staff Member actor.

Use Case Briefs

UC-01: Login / Authenticate

Use Case ID	UC-01
Use Case Name	Login / Authenticate
Actor(s)	Staff Member, Admin Staff
Description	Allows a registered user to securely access the application using their credentials, ensuring only authorized individuals can view or edit organizational data.
Preconditions	The user has a registered account within the system.
Trigger	User opens the application and selects "Login".
Main Flow	1. User navigates to the login screen. 2. User enters their username/email and password. 3. System validates the credentials against stored records. 4. System grants access and directs the user to their role-based dashboard (Staff or Admin).
Alternate / Exception Flow	● Invalid credentials: system displays an error and allows the user to retry. ● Admin accounts are additionally routed to the access-management area of the dashboard.
Postconditions	The user is authenticated, and privacy levels/access rights are enforced for the remainder of the session.

UC-02: Manage Daily Objectives (Kanban Board)

Use Case ID	UC-02
Use Case Name	Manage Daily Objectives (Kanban Board)
Actor(s)	Staff Member, Admin Staff
Description	Enables staff and admin to plan, view, and update daily objectives on a shared digital kanban-style board, reducing miscommunication of planned work.
Preconditions	User is logged in.
Trigger	User selects the "Objectives Board" from the dashboard.
Main Flow	1. System displays the current board with objectives grouped by status (e.g., To Do, In Progress, Done). 2. User creates, edits, or moves an objective card between columns. 3. System saves the change and updates the board for all users in real time.
Alternate / Exception Flow	● Admin can create objectives on behalf of other staff and reassign them.
Postconditions	The objective board reflects the current, agreed status of daily objectives for all staff.

UC-03: Record Meeting Minutes

Use Case ID	UC-03
Use Case Name	Record Meeting Minutes
Actor(s)	Staff Member, Admin Staff
Description	Allows a user to capture and store organisational meeting minutes directly within the application instead of scattered personal devices or WhatsApp chats.
Preconditions	User is logged in, and a meeting record is being created or edited.
Trigger	User selects "New Meeting Minutes" or opens an existing meeting record.
Main Flow	1. User creates a new minutes entry and enters discussion points, decisions, and action items. 2. System automatically attaches metadata such as date, time, and author. 3. User saves the entry. 4. System stores the minutes centrally, making them searchable by date or keyword.
Alternate / Exception Flow	● User can edit or append to previously saved minutes if permitted by their access level.
Postconditions	Meeting minutes are stored centrally and available for future reference by authorised users.

UC-04: Manage & Update Tasks

Use Case ID	UC-04
Use Case Name	Manage & Update Tasks
Actor(s)	Staff Member, Admin Staff
Description	Provides a task view where staff and admin can add details to tasks, assign staff, and record who completed each task, improving communication of task-related information.
Preconditions	User is logged in.
Trigger	User opens the "Tasks" view.
Main Flow	1. System displays existing tasks with their details and status. 2. User adds or edits task details (description, deadline, priority). 3. Admin assigns a staff member to the task. 4. Assigned staff updates progress and marks the task as complete. 5. System records the assignee and the staff member who completed the task.
Alternate / Exception Flow	● Admin only: reassign a task to a different staff member if the original assignee is unavailable.
Postconditions	Task details, assignment, and completion history are accurately recorded and visible to relevant users.

UC-05: Request Resources

Use Case ID	UC-05
Use Case Name	Request Resources
Actor(s)	Staff Member, Admin Staff
Description	Gives staff a formal, trackable way to request additional resources or maintenance support from admin, replacing informal WhatsApp requests that can be lost or misdirected.
Preconditions	User is logged in.
Trigger	Staff member selects "New Request" from the request page.
Main Flow	1. Staff member completes a request form describing the resource or maintenance need. 2. System logs the request with a timestamp and routes it to the relevant admin staff. 3. Admin reviews the request and updates its status (e.g., Pending, Approved, Declined, Completed). 4. System notifies the requesting staff member of status changes.
Alternate / Exception Flow	● Request is escalated automatically if not actioned within a defined period.
Postconditions	The request and its outcome are permanently recorded and traceable by both staff and admin.

UC-06: View Staff & Project Overview

Use Case ID	UC-06
Use Case Name	View Staff & Project Overview
Actor(s)	Admin Staff
Description	Gives admin staff a consolidated overview of all staff, their roles, and current projects, supporting better planning and communication of events or major changes.
Preconditions	User is logged in with admin privileges.
Trigger	Admin selects the "Staff Overview" or "Project Overview" page.
Main Flow	1. System retrieves current staff, role, and project data. 2. System displays the data on dedicated overview pages. 3. Admin filters or drills down into a specific staff member or project for more detail.
Alternate / Exception Flow	● No alternate flow; read access only.
Postconditions	Admin has an up-to-date view of staff and project status to inform planning decisions.

UC-07: Upload / Manage Files

Use Case ID	UC-07
Use Case Name	Upload / Manage Files
Actor(s)	Staff Member, Admin Staff
Description	Allows staff to store and categorise vital files within the application's database, reducing the risk of loss or mismanagement associated with sharing files by email.
Preconditions	User is logged in.
Trigger	User selects "Upload File" within the relevant project or task.
Main Flow	1. User selects a file and assigns it a category/tag. 2. System stores the file in the database and links it to the relevant project or task. 3. Authorised users can search, view and download the file at a later time.
Alternate / Exception Flow	● Admin only: reorganise or archive files across categories.
Postconditions	Files are securely stored, categorised and easily retrievable by authorised users.

UC-08: Manage Notifications

Use Case ID	UC-08
Use Case Name	Manage Notifications
Actor(s)	Staff Member, Admin Staff
Description	Automatically alerts staff and admin about new tasks, approaching deadlines, and outstanding resource requests, ensuring important updates are not missed.
Preconditions	User is logged in and has notifications enabled.
Trigger	A relevant event occurs, e.g., a task is assigned, a deadline nears, or a request is updated.
Main Flow	1. System detects a triggering event (new task, deadline, request update). 2. System generates a notification and delivers it to the relevant staff or admin user. 3. User views the notification from the dashboard and navigates directly to the related item.
Alternate / Exception Flow	● Admin only: manually send a notification to one or more staff members.
Postconditions	Relevant users are promptly informed of updates affecting their tasks, deadlines, or requests.

UC-09: Generate Reports & Analytics

Use Case ID	UC-09
Use Case Name	Generate Reports & Analytics
Actor(s)	Admin Staff
Description	Provides admin staff with a visual reporting dashboard that summarizes task completion rates, staff workload, and overall project progress to support data-driven decision-making.
Preconditions	User is logged in with admin privileges and sufficient historical task/project data exists.
Trigger	Admin selects "Reports" or "Analytics" from the dashboard.
Main Flow	1. System aggregates task, workload, and project data over a selected period. 2. System generates visual charts (e.g., completion rate, workload distribution, progress over time). 3. Admin reviews the dashboard and may export or filter the report.
Alternate / Exception Flow	● Insufficient data available: system displays a message prompting a wider date range.
Postconditions	Admin gains actionable insight into staff performance and project progress for planning purposes.

Project Plan

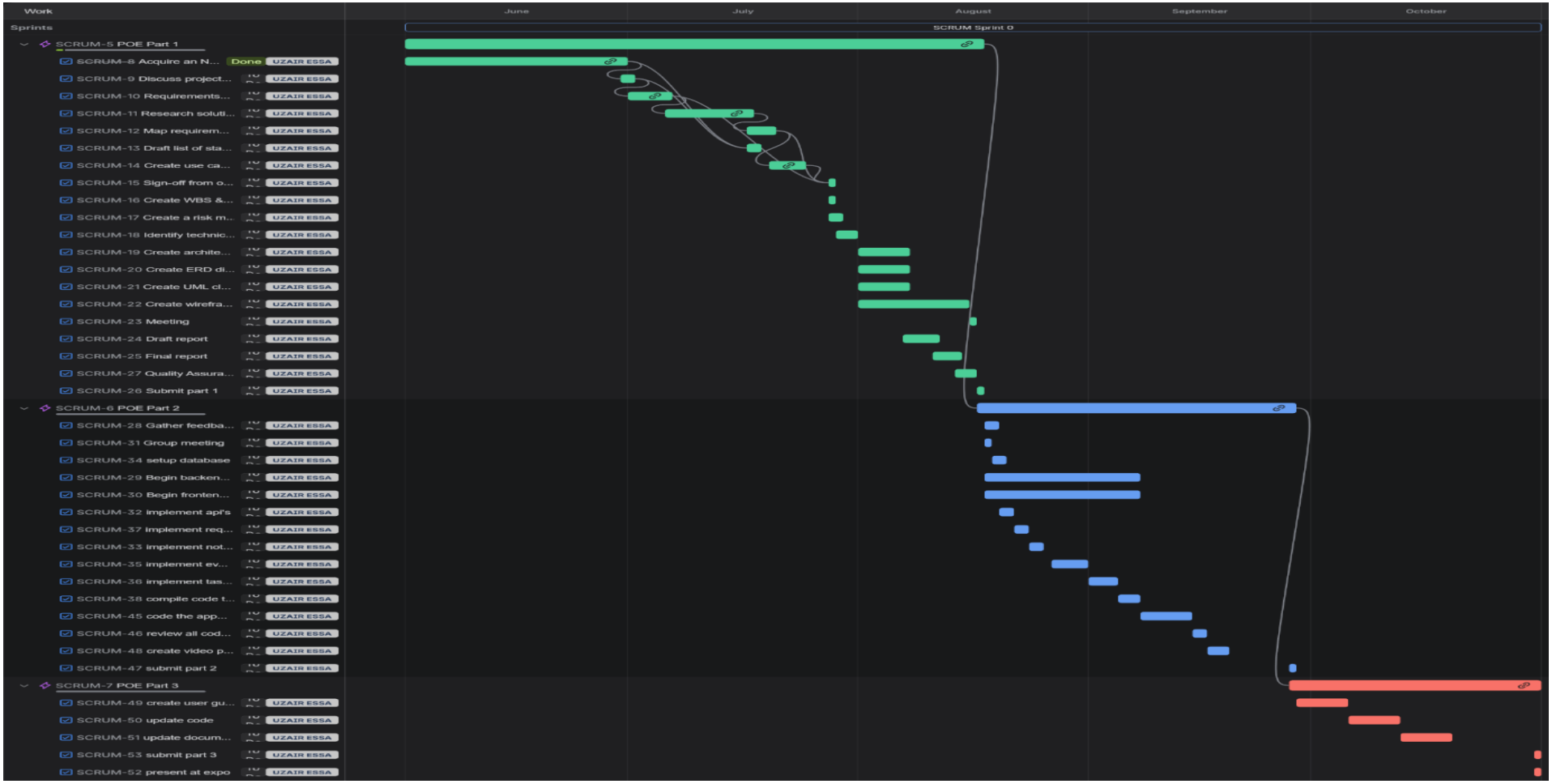


Figure 6: Project Plan

Work Breakdown Structure

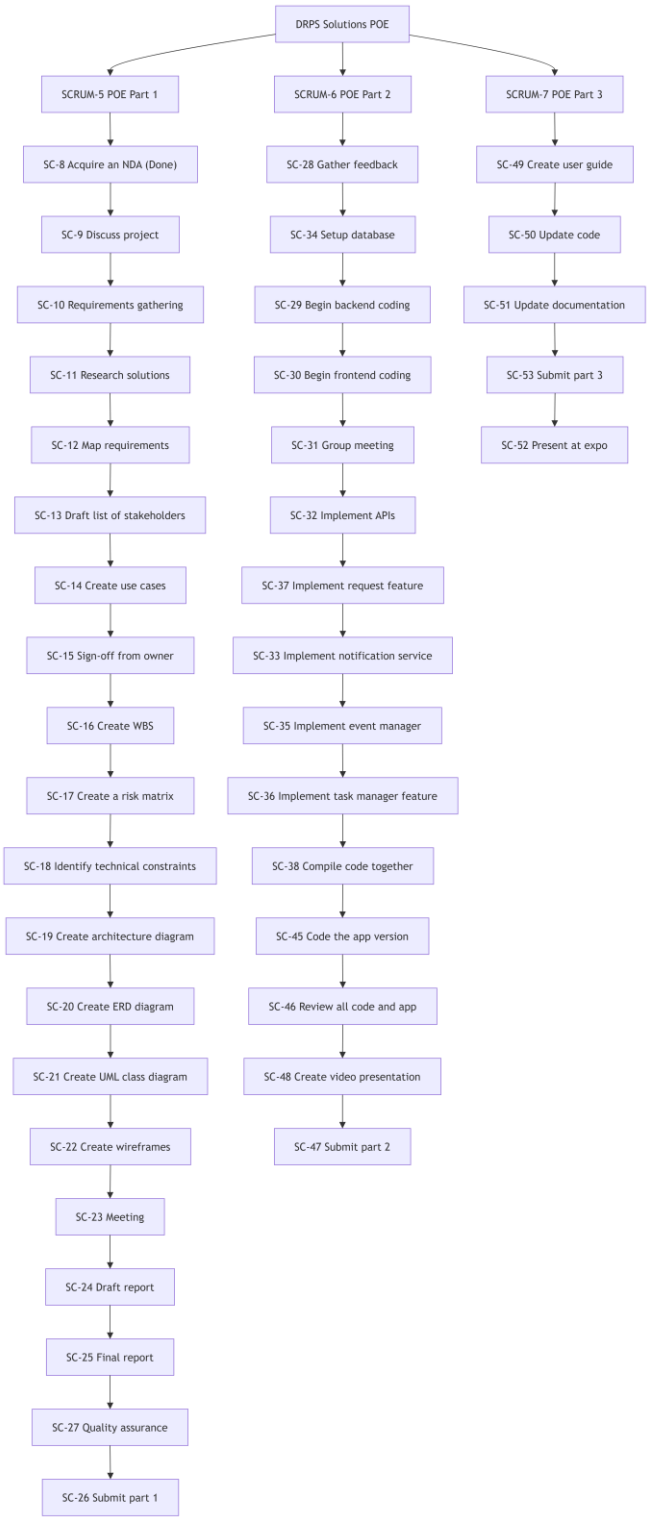


Figure 7: Work Breakdown Structure

Task	Description	Duration	Predecessor(s)	Team Member(s) Responsible	Resources Needed
POE Part 1 - Project Initiation & Planning (SCRUM-5)					
SC-8 Acquire an NDA	Obtain and sign a non-disclosure agreement with the client to protect confidential project information.	1 day	None	Uzair Essa	NDA template, e-signature tool
SC-9 Discuss project	Hold the initial meeting with the client to discuss project scope, goals, and expectations.	1 day	SC-8	Whole team	Meeting room / MS Teams, agenda
SC-10 Requirements gathering	Collect functional and non-functional requirements from stakeholders through interviews and discussion.	2 days	SC-9	Uzair Essa	Interview notes, requirements template
SC-11 Research solutions	Investigate existing tools, platforms and technologies that could address the client's problems.	1 day	SC-10	Dillon Rinkwest	Internet access, comparison spreadsheet
SC-12 Map requirements	Organise the gathered requirements into functional categories and map them to proposed features.	1 day	SC-11	Uzair Essa	Requirements traceability matrix
SC-13 Draft list of stakeholders	Identify and document all individuals and groups affected by or involved in the project.	1 day	SC-12	Uzair Essa	Stakeholder register template
SC-14 Create use cases	Develop use case diagrams and briefs describing how each actor interacts with the system.	2 days	SC-13	Liam Swile	draw.io, use case template
SC-15 Sign-off from owner	Present findings and requirements to the client for approval before proceeding to design.	1 day	SC-14	Uzair Essa	Presentation slides, client meeting
SC-16 Create WBS	Break the project down into manageable tasks, durations, and deliverables.	1 day	SC-15	Uzair Essa	WBS template, Jira
SC-17 Create a risk matrix	Identify potential project risks and assess their likelihood and impact.	1 day	SC-16	Patrick Shutte	Risk matrix template

Task	Description	Duration	Predecessor(s)	Team Member(s) Responsible	Resources Needed
SC-18 Identify technical constraints	Document technical limitations such as budget, hosting and compliance (POPIA) requirements.	1 day	SC-17	Dillon Rinkwest	Technical constraints checklist
SC-19 Create architecture diagram	Design the high-level system architecture showing major components and their interactions.	2 days	SC-18	Dillon Rinkwest	draw.io, Architecture. drawio
SC-20 Create ERD diagram	Model the database structure using an entity-relationship diagram.	2 days	SC-19	Patrick Shutte	draw.io / dbdiagram.io
SC-21 Create UML class diagram	Design the domain and application class diagrams representing system objects and relationships.	2 days	SC-20	Liam Swile	draw.io
SC-22 Create wireframes	Design low-fidelity mockups of the application's key user interface screens.	3 days	SC-21	Liam Swile	Figma
SC-23 Meeting	Team check-in to review progress on Part 1 deliverables before drafting the report.	0.5 day	SC-22	Whole team	MS Teams / Zoom
SC-24 Draft report	Compile the Part 1 research, diagrams and documentation into a first draft report.	2 days	SC-23	Uzair Essa	MS Word, project documentation
SC-25 Final report	Review, edit and finalise the Part 1 report ready for submission.	1 day	SC-24	Uzair Essa	MS Word
SC-27 Quality assurance	Proofread and check the final report and diagrams for errors and consistency before submission.	1 day	SC-25	Patrick Shutte	QA checklist
SC-26 Submit part 1	Submit the completed Part 1 POE deliverable via the LMS.	0.5 day	SC-27	Uzair Essa	LMS portal
POE Part 2 - Development (SCRUM-6)					

Task	Description	Duration	Predecessor(s)	Team Member(s) Responsible	Resources Needed
SC-28 Gather feedback	Collect client/lecturer feedback on the Part 1 submission to inform development.	1 day	SC-26	Uzair Essa	Feedback document
SC-34 Setup database	Provision and configure the SQL Server database and initial schema.	1 day	SC-28	Dillon Rinkwest	SQL Server, Azure/hosting account
SC-29 Begin backend coding	Start implementing the ASP.NET Core backend services and data access layer.	4 days	SC-34	Dillon Rinkwest, Patrick Shutte	Visual Studio, .NET SDK, GitHub repo
SC-30 Begin frontend coding	Start building the Blazor / MudBlazor frontend components and pages.	4 days	SC-29	Liam Swile, Uzair Essa	Visual Studio, MudBlazor, GitHub repo
SC-31 Group meeting	Team sync to review development progress and resolve blockers.	0.5 day	SC-30	Whole team	MS Teams / Zoom
SC-32 Implement APIs	Build REST API endpoints connecting the frontend to backend services.	3 days	SC-31	Dillon Rinkwest	Visual Studio, Postman
SC-37 Implement request feature	Develop the resource/staff request submission and tracking feature.	2 days	SC-32	Patrick Shutte	Visual Studio, GitHub repo
SC-33 Implement notification service	Build real-time notifications using Azure SignalR (Observer pattern).	2 days	SC-37	Dillon Rinkwest	Azure SignalR, Visual Studio
SC-35 Implement event manager	Develop the event planning and management module.	2 days	SC-33	Liam Swile	Visual Studio, GitHub repo
SC-36 Implement task manager feature	Build the daily task assignment and tracking (kanban-style) feature.	2 days	SC-35	Patrick Shutte, Liam Swile	Visual Studio, GitHub repo
SC-38 Compile code together	Merge all feature branches into the main branch and resolve integration conflicts.	1 day	SC-36	Liam Swile	Git / GitHub, Visual Studio

Task	Description	Duration	Predecessor(s)	Team Member(s) Responsible	Resources Needed
SC-45 Code the app version	Build the .NET MAUI Blazor Hybrid mobile version of the application.	2 days	SC-38	Dillon Rinkwest	.NET MAUI SDK, Visual Studio
SC-46 Review all code and app	Perform a full code review and test the web and mobile applications for bugs.	1 day	SC-45	Patrick Shutte	QA checklist, test devices
SC-48 Create video presentation	Record a demo video showcasing the application's features.	1 day	SC-46	Uzair Essa	Screen recording software, video editor
SC-47 Submit part 2	Submit the completed Part 2 POE deliverable via the LMS.	0.5 day	SC-48	Uzair Essa	LMS portal
POE Part 3 - Finalisation (SCRUM-7)					
SC-49 Create user guide	Write end-user documentation explaining how to use the application.	2 days	SC-47	Uzair Essa	MS Word, application screenshots
SC-50 Update code	Address feedback and fix outstanding bugs identified in Part 2.	2 days	SC-49	Dillon Rinkwest, Patrick Shutte	Visual Studio, GitHub repo
SC-51 Update documentation	Update technical and architecture documentation to reflect final code changes.	1 day	SC-50	Uzair Essa	MS Word
SC-53 Submit part 3	Submit the completed Part 3 POE deliverable via the LMS.	0.5 day	SC-51	Uzair Essa	LMS portal
SC-52 Present at expo	Present and demonstrate the finished application at the project expo.	1 day	SC-53	Whole team	Laptop, presentation slides, demo environment

Non-Functional Requirements

Availability and Reliability

The system must remain continuously operational and recover from deployments or failures without disrupting users [1].

Requirement	Detail
24/7 availability	The client requires continuous uptime. This ruled out the serverless Azure SQL tier, which auto-pauses when idle, in favour of a provisioned Standard S0/S1 tier and an “always on” Basic B1 App Service plan.
Zero-downtime releases	Releases are performed as blue-green slot swaps between a staging and a production App Service slot rather than redeploying over the live site. The new build is warmed and smoke-tested before it takes traffic, and a rollback is a swap back rather than a rebuild (sub-2-minute rollback).
Health monitoring	Application Insights runs a /health availability test against the deployed build, and Azure Monitor / App Service health alerts notify the team of unusual activity or outages.
Data durability/backups	Azure SQL automated backups run on a full-weekly, differential-daily, transaction-log-every-5–12-minutes schedule, included at no extra cost on the chosen service tier. A database export is also taken as a restore point before each production release.
No single point of session failure	The API is stateless (no in-process session state), so any request can be served by either deployment slot with the same result, supporting safe swaps and horizontal resilience.
Disaster recovery approach	Phase 1 uses one Azure region with no automated multi-region failover (a deliberate, documented limitation - a regional outage is communicated to the client and waited out). Recovery instead relies on point-in-time restore from the

	automated backup schedule plus a pre-release database export.

Performance and Scalability

The system must handle the current user base efficiently and have a clear path to scale as load grows [1].

Requirement	Detail
Rapid deployment	A merge to the main branch is the sole release trigger. From merge to live is a single CI/CD pipeline run of roughly 8–12 minutes, of which only the final approval gate is manual.
Vertical scalability	Azure SQL service tiers can be raised (e.g. S0 → S1 → higher DTU/vCore) with no code change as load grows; a DTU alert at 80% utilisation is the trigger to scale the database and App Service plan.
Read scalability for reporting	Azure SQL supports read replicas and active geo-replication, so management-reporting queries can be directed to a replica and kept off the transactional workload, using only a second connection string.
Efficient real-time updates	Live kanban-board updates and notifications are pushed to connected clients via Azure SignalR rather than being pulled from the database, so read volume scales with actual changes rather than with the number of open browser tabs or devices.
Unbounded data growth handling	Audit Log and history tables are append-only, time-ordered, and structurally separate from operational tables, so they can be partitioned by date and archived without touching the live schema.
Network performance baseline	A stable broadband connection (≥ 10 Mbps) is the assumed minimum for developers and end users; no specialised network hardware is required since Azure hosts and secures the traffic.
Query performance via indexing	Clustered indexes follow the primary keys. Non-clustered indexes are added on every foreign key (all are join targets) and on the four access paths the use

	cases exercise directly: User.email (login lookup), Task (the task board), Notification (the unread badge), and Audit Log (time-ranged audit queries).
Real-time connection capacity	The free SignalR tier caps at 20 concurrent connections against an expected 24 users - a documented capacity limit where the 21st connection is refused and that user silently loses live updates. Noted as needing the Standard tier (~$1.61/unit/month) if the organisation grows.

Security

The system must protect data in transit and at rest, restrict access to authorised users, and keep credentials out of source control [1].

Requirement	Detail
Encryption in transit	All traffic is HTTPS/TLS 1.2 minimum: App Service enforces HTTPS redirection, Azure SQL Database rejects unencrypted connections, and the .NET MAUI mobile app exposes no plain-HTTP endpoints.
Encryption at rest	Personal and application data is encrypted at rest using Azure SQL Transparent Data Encryption (TDE) and Blob Storage default encryption, enabled by default.
Network access control	The Azure SQL Database firewall accepts connections only from the App Service outbound IP range (plus temporary, removed-before-go-live developer IPs); the frontend never talks to the database directly, only through the API.
Authentication & authorisation	Every API route except /health and login requires a valid JWT; role and permission checks (Admin, Staff, Imaamat / project membership) run server-side on top of that, backed by SQL Row-Level Security and application roles.
Secrets management	Deployment credentials live in GitHub repository/environment secrets scoped to a protected environment; runtime secrets (DB connection string, Blob key, email key, JWT signing key) live in Azure

	Key Vault and are read via the App Service's system-assigned managed identity - no secret is ever committed to source control, and GitHub secret scanning is enabled.
Restricted-tier isolation	The database, file storage and secret store are not reachable by a URL alone: SQL accepts only App Service and named developer IPs, Blob containers are private with short-lived, single-blob-scoped SAS tokens, and Key Vault is opened only via managed identity.
Secure mobile credential storage	JWT bearer tokens on the .NET MAUI client are stored using SecureStorage, an encrypted key-value store backed by the platform's native secure storage (Android Keystore / iOS Keychain); tokens are never written to plain-text files.
Credential security	Passwords are never stored in plain text - ASP.NET Core Identity hashes them with PBKDF2 and a salt before storage. JWT tokens issued to the mobile client use short expiry times to limit the damage of a stolen token.
Injection-attack prevention	Entity Framework Core uses parameterised queries by default; no raw SQL is built by string concatenation anywhere in the codebase, closing off SQL injection as an attack path.
Object-level authorisation (anti-IDOR)	All data access goes through server-side authorisation checks that verify the requesting user's identity against the resource owner on every request, so a user cannot retrieve another user's records by altering an ID in the URL.
Cross-site scripting (XSS) protection	The frontend framework escapes user-supplied content before rendering it to the DOM, and Content-Security-Policy (CSP) headers are configured on the App Service to restrict which scripts can execute in the browser.
Denial-of-service resilience	Azure App Service's built-in rate limiting is the baseline, with Azure Front Door / API Management request-throttling and application-level rate-limiting middleware in the ASP.NET Core pipeline

	identified as add-on layers if the API comes under load-based attack.
Restricted cross-origin access	The ASP.NET Core Web API's CORS policy restricts cross-origin requests to the known frontend origin (the Static Web Apps URL), preventing unauthorised browser clients from calling the API.
Defence in depth	Security controls are layered independently at the network (TLS, Azure firewalls), application (RBAC, input validation, middleware pipeline) and database (Row-Level Security, TDE) tiers, so an attacker who bypasses one layer still faces the others.
Least privilege	Each role has access only to the functions and data it needs; API endpoints are locked to specific roles, and the database service account used by the API has read/write permissions only - never schema-owner or admin rights.
Fail securely	A failed authentication check returns a 401 Unauthorized with no data exposed, and error messages shown to users never reveal internal stack traces or database details.
Secure by default	TLS is enforced by default on App Service, TDE is enabled by default on Azure SQL, and ASP.NET Core Identity hashes passwords with no extra configuration - a developer must actively turn security off rather than turn it on.
Minimal data exposure over the wire	Clients never receive raw EF entities. A shared DTO layer (with validation attributes) is the only wire contract for both the web and mobile clients, so internal fields can't leak and a contract change breaks the build rather than surfacing silently in production.
Consistent security enforcement across endpoints	HTTPS redirection, CORS, JWT authentication, role/permission authorisation, audit capture and the global exception handler are applied as a middleware chain to every request, including endpoints written after the pipeline was set up - so a control can't be

	silently missed on a single controller action.

Data Protection and Regulatory Compliance (POPIA)

The system stores personal information about South African residents and must comply with the Protection of Personal Information Act (Act 4 of 2013) from go-live.

Requirement	Detail
Data minimisation	Only information genuinely needed to operate the system is collected - e.g. a staff member's name, email and assigned role - with no collection of ID numbers, physical addresses or other unnecessary personal data.
Data residency	All personal data is stored in the Azure South Africa North region, keeping it within South African borders and legal jurisdiction as POPIA requires.
Consent, documentation & deletion	Users are informed what data is collected and why before capture; the system supports removing a user's records on withdrawal of consent or departure from the organisation, and the NPO is responsible for maintaining consent records.
Breach detection & response	Azure Monitor / App Service health alerts flag unusual activity so a suspected breach can be identified and, per POPIA, reported to the Information Regulator and affected parties within a reasonable time; the NPO should have a documented incident-response procedure before go-live.

Data Integrity and Consistency

The database must prevent invalid states, protect historical records, and keep business rules enforceable regardless of which client sends the request [1].

Requirement	Detail
Referential integrity rules	Delete behaviour is set per relationship rather than defaulted: cascade for dependants that are meaningless without

	their parent (e.g. Task → Task_Comment), NO ACTION on every foreign key referencing User to protect the audit trail and SET NULL for optional links such as a file's task/meeting reference.
Uniqueness & check constraints	Database-level unique constraints stop a user being assigned or enrolled twice (Task_Assignment, Project_Member) and enforce a unique login email; check constraints enforce rules such as a file having at most one attachment context.
Immutable, append-only history	Task_Status_History and Request_Status_History are append-only logs rather than overwritten status columns, so no state change can be silently lost, and the client can always answer “how long did this sit, and who moved it.”
No hard deletion of users	User.isActive supports deactivating a departed member instead of deleting the row, since 22 relationships point at User; foreign keys on User use ON DELETE NO ACTION and the application never hard-deletes a user, preserving history and the audit trail.
Server-side enforcement of valid state transitions	Task, Resource Request and User lifecycles are implemented as guarded transition tables checked inside the owning service on the server, not just in the UI - a hidden button on the client is access control, and a client that stops sending an illegal transition does not stop an attacker sending it directly.
Booking / scheduling integrity	Double-booking or scheduling conflicts are prevented with server-side validation on all bookings plus unique-constraint enforcement at the database level, not client-side checks alone.

Usability and Support

The system must be learnable by non-technical NPO staff and volunteers and must reliably keep them informed [1].

Requirement	Detail
User onboarding materials	A user manual is created and distributed before go-live specifically to reduce the friction, delays and miscommunication risk of staff and volunteers transitioning from the previous system.
Reliable notification delivery	Staff missing notifications or updates (a risk carried over from the previous system) is mitigated with an automated email notification system (Resend API) and Azure Notification Hubs push alerts on the mobile app, so updates reach users without them needing to check manually.
Usability sign-off before release	A manual test script is run against the staging environment before every release pull request into main, specifically for usability and client sign-off on each use case - covering the areas automated tests are poor at judging.

Auditability and Traceability

Every action on data, and every change to the deployed system, must be traceable to a person and a point in time.

Requirement	Detail
Full audit trail	An Audit_Log table, with foreign keys guaranteeing every entry resolves to a real user, records changes across the system; status-history tables (e.g. Task_Status_History, Request_Status_History) keep a full append-only sequence of state changes and who made each one.
Deployment traceability	Every production build traces back to a merge commit on main, together with its pull request, its review, and its pipeline run, all retained in GitHub; the full source, including DBMS code, lives in the repository.

Consistent audit capture on every request	Audit capture sits in the ASP.NET Core middleware pipeline alongside authentication and authorisation, so it is applied to every endpoint - including ones written after the pipeline was set up - rather than being an authorisation-style check that a developer can forget in a single controller.

Maintainability and Deployability

The codebase and release process must support ongoing change with confidence and without manual, error-prone steps [1].

Requirement	Detail
Automated quality gates	A merge to a protected branch is blocked unless the code compiles with zero warnings (TreatWarningsAsErrors), all automated tests pass, a coverage threshold (Coverlet) is met, at least one reviewer has approved, and required status checks are green.
Clean layered architecture	The API is organised into presentation, application/domain, and data-access layers (controllers → services → EF repositories), separating concerns that vary independently and keeping business rules out of the presentation layer.
Safe, reversible migrations	Database schema changes are applied through EF Core migrations as a dedicated stage of the deployment pipeline and are exercised on every pipeline run before a release is allowed to proceed.
Single source of business logic	The web and mobile clients both consume the same ASP.NET Core Web API and the same NPO. Shared DTOs, so there is one schema and one set of business rules rather than duplicated logic per client.
Automated test coverage across layers	Unit tests (xUnit/FluentAssertions/Moq) cover business rules in isolation; integration tests exercise controllers end-to-end against a disposable test database; smoke tests confirm the deployed slot starts and answers an

	authenticated request - all three run on every push and pull request or right after deployment.
Configuration over code changes	Every dependency is constructor-injected and registered centrally; connection strings and the JWT signing key come from Key Vault in production and from .NET user secrets locally, so switching environments is a registration/configuration change, not a code change.

Portability and Multi-Client Support

The application must work consistently across the web and mobile platforms the client uses, including offline [1].

Requirement	Detail
Cross-platform mobile client	.NET MAUI produces a single C# codebase that compiles to both Android and iOS, consuming the same REST API as the web client with no backend changes required.
Offline capability with sync	A local SQLite cache lets staff view tasks and records without a network connection; write operations attempted offline are queued and synced automatically once connectivity returns.
Consistent real-time behaviour across clients	Azure SignalR Service's first-party .NET MAUI client SDK gives the mobile app identical live kanban updates and in-app notifications to the web frontend, with no separate real-time service.
Minimum supported platform versions	Mobile testing targets Android 10+ or iOS 15+ smartphones, setting the minimum OS versions the app is expected to run on.

Note: this list consolidates non-functional requirements stated or clearly implied. It does not include purely financial/budgetary constraints (covered in the Financial Feasibility document) or functional (use-case level) requirements.

Domain Class Diagram

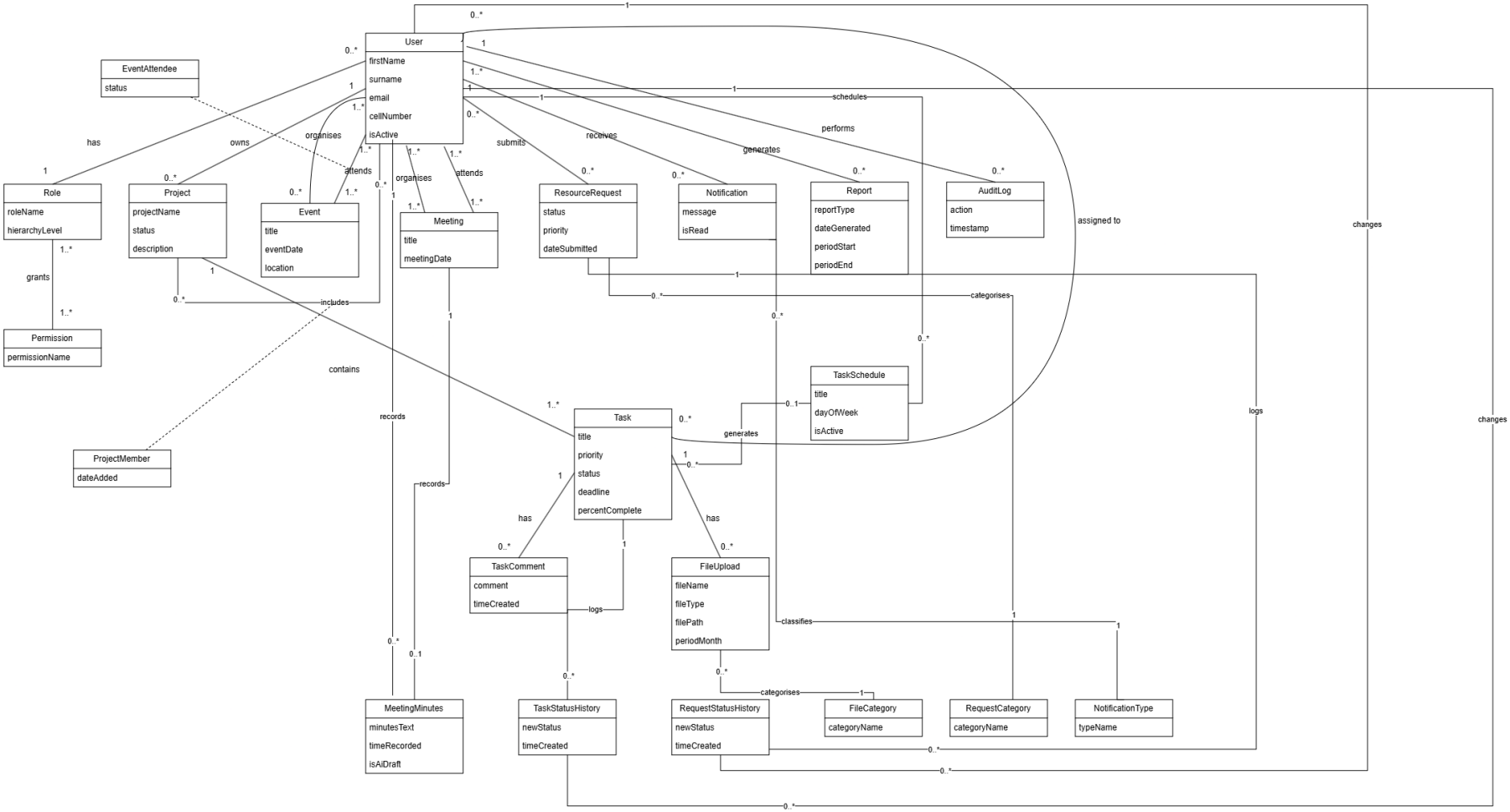


Figure 8: QULOOBUL MOE'MIENEEN Domain Class Diagram

Explanations

- **User**: Any person with an account in the same system (staff, managers, admins). Holds identity and contact details plus an active/inactive flag and is the actor behind almost every other class. Owning projects, submitting requests, completing tasks, and receiving notifications.
- **Role**: A named job function with a hierarchy level used to rank authority. Each user has one role.
- **Permission**: A single named action a user is allowed to perform. Roles grant permissions, so access control is defined once per role rather than per user.
- **Project**: A body of work with a name, status, and description, owned by one user. Acts as the container that tasks, events, and members are grouped under.
- **ProjectMember**: The association between a user and a project, recording when they joined. Exists as its own class because membership carries data beyond the plain link.
- **Event**: A scheduled occurrence tied to a project (data, title, location). Organised by one user and attended by many.
- **EventAttendee**: The link between a user and an event, holding that person's attendance status.
- **Meeting**: A scheduled gathering with a title and date, organised by one user and attended by many. Separate from Event because meetings produce minutes and follow a different cycle.
- **MeetingMinutes**: The written record of a meeting, with the minutes text, when it was recorded, and a flag marking whether it is still an AI-generated draft waiting for human approval.
- **Task**: The core unit of work: title, priority, status, deadline, and percent complete. Assigned to a user, belongs to a project, and may be broken into sub-tasks via a self-relationship.
- **TaskComment**: A dated note left by a user on a task, giving a discussion thread per task without altering the task record itself.
- **TaskStatusHistory**: An immutable log of every status change on a task, recording the new status, when it changed, and who changed it. Preserves history that the task's single current field cannot.
- **TaskSchedule**: A recurring template that automatically generates tasks on a given day of the week. The isActive flag lets a recurrence be paused without deleting it.
- **ResourceRequest**: A request submitted by a user for a resource, with a status, priority, and submission date. Routed for approval and tracked to resolution.
- **RequestCategory**: A classification for resource requests (e.g., equipment, facilities) used for grouping and reporting.

- **RequestStatusHistory**: An audit trail of each status change on a resource request, recording the new status, the timestamp, and the user responsible.
- **FileUpload**: A document or attachment linked to a task, storing its name, type, storage path, and the reporting month it belongs to.
- **FileCategory**: A classification for uploaded files, used to group documents by type or purpose.
- **Notification**: A message delivered to a user with a read/unread flag so the system can surface what still needs attention.
- **NotificationType**: The category of a notification (e.g., task assigned, request approved, deadline due), allowing consistent formatting and user-level preferences per type.
- **Report**: A generated summary covering a defined period, recording the report type, when it was produced, and the period start and end dates. Generated by a user and built from task, request, and file data.
- **AuditLog**: A system-wide record of significant actions, storing what was done, when, and by which user. Provides accountability and traceability across the whole system.

Design Class Diagram

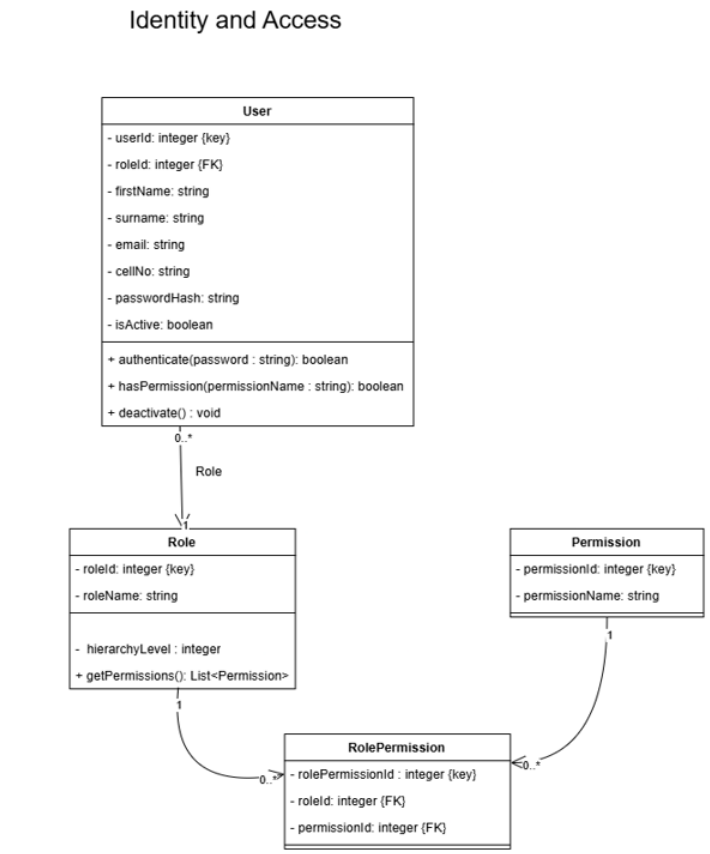


Figure 9: Design Class Diagram (Identity and Access)

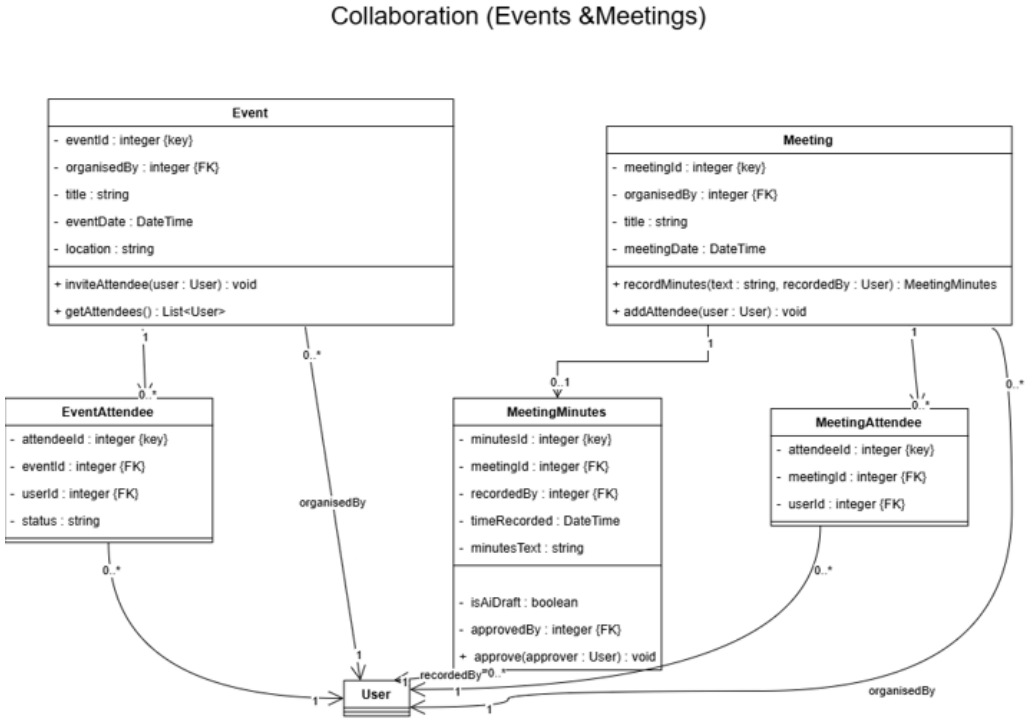


Figure 10: Design Class Diagram (Collaboration- Events & Meetings)

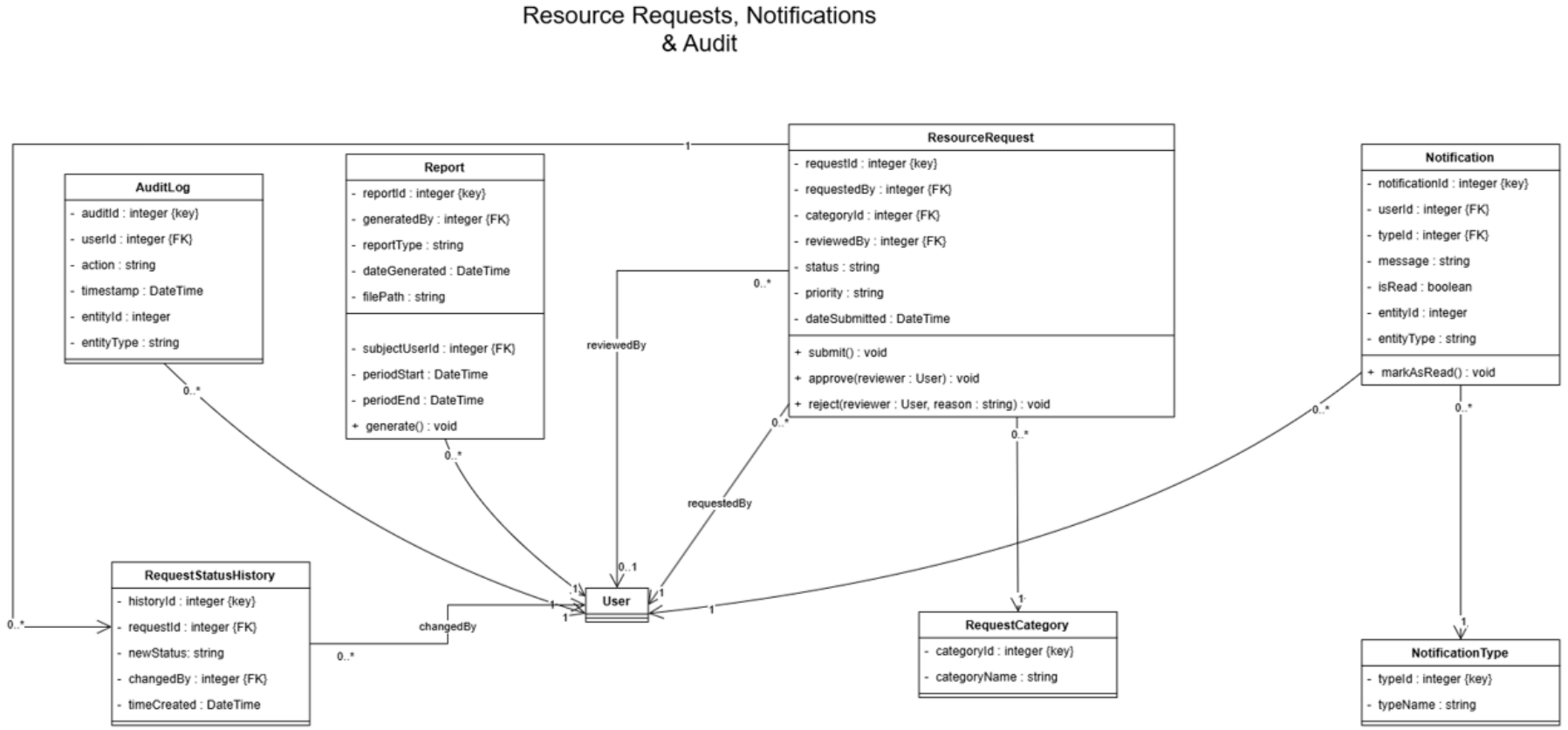


Figure 11: Design Class Diagram (Resource Request, Notifications & Audit)

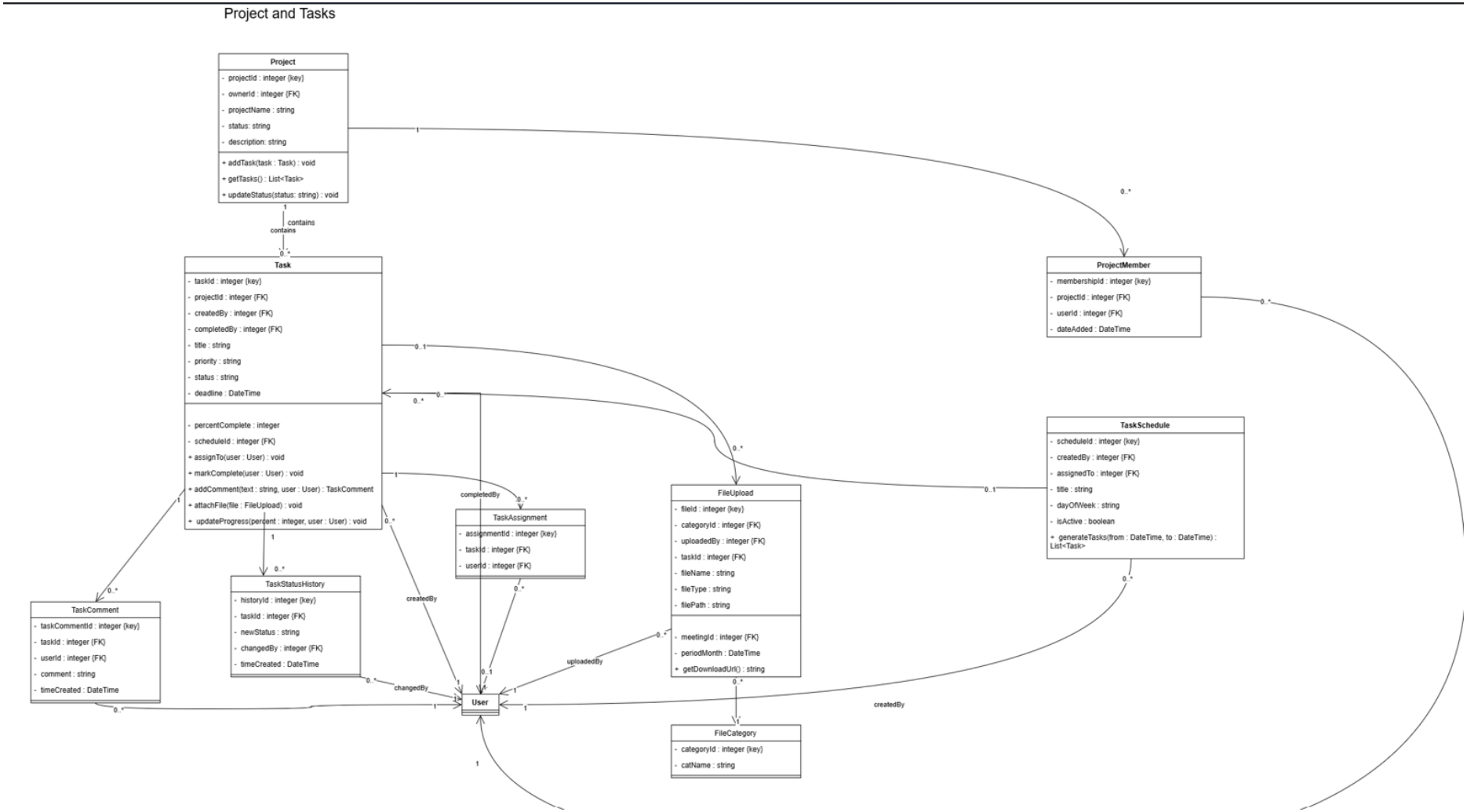


Figure 12: Design Class Diagram (Projects & Tasks)

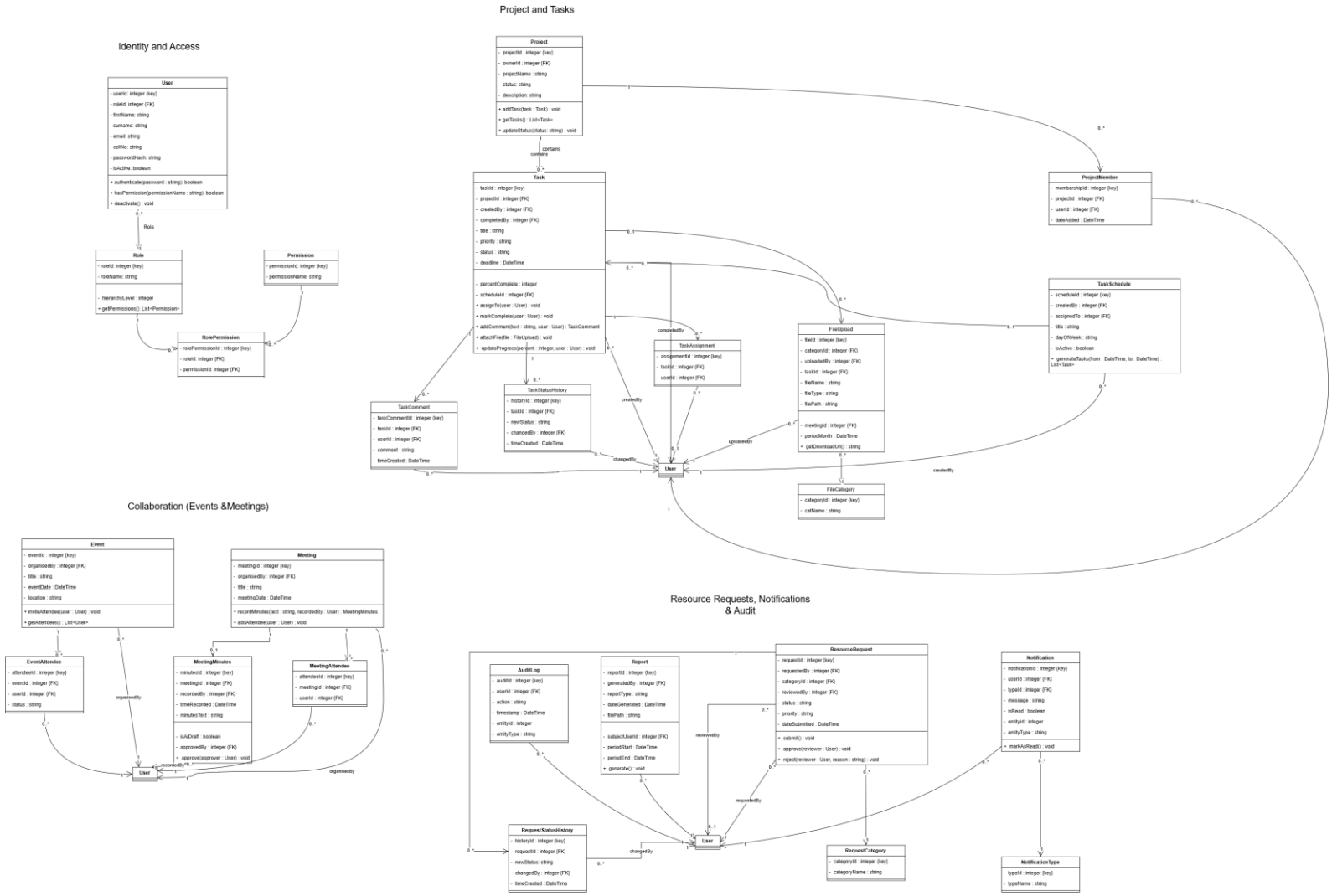


Figure 13: Full Design Class Diagram for QULOOBUL MOE'MIENEEN

Implementation

Sequence Diagrams

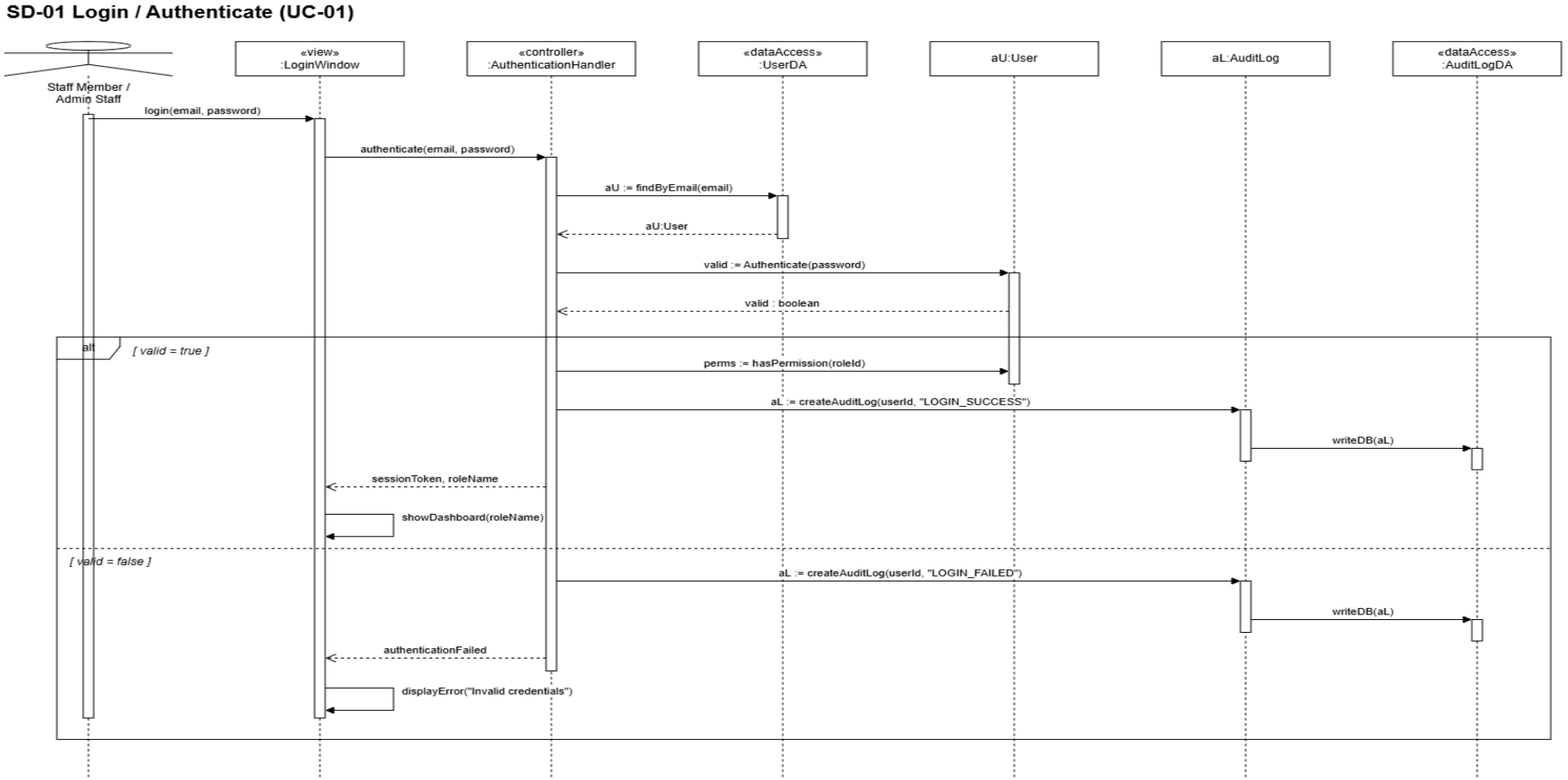


Figure 14: SD-01 Login/Authentication (UC-01)

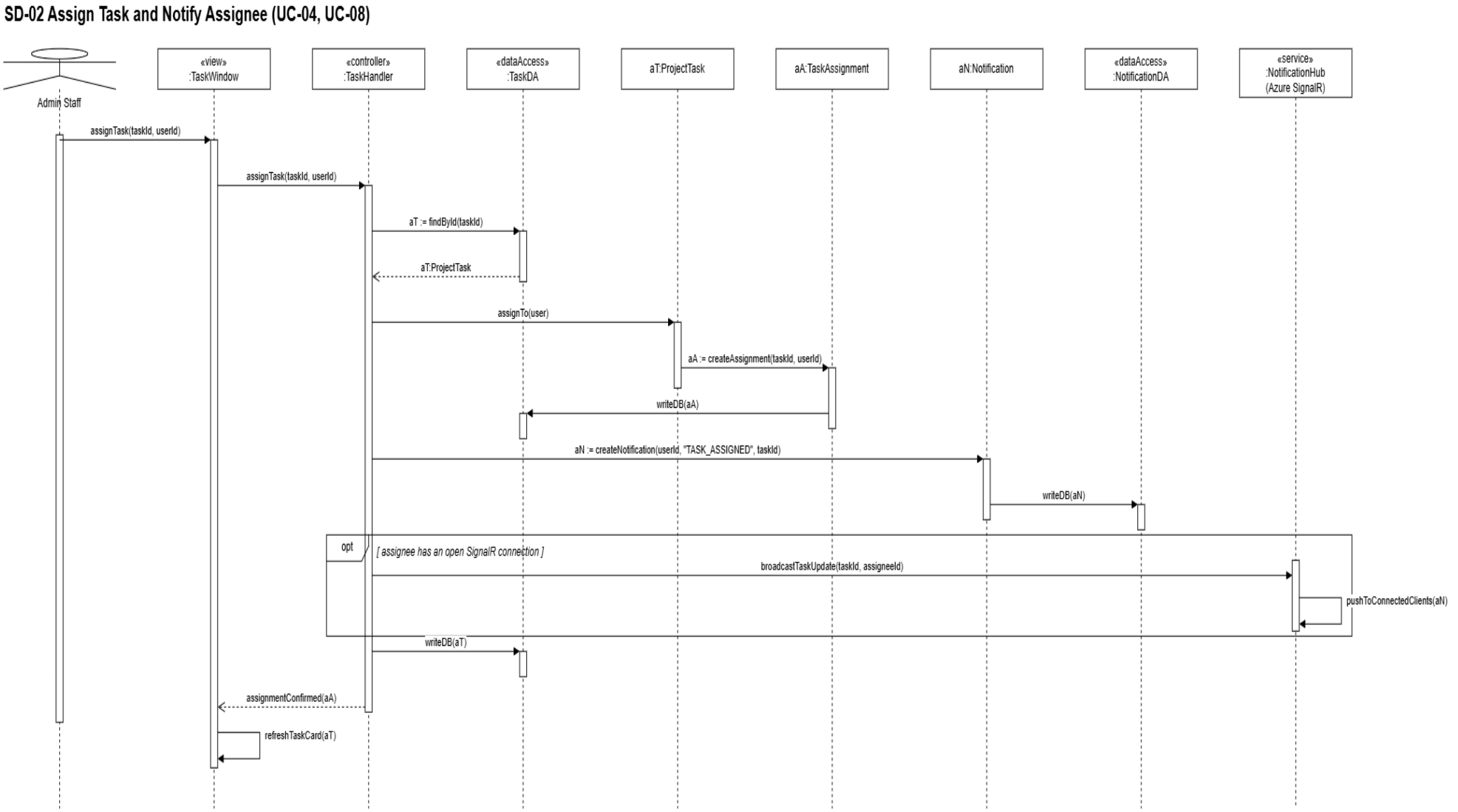


Figure 15: SD-02 Assign Task and Notify Assignee (UC-04, UC-08)

SD-03 Submit and Review Resource Request (UC-05)

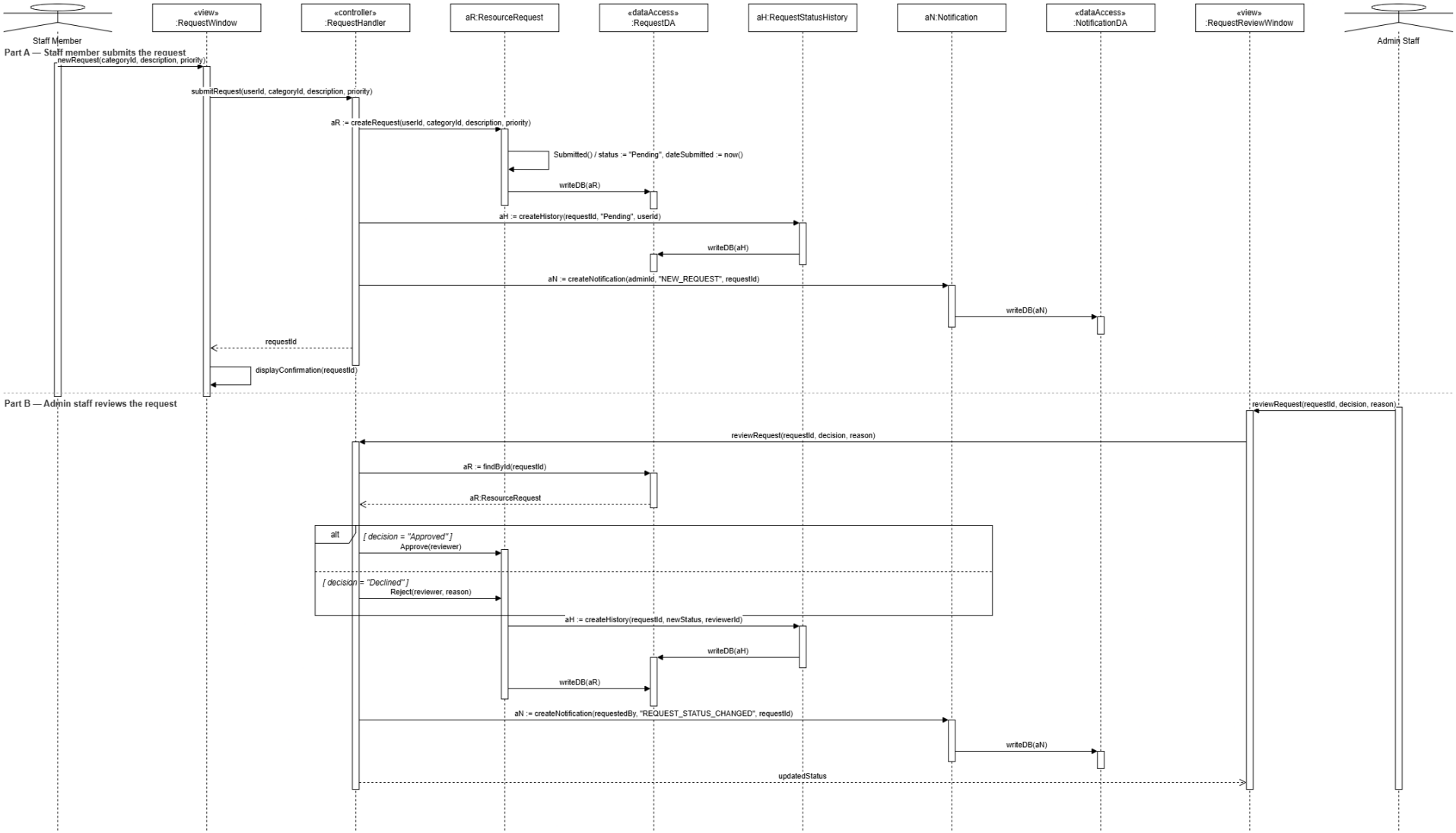


Figure 16: SD-03 Submit and Review Resource Request (UC-05)

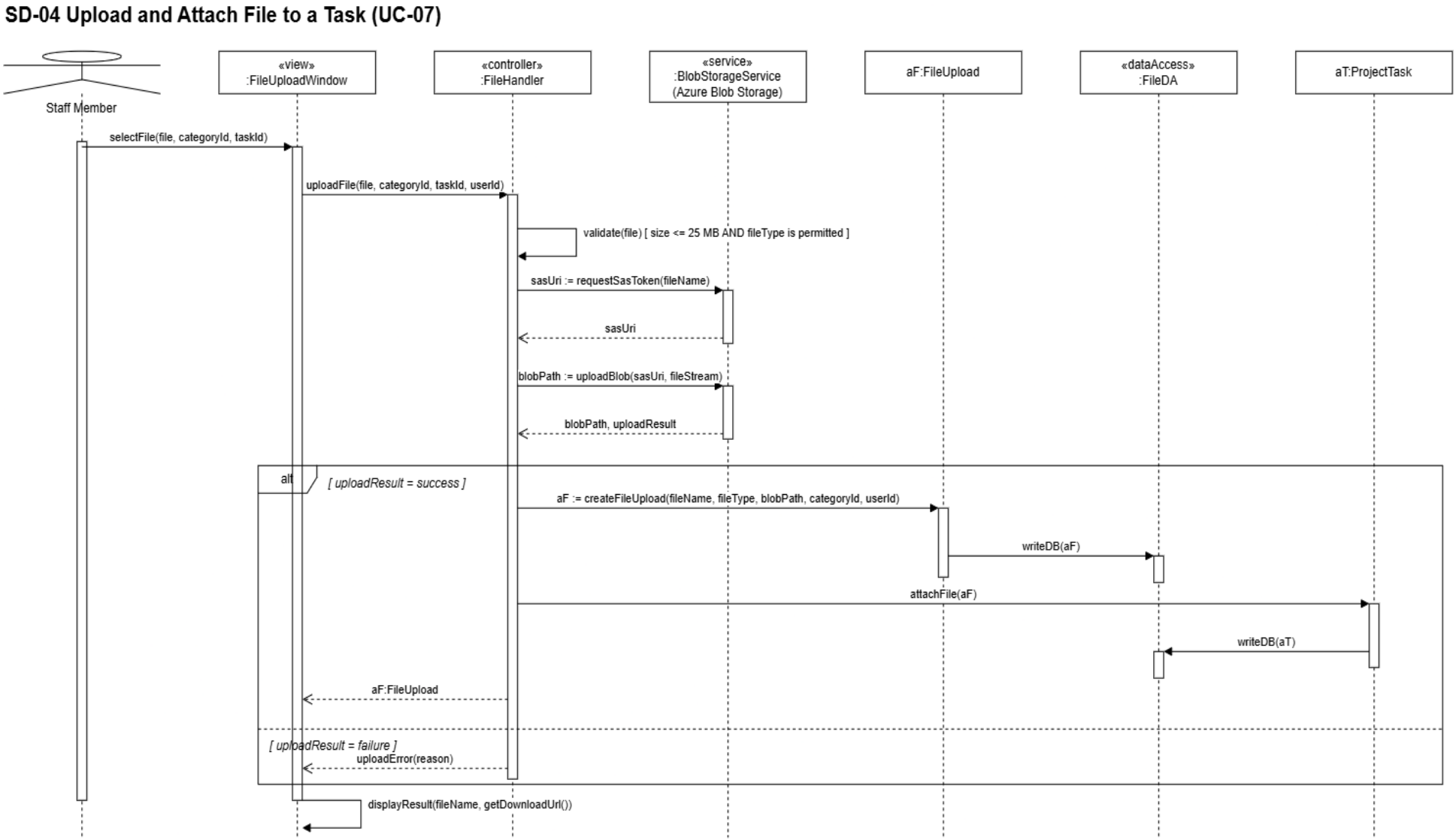
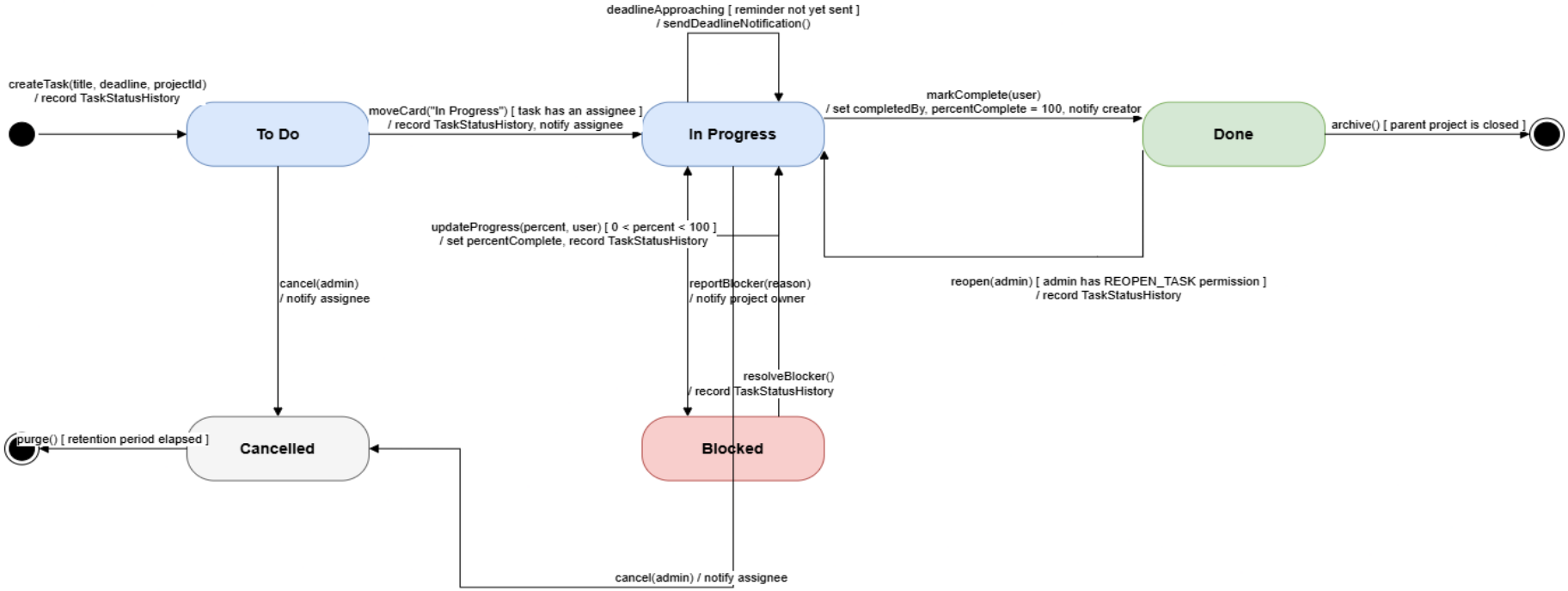


Figure 17: SD-04 Upload and Attach File to a Task (UC-07)

State Diagrams

SM-01 State Machine Diagram — ProjectTask (UC-02, UC-04)



Transition syntax (Satzinger, Ch.4): transition-name(parameters) [guard-condition] / action-expression

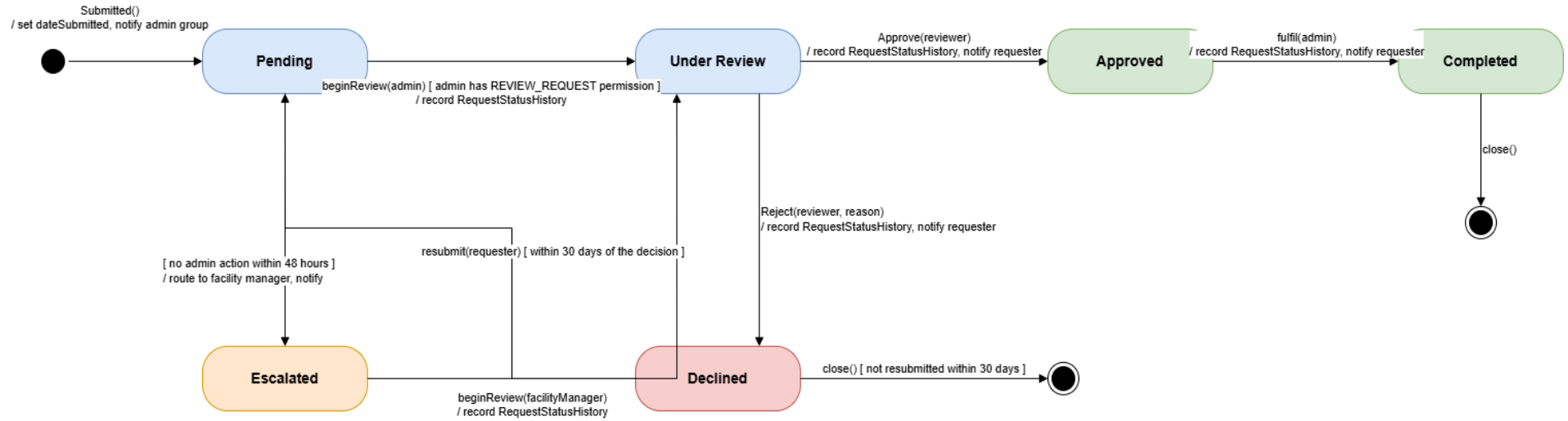
Why this class needs a state machine: ProjectTask is the only class in the kanban flow whose status controls what the system will allow next — a task cannot be completed before it is started, and it cannot be started before it has an assignee. Every transition shown writes a TaskStatusHistory row, which is what makes the completion-rate reporting in UC-09 possible.

Why In Progress carries a percentage: the client asked for progress to be reported as a percentage rather than a yes/no, because a task can be substantially advanced without being finished — three quotes obtained out of four, or a contractor who has measured but not yet installed. percentComplete is an attribute of the In Progress state, not a state of its own: 10% and 70% permit exactly the same transitions, so adding states for them would express nothing the guard does not already. markComplete() sets it to 100.

Mapping to the design class diagram: the state name is stored in ProjectTask.status and the percentage in ProjectTask.percentComplete; the transitions correspond to assignTo(), updateProgress(), markComplete() and the board's moveCard action.

Figure 18: SM-01 ProjectTask (UC-02, UC-04)

SM-02 State Machine Diagram — ResourceRequest (UC-05)



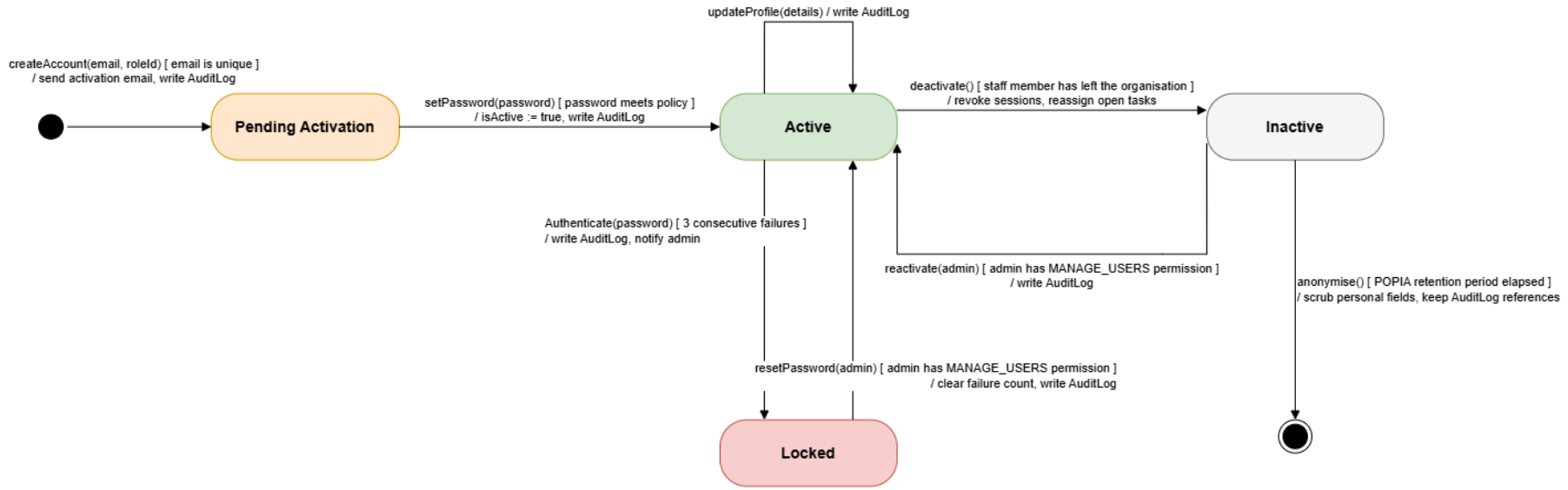
Transition syntax (Satzinger, Ch.4): transition-name(parameters) [guard-condition] / action-expression

Why this class needs a state machine: the whole point of UC-05 is that a request is never lost or misdirected. The status is what the requester tracks and what the admin acts on, so the legal transitions between statuses are a business rule, not an implementation detail.

Note on the escalation transition: Pending → Escalated has a guard but no triggering event. Per Satzinger, the trigger is then treated as constantly firing, so the transition occurs as soon as the guard becomes true. In implementation this is a scheduled background job on the API, not a user action.

Figure 19: SM Resource Request (UC-05)

SM-03 State Machine Diagram — User (UC-01, UC-06)



Transition syntax (Satzinger, Ch.4): transition-name(parameters) [guard-condition] / action-expression

Why this class needs a state machine: User.isActive is a single boolean in the design class diagram, but the access rules behind it are not. An account awaiting activation, a locked account and a deactivated account are all "not active" and must be treated differently at login. This diagram is the specification for those rules.

Note on deactivation: the action-expression reassigns open tasks. Deleting a user is never a valid transition - task history, meeting minutes and audit records reference the account, and POPIA is satisfied by anonymisation rather than deletion.

Figure 20: SM-03 User (UC-01, UC-06)

Deployment Diagram

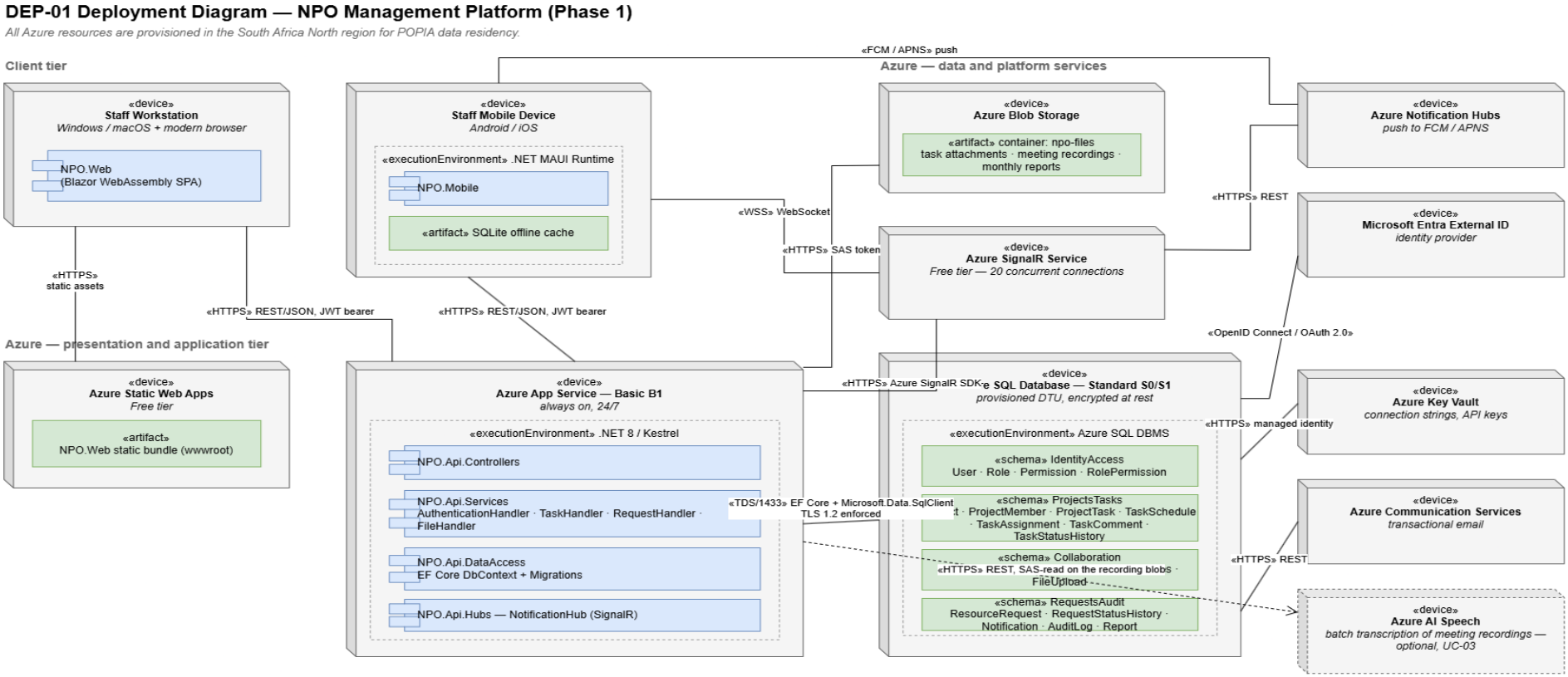


Figure 21: DEP-01 Deployment Diagram

Data Schema

Choice of Technology

The platform stores its data in Azure SQL Database; a fully managed relational database delivered as a Paas (Platform- as a Service) offering[2]. The schema is written and version-controlled by the team as Entity Framework Core 10 entity classes and migrations, consumed by an ASP.NET Core (.NET 10) Web API[3]. Binary content (task attachments, meeting recordings, generated report PDFs) is held in Azure Blob Storage, with only the metadata and the blob path stored as rows in the relational database[4].

This is a two-store design, and the split is deliberate. Structured, relational, queryable data lives in SQL. Unclear binary content lives in object storage. Neither store is used for the job the other does better.

Why relational database

- **The data is densely relational**. The schema contains 26 entities joined by 41 foreign key relationships. Almost every query the application performs is a join: “the tasks in this project, assigned to this member, with their comment count and current status”, “every resource request in this category awaiting review, with who submitted it”[5]. In a relational engine, these are single set-based statements the query optimiser handles. In a document store, the same questions require either denormalised duplication of user and project data into every document, or application-side joins that move the work out of the database and into C#[6].
- **Referential integrity is a requirement**. The organisation is a non-profit that must be able to demonstrate an intact audit trail. A Task_Status_History row that points at a deleted task, or an Audit_Log row whose user cannot be resolved, is not only untidy data, but it also destroys the value of the log. Foreign-key constraints enforced by the database engine guarantee this at the storage layer, where no application bug or future developer can bypass them[6]. A document store places that guarantee in application code, which is exactly the layer most likely to change.
- **Reporting is combined and set-based**. The management-reporting module (Report) produces per-person and per-period summaries: tasks completed per member per month, average time in each status, request turnaround by category. These are GROUP BY and window-function workloads[6].
- **The schema is known and stable**. This is an internal administrative system for a known organisation with a fixed set of roles, task states, and workflows, elicited from the client before design began. The principal argument for a schemaless store, that the shape of the data is not yet known and varies per record, does not

hold here. Where the requirements did leave genuine flexibility, it was handled with two nullable columns rather than by abandoning the relational model for the entire system.

Why Azure SQL Database

1. **Full ownership of the DBMS code**. The schema, constraints, indexes, views, and any stored procedures are authored by the team as EF Core migrations and T-SQL, committed to the repository. Therefore, submittable and markable in full. There is no generated middleware layer whose behaviour the team cannot account for.
2. **Toolchain alignment**. EF Core's SQL Server provider is the reference implementation and the best-supported path from the UML Class model to code[3]. The domain classes in the/models folder are the forward-engineered form of the ERD, and the migration files are the forward-engineered form of the database. The traceability from the diagram to the entity class to the physical table is direct and demonstrable.
3. **Funding fit**. The Microsoft for Nonprofits grant provides up to $2 000 per year in Azure credits[7]. Consolidating the database, API hosting, blob storage, and real-time service into one Azure subscription means the entire $45-90 per month runs against that single credit, with no out-of-pocket cost to the organisation.
4. **Operational burden the organisation can carry**. Automated backups with point-in-time restore, patching, high availability, and threat detection are managed by the platform. The client is a non-profit with no IT staff. A self-administered database server was never a realistic option, whatever its licence cost[8].

Support for the project requirements

Requirement	How the storage technology supports it
Role-based login hierarchy (UC-01)	`Role`, `Permission`, `Role_Permission`, and `Role.hierarchy_level` are relational lookups resolved in one join at authentication. Permissions change by editing rows, not by redeploying code.
Task assignment and tracking (UC-02)	`Task_Assignment` as a resolved many-to-many, with `Task_Status_History` giving an append-only movement log per task.
Project-scoped visibility	`Project_Member` lets every read be filtered by membership in a single predicate, so a member allocated to one project cannot see another.
Resource requests and approval (UC-05)	`Resource_Request` plus `Request_Status_History` records not only the current state but the full sequence of state changes and who made each.
Meeting minutes, including AI drafts	`Meeting_Minutes.is_ai_draft` and the nullable `approved_by` make "drafted by AI, not yet approved by a person" a state the database itself can represent and query.
File uploads (UC-07)	Blob Storage holds the bytes; `File_Upload` holds the searchable metadata and the path. Files stay queryable by uploader, category, task, meeting, or reporting period.
Management reporting	`Report` plus `GROUP BY` queries over the history tables. No pre-aggregation pipeline needed.
Full audit trail	`Audit_Log`, with foreign keys guaranteeing every entry resolves to a real user.
Two clients (web + .NET MAUI)	Both consume the same Web API over the same database. There is one schema and one set of business rules, not two.

Figure 22: Support for the project requirements table

Future scalability

- **Vertical scale**. Azure SQL tiers are changed with a slider and no code change. The project starts on a Standard S0/S1 provisioned tier appropriate to 24 users and can move up the DTU or vCore range as load grows[9]. The provisioned tier was chosen over serverless specifically because the client requires continuous uptime, and serverless per-second billing exceeds provisioned hourly billing at high utilisation. If the usage pattern ever becomes intermittent, the serverless tier becomes the cheaper option and is a configuration change away[10].
- **Read scale**. Reporting is the workload most likely to grow disproportionately, as management asks for longer periods and more cross-cutting summaries. Azure SQL supports replicas and active geo-replication, so report generation can be directed to a replica and kept away from the transactional workload[11],[12]. The application needs only a second connection string to take advantage of this.
- **Data growth**. The two tables that grow without bound are Audit_Log and the two history tables. These are append-only and time-ordered, which makes them natural candidates for partitioning by date and for an archival policy that moves rows older than a retention threshold to cheaper storage[6]. Because they are separate tables rather than columns on Task and Resource_Request, this can be done later without touching the operational schema. The current-value columns (Task.status, Task.percent_complete) mean everyday screens never read the history tables at all.
- **Load and concurrency**. Real-time task-board updates are served by SignalR pushing change notifications to connected clients, rather than clients polling the database[13]. This keeps read volume proportional to actual changes rather than the number of browser tabs, which matters as the user count rises.
- **Adding a document store later**. Nothing in this design prevents introducing Cosmos DB or a similar store for a genuinely document-shaped future feature, such as storing raw meeting-transcript payloads from the speech-to-text service. The relational core would continue to own the entities and relationships; the document store would hold the unstructured payload and be referenced by path, exactly as Blob Storage is now. Choosing a relational now does not close that door.

Entity Relationship Diagram (ERD)

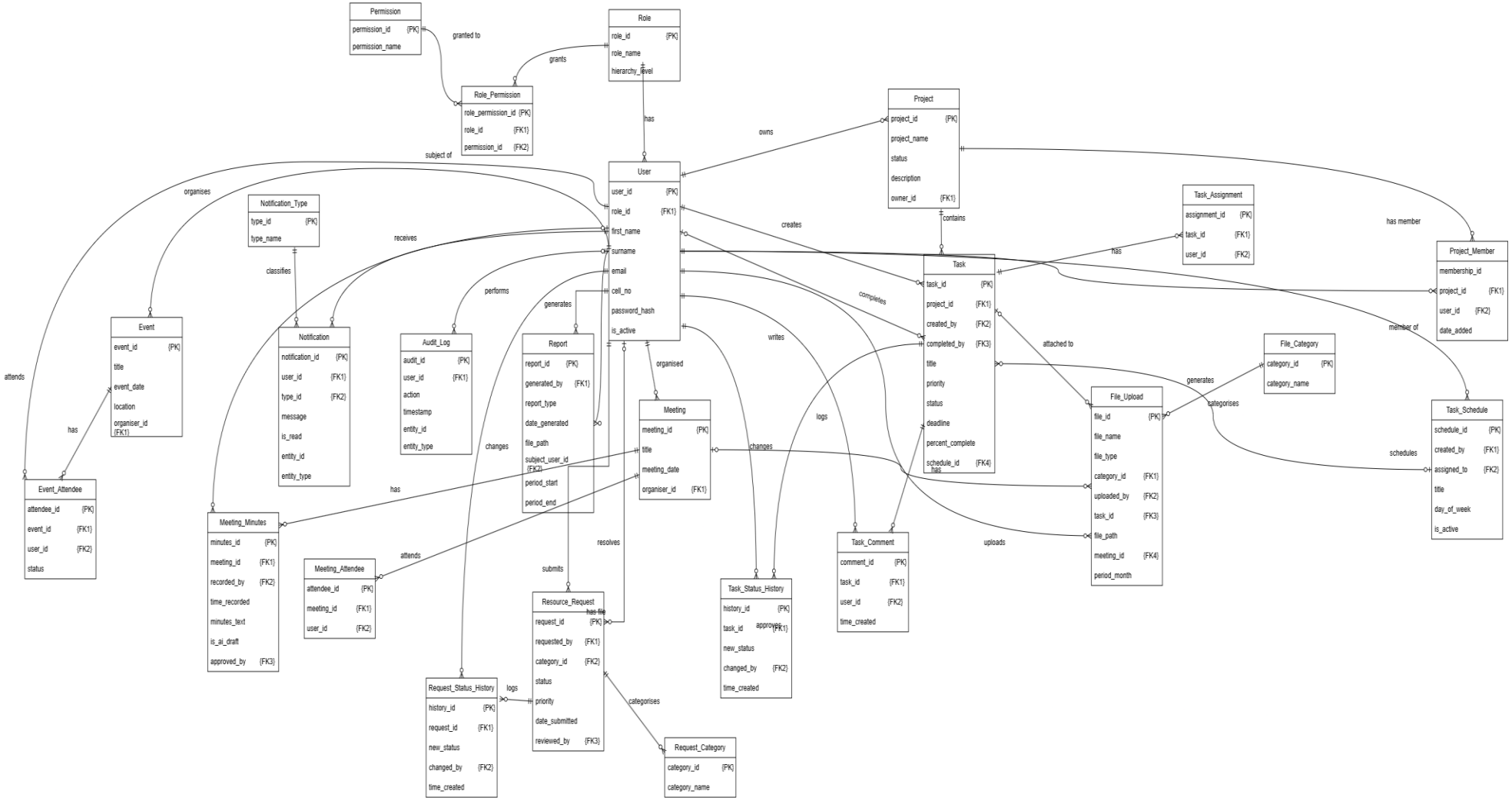


Figure 23: QULOOBUL _MOE'MIENEEN ERD

Notation and conventions

The diagram is a logical data model drawn in draw.io using crow's-foot notation[14]. A single bar at the parent reads "exactly one"; a crow's foot at the child end reads "zero to many". Primary keys are marked {PK}, foreign keys {FK} with an ordinal where an entity holds more than one. Every relationship carries a verb phrase from parent to child.

Five conventions applied

- **Surrogate integer primary keys**. Every entity is identified by a single auto-incrementing integer rather than a composite or natural key[6]. This keeps the foreign keys narrow, keeps join predicates single-column, and means a change to a business value (a renamed role, a corrected email) never cascades into child rows.
- **Junction tables carry their own surrogate key**. Task_Assignment, Project_Member, Meeting_Attendee, and Role_Permission resolve many-to-many relationships, and each has its own {PK} in addition to the two foreign keys, with a unique constraint on the pair preventing duplicates. This allows the junction to be referenced by other records later if the requirements demand it and keeps the EF Core mapping uniform[6].
- **Nullable foreign keys express genuine optionality**. Task.completed_by is null until someone completes the task. Meeting_Minutes.approved_by is null until the minutes are approved. Resource_Request.reviewed_by is null while the request is pending. The absence of a value is a meaningful state, not missing data.
- **History is append-only and separate**. State changes are recorded as new rows in a dedicated history table rather than by overwriting a column, so the record of what happened cannot be lost by a subsequent update.
- **Snake_case in the database, PascalCase in C#**. The physical names in the diagram match the table and columns produced by the EF Core migrations; the entity classes map to them via [Table] and [ForeignKey] attributes[3].

Subject areas

#	Subject area	Entities	Purpose
1	Identity and access	`Role, Permission, Role_Permission, User`	Who exists, what role they hold, what that role may do.
2	Projects and tasks	`Project, Project_Member, Task, Task_Assignment, Task_Comment, Task_Status_History, Task_Schedule`	The core work-management domain.
3	Meetings and events	`Meeting, Meeting_Attendee, Meeting_Minutes, Event, Event_Attendee`	Scheduling, attendance, and the minute of record.
4	Resource requests	`Request_Category, Resource_Request, Request_Status_History`	The request-and-approval workflow.
5	Files and reporting	`File_Category, File_Upload, Report`	Document metadata and generated management reports.
6	Notifications and audit	`Notification_Type, Notification, Audit_Log`	Delivery of alerts and the accountability trail.

Figure 24: Subject Areas

Identity and access

- **Role**: the organisational positions. hierarchy_level is an integer where a lower number means more authority. It exists so the rule "a task may be assigned downwards, never upward" can be evaluated by comparing two integers, rather than by hard-coding a list of role names in the application.
- **Permission**: an individual capability, such as approving requests or generating a report.
- **Role_Permission**: resolves the many-to-many between roles and permissions. Permissions are granted to roles rather than to users; adding a capability to every treasurer is one row, not one row per treasurer.
- **User**: a person with an account. password_hash stores a hash and never a password. is _active allows departed members to be deactivated instead of deleted, which preserves every historical row that references them; deletion would either break the audit trail or force loss of history[15],[16].

Projects and tasks

- **Project**: a body of work with an owner and a status.
- **Project_Member**: the users allocated to a project. This is what makes project-scoped visibility enforceable: a member only sees the projects they belong to, applied as one predicate on every read.
- **Task**: the central work item. It carries percent_complete in addition to status, because the client asked to see partial progress rather than only done or not done. schedule_id is populated when the task was generated from a recurring template and null when it was created.
- **Task_Assignment**: resolves many-to-many between tasks and users, because a task can be shared by several people and a person holds many tasks.
- **Task_Comment**: threaded discussion against a task, preserving who said what and when.
- **Task_Status_History**: one row per status change, giving both the audit trail and the raw data for cycle-time reporting.
- **Task_Schedule**: a recurring weekly template, dictating from the Task instances generated from it, so that editing the template does not rewrite history and deactivating it does not delete past work.

Meetings and events

- **Meeting**: a scheduled internal meeting with an organiser.
- **Meeting_Attendee**: resolves attendance as many-to-many.
- **Meeting_Minutes**: the minutes of record. is_ai_draft marks minutes produced by the AI summarisation feature, and approved_by stays null until a person signs them off. Together they make the governance rule structural: an unapproved AI draft is a different queryable state, not something someone has to remember.
- **Event**: an outward-facing event, distinguished from a meeting by having a location and different attendance semantics.
- **Event_Attendee**: attendance with a status attribute, which is the reason this junction is not interchangeable with Meeting_Attendee.

Resource requests

- **Request_Category**: a lookup table for request types, so categories are administered as data rather than as a code change.
- **Resource_Request**: a member's request for resources, with status, priority, and a nullable reviewed_by.
- **Request_Status_History**: one row per status transition, mirroring Task_Status_History so that the turnaround time is derivable.

Files and reporting

- **File_Category**: a lookup for document types.
- **File_Upload**: metadata for a stored file: name, type, and the Blob Storage path. task_id and meeting_id are both nullable because a file may be attached to a task, attached to a meeting, or filed against a reporting period via period_month with no parent record at all. A table-level check constraint enforces that at most one attachment context is set.
- **Report**: a generated management report. generated_by is who ran it; subject_use_id is nullable and only set for per-person reports, which is what distinguishes it from an organisation-wide summary. file_path points at the rendered PDF in Blob Storage; the document bytes are not stored in a row.

Notifications and audit

- **Notification_Type**: a lookup classifying notifications, so a new alert type does not require a schema change.
- **Notification**: a message delivered to one user, with is_read tracking delivery state. entity_type and entity_id form a deliberately polymorphic pointer to whatever the notification concerns.
- **Audit_Log**: an append- only record of who performed which action on which entity, at what time. Nothing in the application updates or deletes from this table.

Relationships

Identity and access

#	Parent	Child	Card.	Reads as	Foreign key	Why
1	`Role`	`Role_Permission`	1:M	a role **grants** many permissions	`role_id`	Half of the role-to-permission many-to-many.
2	`Permission`	`Role_Permission`	1:M	a permission is **granted to** many roles	`permission_id`	Other half. The same capability is reused across roles.
3	`Role`	`User`	1:M	a role **has** many users	`User.role_id`	A user holds exactly one role at a time, so this is a mandatory single-valued FK rather than a junction. Multiple simultaneous roles were not a requirement.

Figure 25: Identity and access relationships

Projects and tasks

#	Parent	Child	Card.	Reads as	Foreign key	Why
4	User	Project	1:M	a user **owns** many projects	Project.owner_id	Accountability. Every project has one identifiable owner.
5	Project	Project_Member	1:M	a project **has member**	project_id	Half of the project-to-user many-to-many.
6	User	Project_Member	1:M	a user is **member of** many projects	user_id	Other half. date_added is an attribute of the membership itself, which is a further reason the junction is a table rather than an implicit link.
7	Project	Task	1:M	a project **contains** many tasks	Task.project_id	Mandatory. Every task belongs to exactly one project, which is what gives project-scoped visibility a path to reach tasks.
8	User	Task	1:M	a user **creates** many tasks	Task.created_by	Records authorship, which is distinct from assignment and must survive reassignment.
9	User	Task	1:M (opt.)	a user **completes** many tasks	Task.completed_by	Nullable. Records who actually closed the task, which may be neither the creator nor the original assignee.
10	Task_Schedule	Task	1:M (opt.)	a schedule **has** many task instances	Task.schedule_id	Nullable. Links a generated task back to the template that produced it, so a recurring duty can be reported on as a series.
11	Task	Task_Assignment	1:M	a task **has** assignments	task_id	Half of the task-to-user many-to-many.
12	User	Task_Assignment	1:M	a user is **assigned** many tasks	user_id	Other half. A junction rather than an assigned_to column on Task, because the client requires that several people can share one task.
13	Task	Task_Comment	1:M	a task **has** comments	task_id	Discussion is scoped to its task and cascades on task deletion.
14	User	Task_Comment	1:M	a user **writes** comments	user_id	Attribution of each comment.
15	Task	Task_Status_History	1:M	a task **logs** status changes	task_id	The movement log. Its existence is why Task.status can safely be a single current-value column.
16	User	Task_Status_History	1:M	a user **changes** status	changed_by	Accountability for each individual transition.
17	User	Task_Schedule	1:M	a user **schedules** recurring tasks	created_by	Who created the template.
18	User	Task_Schedule	1:M	a schedule is **assigned to** a user	assigned_to	Who the recurring duty falls to by default. Separate from created_by because a chairperson typically schedules work for someone else.

Figure 26: Projects and tasks relationships

Meetings and events

#	Parent	Child	Card.	Reads as	Foreign key	Why
19	`User`	`Meeting`	1:M	a user **organises** meetings	`organiser_id`	Every meeting has one convener.
20	`Meeting`	`Meeting_Attendee`	1:M	a meeting **has** attendees	`meeting_id`	Half of the attendance many-to-many.
21	`User`	`Meeting_Attendee`	1:M	a user **attends** meetings	`user_id`	Other half.
22	`Meeting`	`Meeting_Minutes`	1:M	a meeting **has** minutes	`meeting_id`	Modelled as one-to-many rather than one-to-one so a draft and an approved version can coexist as separate rows, preserving what the AI produced alongside what the secretary approved.
23	`User`	`Meeting_Minutes`	1:M	a user **records** minutes	`recorded_by`	Who took or generated the minutes.
24	`User`	`Meeting_Minutes`	1:M (opt.)	a user **approves** minutes	`approved_by`	Nullable, and the mechanism enforcing that an AI draft requires human sign-off before it is the minute of record.
25	`User`	`Event`	1:M	a user **organises** events	`organiser_id`	As for meetings.
26	`Event`	`Event_Attendee`	1:M	an event **has** attendees	`event_id`	Half of the event attendance many-to-many.
27	`User`	`Event_Attendee`	1:M	a user **attends** events	`user_id`	Other half, carrying the RSVP `status` attribute.

Figure 27: Meetings and events relationships

Resource requests

#	Parent	Child	Card.	Reads as	Foreign key	Why
28	User	Resource_Request	1:M	a user **submits** requests	requested_by	The requester.
29	Request_Category	Resource_Request	1:M	a category **categorises** requests	category_id	A lookup, so categories are maintained as data and remain reportable.
30	User	Resource_Request	1:M (opt.)	a user **resolves** requests	reviewed_by	Nullable while the request is pending. Separate from requested_by so the model can express, and the application can enforce, that an approver is not the requester.
31	Resource_Request	Request_Status_History	1:M	a request **logs** status changes	request_id	Full transition history, giving turnaround time by subtraction.
32	User	Request_Status_History	1:M	a user **changes** status	changed_by	Accountability per transition.

Figure 28: Resource requests relationships

Files and reporting

#	Parent	Child	Card.	Reads as	Foreign key	Why
33	File_Category	File_Upload	1:M	a category **categorises** files	category_id	Lookup, keeping the document library filterable.
34	User	File_Upload	1:M	a user **uploads** files	uploaded_by	Attribution, required by the audit trail.
35	Task	File_Upload	1:M (opt.)	a file is **attached to** a task	task_id	Nullable. Task attachments.
36	Meeting	File_Upload	1:M (opt.)	a meeting **has file**	meeting_id	Nullable. Agendas, recordings, and supporting documents.
37	User	Report	1:M	a user **generates** reports	generated_by	Who ran the report.
38	User	Report	1:M (opt.)	a report is about a **subject** user	subject_user_id	Nullable, set only for per-person reports. This is what separates an individual performance report from an organisation-wide one.

Figure 29: Files and reporting relationships

Notifications and audit

#	Parent	Child	Card.	Reads as	Foreign key	Why
39	User	Notification	1:M	a user **receives** notifications	user_id	Notifications are per-recipient rows, so is_read is tracked individually.
40	Notification_Type	Notification	1:M	a type **classifies** notifications	type_id	Lookup, so a new alert type is a row rather than a migration.
41	User	Audit_Log	1:M	a user **performs** actions	user_id	Every audited action resolves to a real, non-deletable user.

Figure 30: Notifications and audit relationships

Design Decisions

- **Separate history tables rather than more columns**. Task_Status_History and Request_Status_History could have been avoided by keeping only the status and a last_changed_by. They exist because the client needs to answer, “how long did this sit in review, and who moved it?” and because an append-only log is evidence in a way an overwritten column is not. The cost is one extra table per workflow; the benefit is that no state change can be silently lost.
- **Current-value columns kept alongside the history**. Task.status and Task.percent_complete duplicate information derivable from the latest Task_Status_History row. This is a deliberate, documented departure from strict normalisation. Every task-board screen needs the status of many tasks at once, and deriving it would mean a correlated subquery or window function per row on the most frequently loaded view in the system[6]. The history table remains the system of record; the columns on Task are a cache of its latest row, maintained in the same transaction as the history insert.
- **Task_Schedule as a template, Task as the instance.** A recurring weekly duty could have been modelled as a recurrence_rule column on Task. It is separate because a template and an occurrence have genuinely different lifecycles: editing next Saturday’s kitchen duty must not rewrite last Saturday’s completed record, and deactivating a recurring duty must not delete the generated tasks in one table so that all reporting queries see both.
- **A polymorphic pointer on Notification and Audit_Log.** A notification can concern a task, a request, a meeting, a file, or several other things. The alternatives were ten nullable foreign-key columns, of which nine are always null, or a separate notification table per target type. Both are worse: the first is mostly empty and grows a column per feature, the second multiplies tables and makes “all my unread notifications” a union. The chosen entity_type and entity_id pair gives up database-enforced referential integrity on that one link, which is accepted because a notification pointing at a deleted record is a cosmetic problem rather than a data-integrity one[17]. This is the single place in the schema where integrity is enforced by the application rather than the engine, and it is deliberate.
- **Nullable task_id and meeting_id on File_Upload.** A file has at most one attachment context, so the alternative was two junction tables or a separate upload table per context. Two nullable foreign keys plus a check constraint permitting at most one of them keeps every file in one queryable library, which is what the client asked for: one place to search all documents. period_month covers the third case, a monthly report filed against a period with no parent record.

- **Deactivation rather than deletion of user.** User.is_active exists so a departed member's account can be closed without deleting the row. Because 22 relationships point to User, deleting one would either cascade away a large amount of history or fail on constraint violations. Foreign keys on User therefore use ON DELETE NO ACTION, and the application never hard-deletes a user[6].

Referential Integrity

Rule	Applied to	Reason
`ON DELETE CASCADE`	`Task` → `Task_Comment`, `Task_Assignment`, `Task_Status_History`; `Meeting` → `Meeting_Attendee`; `Event` → `Event_Attendee`; `Resource_Request` → `Request_Status_History`	These children have no meaning without their parent, so deleting the parent should remove them.
`ON DELETE NO ACTION`	every foreign key referencing `User`	Protects the audit trail. Users are deactivated, never deleted.
`ON DELETE SET NULL`	`Task.schedule_id`, `File_Upload.task_id`, `File_Upload.meeting_id`	An optional link may be removed without destroying the child record; a file outlives the task it was attached to.
`NO ACTION` on lookups	`Role`, `Permission`, `Request_Category`, `File_Category`, `Notification_Type`	A lookup value still in use cannot be deleted.
Unique constraint on the FK pair	`Task_Assignment`, `Project_Member`, `Meeting_Attendee`, `Event_Attendee`, `Role_Permission`	Prevents the same user being assigned or enrolled twice.
Check constraint	`File_Upload` (at most one of `task_id`, `meeting_id`)	Enforces the single-context rule described above.
Unique constraint	`User.email`	The login identifier must be unique.

Figure 31: Referential Integrity table

Indexing

Clustered indexes follow the primary keys. Non-clustered indexes are created on every foreign key, since all of them are join targets, and on four access paths that the use cases exercise directly: User.email (login lookup), Task (the task board), Notification (the unread badge), and Audit_log (audit queries, which are always time-ranged)[6].

Forward engineering

Each entity in the ERD corresponds to a C# class under repository, annotated with [Table], [Key], [MaxLength], and [ForeignKey] attributes that map it to the physical schema. EF Core migrations generate the T-SQL that creates the database, and those migration files are linked to the repository[18].

Architecture Plans

QULOOBUL MOE'MIENEEN Management Platform

Architecture Artifacts

1. Purpose and scope

This document records the architectural decisions behind the QULOOBUL MOE'MIENEEN Management Platform and the reasoning that produced them. It covers three things the brief asks for: the design patterns used inside the application and why they were selected, the architecture pattern that organises the system as a whole, and the cloud architecture the system is deployed on.

The decisions here are constrained by facts established in the earlier deliverables: a single small development team working in C#/.NET, one client organisation with roughly 24 staff, a hard POPIA data-residency requirement, and an infrastructure budget of roughly $45-$90 per month covered by a Microsoft for Nonprofits grant. Several patterns that would be defensible on a larger system are rejected below for exactly these reasons, and the rejections are recorded rather than left implicit.

Design Patterns

OBJ-01 Object Diagram — Repository / Unit of Work and Observer at the instant task 142 is assigned (UC-04, UC-08)

Snapshot taken inside the transaction opened by SD-02, after the assignment and notification have been added to the context but before SaveChanges() commits.

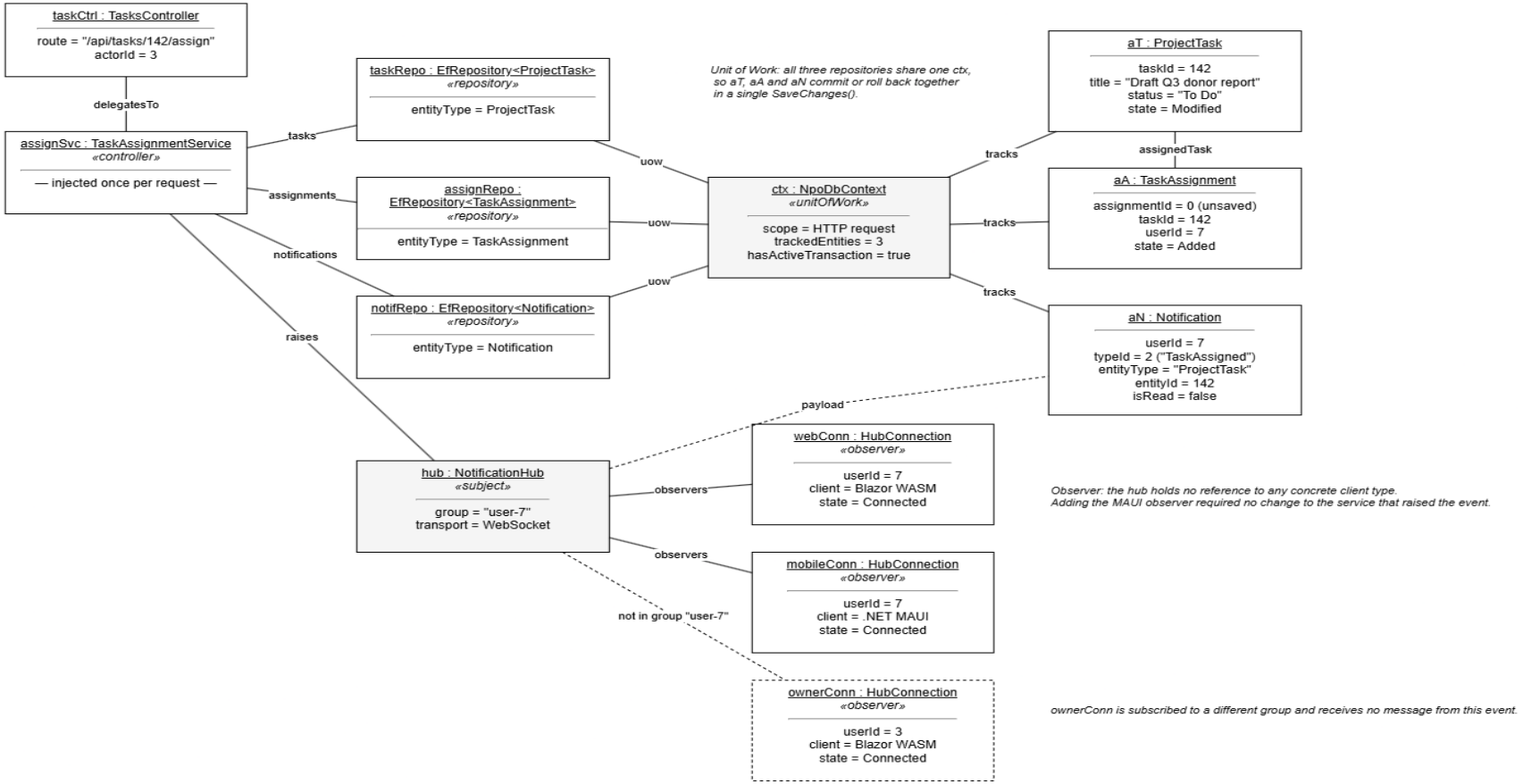


Figure 32: OBJ-01 Object Diagram

Repository and Unit of Work

Assigning a task writes to three tables: ProjectTask, TaskAssignment, Notification, and AuditLog. If those writes are issued independently, a failure partway leaves a task marked as assigned with nobody notified, and the service layer ends up holding query code that cannot be tested without a database[17],[19].

Entity Framework Core is used to implement both patterns rather than as a layer underneath a hand-written one. DbSet<T> already is a Repository, and DBContext already is a Unit of Work: it tracks every changed entity and commits them in a single transaction on SaveChanges(). A thin generic EfRepository<T> is exposed to the service layer only so that services depend on IRepository<T> rather than on EF types directly[3].

Observer

UC-08 requires a staff member to see a task appear on their board as soon as it is assigned, and the same event must reach a browser and, once the MAUI client ships, a phone. Polling for it would mean every connected client hitting the API on a timer, which on a Basic B1 App Service is measurable load for something that happens a few times an hour.

NotificationHub is the subject; each live SignalR connection is an observer, subscribed to a group named after its user. TaskAssignmentService raises one event and knows nothing about who is listening or on what device[20],[21]. This is the usual intent of Observer: a one-to-many dependency where dependants are updated automatically[20]. The MAUI app subscribes to the same hub, and no server code changes.

Use-case Controller and Facade

Each use case has one service class that the HTTP controller delegates to the <<controller>> stereotype shown in the middle of every sequence diagram, following Satzinger's[22] three-layer design. TaskAssignmentService is the single place that decides whether an assignment is legal, writes the assignment, creates the notification, raises the SignalR event, and records the audit entry.

The reason that this pattern earns its place is because of the second client. A rule that lives in a controller action is reachable only over the route that action is bound to; a rule that lives in a use-case service is reachable from the REST endpoint, from a future background job that auto-escalates requests 48 hours (SM-02); and from an integration test that never starts a web server. The HTTP controller is left with model binding, DTO mapping, and status-code selection.

Dependency injection

Every dependency is constructor-injected and registered in the ASP.NET Core container at startup[17],[23]. Repositories and the DbContext are scoped to the HTTP request; the

hub context and configuration are singletons. Nothing in the service layer calls new on a dependency[23].

The request-scoped DbContext registration is what makes the Unit of Work boundary correct. Every repository resolved in a single request receives the same context instance, as illustrated by property OBJ-01. The connection string and JWT signing key come from Key Vault in production and from .NET user secrets locally[24]. Because the configuration is injected rather than read inline, that difference is a registration change, not a code change.

Data Transfer objects

Clients never receive EF entities[17]. QULOOBUL MOE'MIENEEN.Shared holds the DTO and its validation attributes and is referenced by the API and by both clients, so the wire contract is declared once, and a change to it breaks the build rather than a demo[25],[26]. This pattern makes .NET MAUI decisions pay off. A JavaScript or Dart client would have to re-declare every one of these types by hand and keep them in sync manually.

State machine as guarded transitions

ProjectTask, ResourceRequest, and User each have a defined life cycle, documented as SM-01 to SM-03. These are implemented as a transition table with guard conditions checked inside the owning use-case service.

This is simpler, as ProjectTask has 6 states whose behaviour differs in which transitions are legal, not in what the object does. A class per state would add six types and their wiring to express a 2-column table rule. The State pattern shines when each state carries different behaviour[20]. What isn't simplified is the guard checked server-side in the service layer on every transition, because a hidden button in the client is access control, and a client that stops sending an illegal transition does not stop an attacker from sending it[15].

Chain of Responsibility

The ASP.NET Core middleware pipeline is a chain of responsibility: HTTPS redirection, CORS, JWT authentication, role and permission authorisation, audit capture, and the global exception handler each inspect the request, act, and pass it on[20],[27]. The pattern is what allows a security control to be added in one place and applied to every endpoint, including endpoints written after it. The alternative would be an authorisation check written into each controller action, which fails the first time somebody forgets one, and that failure is silent.

Patterns deliberately not used

Pattern	Why it was rejected
CQRS with a mediator	Splits reads from writes and adds a handler class per operation. Justified when read and write loads scale

	separately; this system has 24 users and one database. It would roughly double the class count for no measurable benefit.
Abstract Factory/provider abstraction over storage	Would let the platform swap Azure Blob Storage for another provider. There is no second provider and no plan for one - the whole cost model depends on the Azure nonprofit grant. Speculative flexibility.
Singleton for the DbContext	Actively wrong, not merely unnecessary: DbContext is not thread-safe, and its change tracker would accumulate every entity ever loaded. Request scope is the correct lifetime.
Specification pattern for queries	Composable query objects over LINQ. LINQ already composes; this would add an indirection layer to reproduce what the language does natively.
Full GoF State pattern	A class per state for states that differ only in legal transitions.

Object diagram walkthrough

OBJ-01 is a screenshot of live objects at one instant during UC-04: after TaskAssignmentService has modified task 142, created the assignment, and created the notification, but before the transaction commits. It shows 2 of the patterns above at the instance level.

- Unit of Work (uow): taskRepo, assignRepo, and notifRepo are 3 distinct repository instances, but every uow link points to the same object, ctx: QULOOBUL MOE'MIENEENDbContext, whose trackedEntities = 3 and hasActiveTransaction = true. That shared instance is the pattern: aT (Modified), aA (Added, assignmentId still 0 because the database has not issued it), and aN (Added) will commit or roll back together.
- Observer hub: NotificationHub is linked to a webConn and mobilrConn, both belonging to userId 7, the same person on 2 devices, each an independent observer. ownerConn (userId 3) is connected to the hub but is in a different group and receives nothing from this event, which is what the dashed link records. The hub holds no reference to any concrete client type, so the MAUI observer was added without changing the object that raised the event.

The diagram shows one moment rather than a sequence; the ordering of the same interaction is already documented in SD-02.

Architecture Patterns

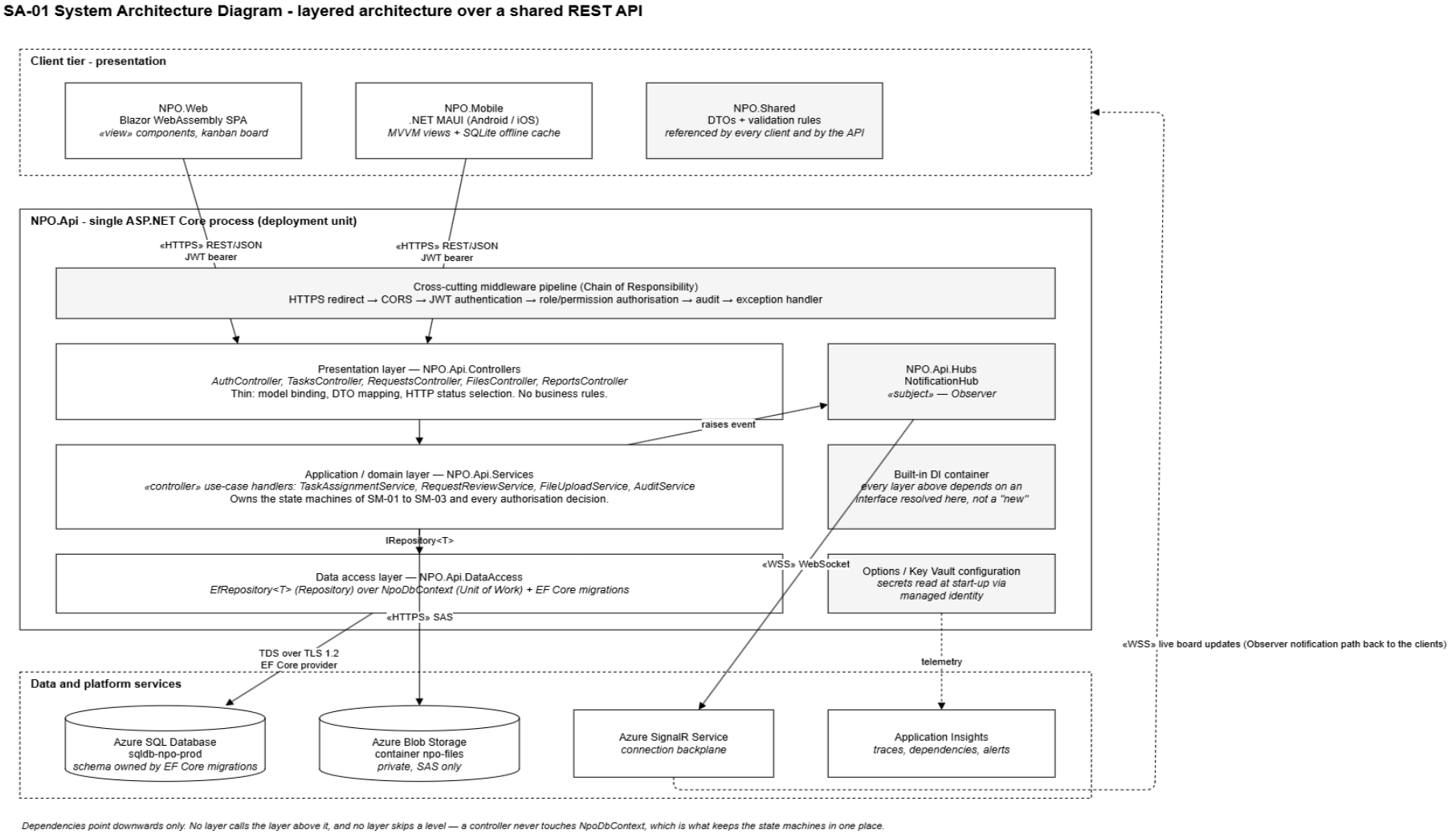


Figure 33: SA-01 System Architecture Diagram

Pattern chosen

The system is a layered (3-layer) architecture deployed as a single client- server application, with a stateless REST API serving multiple thin clients and a secondary event-driven channel for push updates. One deployable API process organised into presentation, application, and data-access layers, consumed over HTTPS by a Blazor WebAssembly single-page application and a .NET MAUI mobile application, with Azure SignalR carrying server-initiated messages back to them[25],[28].

Element of the pattern	Realisation in this system
Presentation Layer	QULOOBUL MOE'MIENEEN.Api.Controller: routing, model binding, DTO mapping, status code. No business logic
Application/domain layer	QULOOBUL MOE'MIENEEN.Api.Services: use-case handlers, life-cycle guard from SM-01 to SM-03, all authorisation decisions
Data access layer	QULOOBUL MOE'MIENEEN.Api.DataAccess: EfRepository<T>, Quloobul Moe'MieneenDbContext, EF Core migrations, T-SQL for indexes and views.
Client tier	QULOOBUL MOE'MIENEEN.Web and QULOOBUL MOE'MIENEEN.Mobile, both consuming QULOOBUL MOE'MIENEEN.Shared DTOs
Event Channel	QULOOBUL MOE'MIENEEN.Api.Hubs.NotificationHub over Azure SignalR Service
Data tier	Azure SQL Database; Azure Blob Storage for Attachment

Reasoning

- The system is a records and workflow application: forms, lists, status transitions, and an audit trail over one relational schema. Layering separates the concerns that vary here: presentation, rules, persistence; without inventing boundaries, the domain does not have[29].
- One API, several clients. The web client and the mobile client are two presentations of the same use cases. Putting the rules behind the REAT API means the mobile client required no backend rework at all. Only a JWT bearer configuration addition alongside the existing cookie scheme[30].
- No session state is held in the process, which makes the blue-green slot swap in the deployment plan safe: a request can be served by the staging build or the production build with the same result, and a rollback is a swap back rather than a restore[28],[31].
- It fits the budget. A layered monolith runs on one Basic B1 App Service instance at $13-$55 per month. That is why the platform fits inside the nonprofit grant.

Known limits

- Whole API scales as one unit. If reporting queries (UC-09) ever load the process, everything scales with them. Mitigation is vertical: S0 to S1 on the database, B1 to a standard plan on the App Service. DTU alert at 80% is the trigger.
- Layer discipline is a convention, not a compiler rule, since presentation, application, and data access are folders in one project rather than separate assemblies. It is enforced in code review. Splitting them into 3 assemblies is a mechanical change if the discipline slips.
- A single deployment unit means one bad deploy affects every feature. This is covered by the slot swap and the sub-2-minute rollback in the deployment plan rather than by the architecture[32].

Alternatives considered and rejected

Alternative	Why it was not selected
Microservices	Would split tasks, requests, files, and identity into separate deployables. The cost is real and immediate - several App Service plans instead of one, a distributed transaction problem where SaveChanges() currently suffices, and cross-service tracing - while the benefit (independent scaling and independent deployment by separate teams) does not exist for one team serving 24 users on one database. It would also break the nonprofit budget on hosting alone.
Server-rendered MVC monolith (Razor Pages)	Simpler to build for the web client alone, and a legitimate option until the mobile client entered scope. It was rejected because it has no answer for a native mobile client: the mobile app would need a REST API built alongside the MVC controllers, which is the layered API design plus duplicated presentation logic.
Two-tier client-server (client talks to Azure SQL directly)	Rejected on security grounds. It requires database credentials on every device and makes server-side enforcement of role and permission checks impossible, which is incompatible with the POPIA obligations set out in the deployment plan.
Serverless (Azure Functions)	Attractive on cost at low traffic, but the client requires 24/7 availability, and cold starts degrade the interactive kanban experience. SignalR also needs a persistent connection story that Functions complicates rather than simplifies.
Event-driven architecture as the primary style	Message-broker-first design would decouple components further, but the workflows here are synchronous by nature - a user submits a request and expects a confirmation in the response. Events are used where they genuinely apply (notifications) and nowhere else.

Cloud Architecture

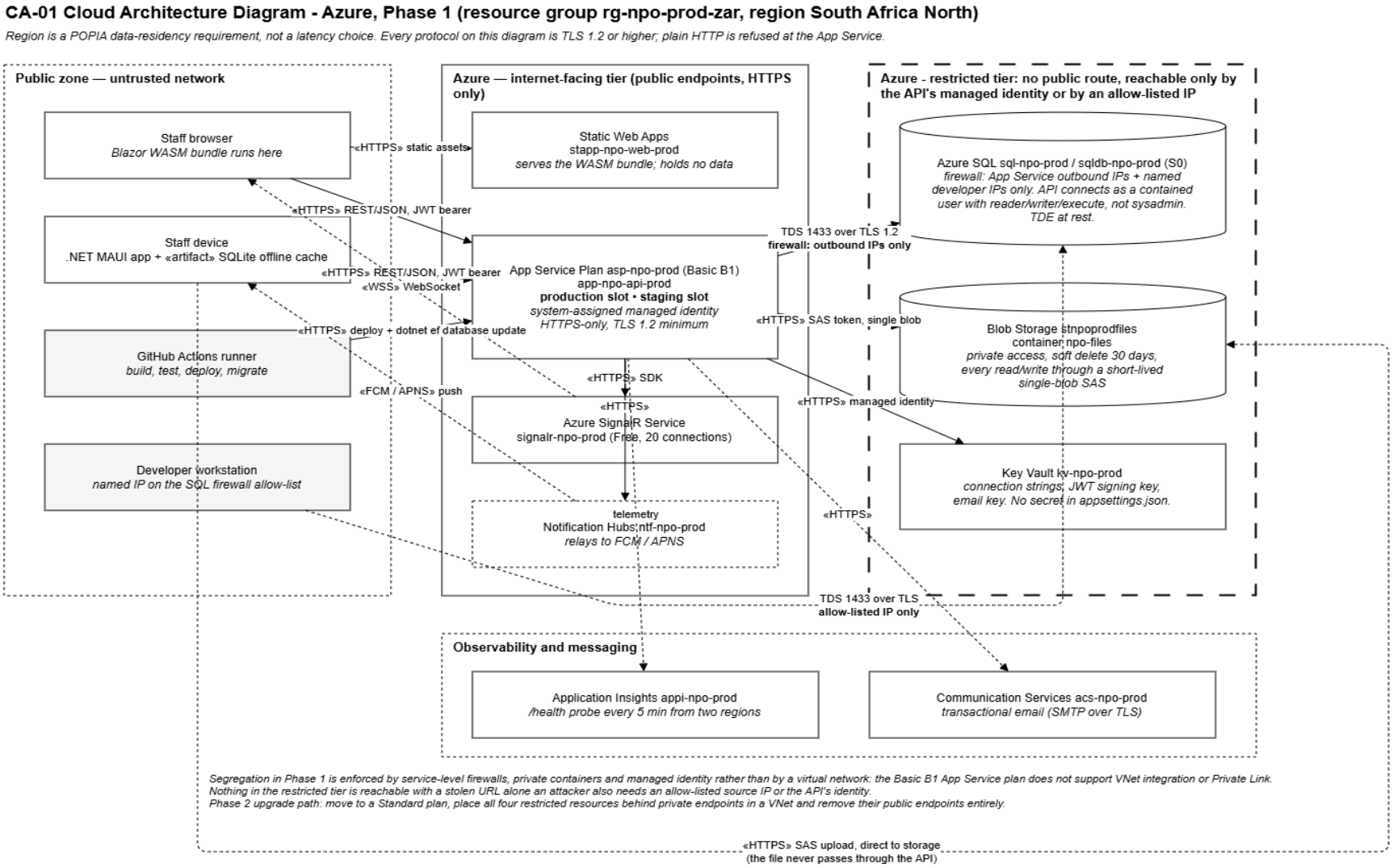


Figure 34: CA-01 Cloud Architecture Model

Services used

Microsoft Azure was chosen based on Microsoft for Nonprofits providing up to $2 000 per year of Azure credit through TechSoup SA, which covers the estimated $45-$90 monthly costs for phase 1[7]. The client had requested a cloud database solution with minimal costs; hence, this nonprofit grant was essential. Azure has a South Africa North region, which is a POPIA requirement rather than a latency preference[33],[16].

Service	Resource	Role in the architecture
App Service Plan	asp-Quloobul Moe'Mieneen-prod (Basic B1, always on)	Compute for the API. Free F1 is not viable; its 60 CPU-minute daily cap is exhausted before midday under 24/7 operation[34].
App Service	app-Quloobul Moe'Mieneen-api-prod	Hosts QULOOBUL MOE'MIENEEN.Api. Two slots: production and staging, which is the release mechanism[35].
Azure SQL Database	sqldb-Quloobul Moe'Mieneen-prod (Standard S0/S1)	System of record. Provisioned, not serverless[10].
Static Web Apps	stapp-Quloobul Moe'Mieneen-web-prod (Free)	Serves the Blazor WebAssembly bundle. Holds no data; the bundle is public static content[36].
Blob Storage	stQuloobul Moe'Mieneenpodfiles, container Quloobul Moe'Mieneen-files	Task attachments and database exports. Private container, SAS-only access; 30-day soft delete.
SignalR Service	singalr-Quloobul Moe'Mieneenprod (Free)	Backplane for the Observer channel
Key Vault	kv-Quloobul Moe'Mieneen-prod (Free)	Connection strings, JWT signing key, email key. Read at start-up via managed identity.
Notification Hub	ntf-Quloobul Moe'Mieneen-prod (Free)	FCM/APNS push for the MAUI client when the app is closed. Only required once that client ships.
Communication Services	acs-Quloobul Moe'Mieneen-prod	Transactional email for request and notification workflows.
Application Insights	appi-Quloobul Moe'Mieneen-prod	Traces, dependency calls, exceptions, and the /health availability test that backs the 24/7 requirement.

Network segregation

CA-01 divides the deployment into 3 zones. The boundary that matters is the one around the restricted tier: no client, on any network, reaches the database, the file container, or the secret store directly.

Zone	Contains	Who can reach it
Public Zone	Staff browser, staff devices, the GitHub Actions runner, developer workstations	Untrusted. Assumed volatile
Internet-facing tier	Static Web Apps, App Service (both slots), SignalR Service, Notifications Hub	Anyone on the internet can reach the endpoint. HTTPS-only with TLS 1.2 minimum enforced at the App Service; every API route except /health and login requires a valid JWT, and role and permission checks run server-side on top of that.
Restricted tier	Azure SQL, Blob Storage, Key Value	Not reachable by a URL alone. SQL accepts connections only from the App Service outbound IP addresses and a short list of named developer IPs. Blob containers are private, and every read or write uses a short-lived SAS scoped to a single blob. Key Vault is opened only by the App Service system-assigned managed identity; there is no key to steal because there is no key.

Phase 1 segregation is enforced by service-level firewalls, private containers, and managed identity, not by a virtual network. The Basic B1 App Service plan does not support VNet integration or Private Link, so the restricted resources technically retain public endpoints that refuse unauthorised callers, rather than having no public endpoint at all.

Protocols between services

From	To	Protocol	Authentication
Browser	Static Web Apps	HTTPS	None; public static bundle
Browser/MAUI App	App Service (APi)	HTTPS, REST/JSON	JWT bearer token
MAUI app	Identity provider	OpenID Connect/OAuth 2.0	MSAL, if Entra External ID is used
SignalR Service	Browser/MAUI app	WSS (WebSocket)	Access token negotiated through the API
App Service	SignalR Service	HTTPS, SignalR SDK	Connection string from Key Vault
App Service	Azure SQL	TDS over TLS 1.2, port 1433	Contained database user; server firewall restricted to outbound IPs
App Service	Blob Storage	HTTPS	Short-lived SAS, scoped to one blob

Client	Blob Storage	HTTPS	SAS issued by the API. The file itself never passes through the API (SD-04)
App Service	Key Vault	HTTPS	System-assigned managed identity: no stored credentials
App Service	Communication Service	HTTPS	Key from Key Vault
Notification Hubs	Device	FCM/APNS push	Platform credentials held in the hub
GitHub Actions	App Service/Static Web Apps	HTTPS	Repository secrets AZURE_CREDENTIALS, AZURE_STATIC_WEB_APPS_TOKEN

Every connection on this list is encrypted in transit. Data at rest is encrypted by Transparent Data Encryption on Azure SQL and Storage Service on Blob Storage, both on by default.

Technology and their trade-offs

Decision	Reasoning
Basic B1 App Service, not Free F1	F1 caps at 60 CPU minutes per day, which is exhausted by midday under continuous use, and it has no Always On, so the first request after an idle period pays a cold start. The client requires 24/7 availability
Provisioned Azure SQL (S0/S1), not serverless	Serverless bills per vCore-second and auto-pauses. Under continuous load, its 0.5 vCore minimum works out at roughly $190 per month; several times the provisioned tier. Auto-pause would add a resume delay to the first request of the day
Deployment slots, not 2 App Service plans	Staging and production share one Basic plan through slots. This keeps compute at one instance and makes the near-instantaneous slot swap possible, which is also the rollback mechanism.
Static Web Apps for the client, not the API's wwwroot	The WASM bundle is static content. Serving it from the free Static Web Apps tier keeps that traffic off the B1 instance entirely, at no cost, with its own CI/CD from GitHub.
Azure SignalR Service, not self-hosted SignalR	Self-hosting would tie WebSocket connections to the App Service instance and complicate the slot swap. The managed service holds the connections outside the process.
Key Vault with managed identity, not app settings	App settings are readable by anyone with portal access. Managed identity removes the bootstrap credential problem entirely, and Key

	Vault versioning means that rotating an exposed secret requires no code changes.
One region, no multi-region failover	Multi-region would multiply the cost of every resource. POPIA constrains the platform to South African regions in any case. Accepted as Phase 1 limitation; mitigation by point-in-time restore rather than by failover.

Known limits of this cloud architecture

- The free SignalR tier caps at 20 concurrent connections and the organisation is expecting 24 users. The 21st connection is refused, and the user silently loses live board updates. The Standard tier is about $1.61 per unit per month.
- No automated multi-region failover in Phase 1. A regional outage is communicated to the client, and the client waits it out.
- Segregation is identity- and firewall-based rather than network-based, for the plan tier.
- The cost model depends on a grant that must be approved and annually renewed.

Diagram index

Diagram	Type	Documents
OBJ-01	UML object diagram	Repository, Unit of Work, and Observer at instance level during UC-04 task assignment.
SA-01	System architecture diagram	The layered architecture, the boundaries between layers, and how the major components are organised.
CA-01	Cloud architecture diagram	Azure services, the three network zones, and the protocol and authentication on every connection.

These sit alongside the diagrams already delivered in Implementation.drawio (SD-01 to SD-04, SM-01 to SM-03, DEP-01). SA-01 and DEP-01 are complementary, not duplicates. DEP-01 shows where software components physically run; SA-01 shows how the code inside them is organised.

DevOps

Version control

All source code lives in a single GitHub repository owned by DRPS Solutions Ltd. A single repository is deliberate: the API, the web client, the database migrations, and the pipeline definitions version together, so any commit identifies exactly which application code and which database schema were released as a pair.

Path in repository	Contents
/src/Quloobul Moe'Mieneen.Api	ASP.NET Core Web API: controllers, services, business logic, validation
/src/Quloobul Moe'Mieneen.Web	Blazor WebAssembly client
/src/Quloobul Moe'Mieneen.Shared	Shared class library: DTOs and entities referenced by the API, the web client, and the MAUI app
/src/Quloobul Moe'Mieneen.Data	EF Core DbContext, entity configurations, and generated migrations
/db	T-SQL scripts: stored procedures, views, seed data, role grants
/tests	Quloobul Moe'Mieneen.UnitTests and QULOOBUL MOE'MIENEEN.IntegrationTests
/.github/workflows	Pipeline definitions (ci.yml, cd-staging.yml, cd-production.yml)
/docs	POE artefacts: documentation

Commit discipline: small, single-purpose commits with a message in the form <type>: <what changed>. Secrets, connection strings, and publish profiles are never committed; appsettings.Development.json and .env files are listed in the .gitignore, and the real values live in the repository secrets on GitHub and Azure Key Vault.

Branching model and development flow

Branch	Purpose	Protection	Deploys to
main	Production. Always represents what the QULOOBUL MOE'MIENEEN is currently running.	Protected. No direct pushes. Pull request plus one reviewer approval. All status checks must pass.[37].	Production, after a manual approval gate and a slot swap. Project lead only
develop (dev)	Integration branch. Everything that is finished but not yet released.	Protected. No direct pushes. Pull request plus passing status checks.	Staging, automatically on every merge.

feature/<name>	One branch per use case or defect, short-lived (target: under three days).	None, but the pull request into develop cannot merge while checks are failing.	Nothing. Continuous integration only.

The GitHub Actions pipeline

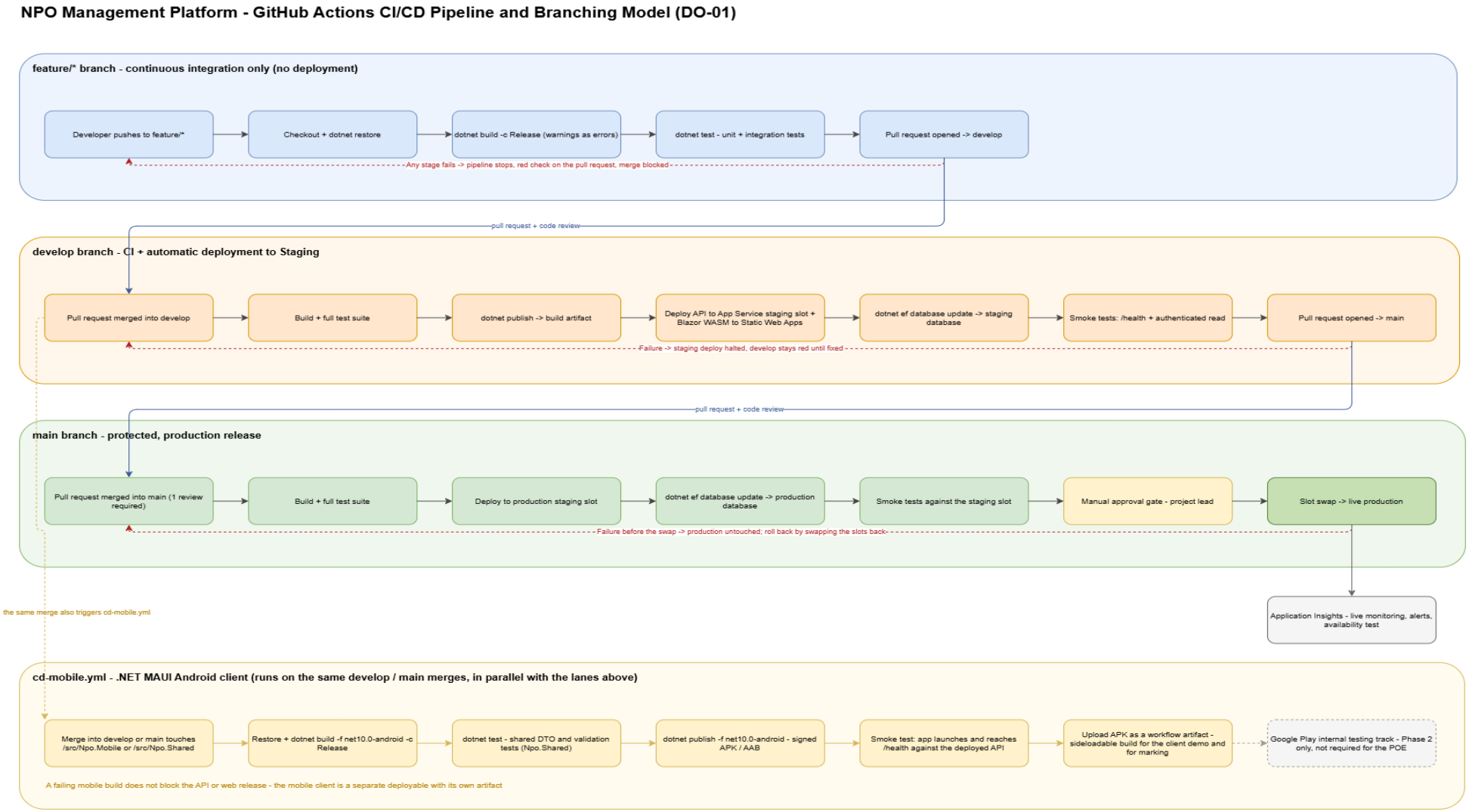


Figure 35: CI/CD Pipeline and Branching Model (DO-01)

The pipeline is defined as 3 workflow files in /.github/workflows. Splitting them by trigger keeps each file short and makes it obvious from the file name which branch causes which behaviour.[38].

Workflow	Trigger	What it does
ci.yml	Push to feature/**; pull request targeting develop or main	Checkout, dotnet restore, dotnet build -c Release, dotnet test. Builds and tests only. It deploys nothing and holds no deployment credentials.
cd-staging.yml	Push to develop	Everything in ci.yml, then dotnet publish, deploy the API to the App Service staging slot and the Blazor bundle to Azure Static Web Apps, apply migrations to the staging database, run smoke tests.
cd-production.yml	Push to main	Everything in cd-staging.yml against production resources, then a manual approval gate on a protected GitHub environment, then the slot swap.[38],[35].

Stages in detail

#	Stage	Command/action	Fails the build if
1	Checkout and restore	actions/ checkout, actions/setup-dotnet, dotnet restore	A NuGet package cannot be resolved
2	Build	dotnet build -c Release with TreatWarningsAsErrors	Any compilation error or any compiler warning
3	Automated tests	dotnet test (unit and integration projects)	Any test fails, or line coverage falls below the agreed threshold
4	Publish	dotnet publish -c Release -o ./puvlish, uploaded as a build artefact	Publish error
5	Deploy API	azure/webapps-deploy to the staging slot	Deployment error or a failed health probe
6	Deploy web client	Azure/static-web-apps-deploy	Build or upload error

7	Database migration	dotnet ef database update against the target database	Migration error: deployment stops, and no slot swap occurs
8	Smoke tests	Automated request to /health plus one authenticated read endpoint	Any non-200 response
9	Approval gate	GitHub environment reviewer	Manual: the project lead's decision; the run expires if not approved
10	Slot swap	az webapp deployment slot swap	Swap error: the previous build remains live

Automated testing in the pipeline

Layer	Tooling	What it covers	Where it runs
Unit tests	xUnit, FluentAssertions, Moq	Business rules in isolation: role permissions in the login hierarchy, task state transitions, request approval rules, validation	Every push, every pull request, both deployment workflows[39],[40].
Integration tests	xUnit with WebApplicationFactory, EF Core against disposable test database[41]	Controller endpoints end-to-end. Routing, model binding, authentication and authorisation, EF Core queries, migrations applying cleanly to an empty database	Every push, every pull request, both deployment workflows
Smoke tests	Workflow step issuing HTTP requests against the deployed slot	The deployed build actually starts, reaches its database, and answers an authenticated request	After deployment, before the production swap
Manual acceptance	Test script against the staging environment	Usability and client sign-off on each use case; the areas automated	Staging, before a release pull request into main

		tests are poor at judging	

Quality gates

Gate	Enforced by	Blocks
Compiles with no warnings	TreatWarningsAsErrors in the build stage	The pipeline run
All automated tests pass.	Dotnet test exit code	The pipeline run
Coverage threshold met	Coverlet threshold in the test stage[42]	The pipeline run
One reviewer approval	GitHub branch protection on develop and main	The merge
Status checks green	Required status checks on the protected branch	The merge
Migrations apply cleanly	Migration stage of the deployment branch	The deployment and the swap
Deployed build answers its health probe.	Smoke test stage	The swap to production
Released approved	GitHub environment reviewer (Project Lead)	The swap to production

Secrets and configuration

Deployment credentials are stored as GitHub repository and environment secrets: AZURE_CREDENTIALS, AZURE_STATIC_WEB_APPS_API_TOKEN, and the migration connection string. The production secrets are scoped to a protected GitHub environment, so a workflow running from a feature branch cannot use them even if the workflow file is edited on that branch.

Application runtime secrets: the database connection string, the Blob Storage account key, and the email service key are held in the Azure Key Vault and read by the App Service through its system-assigned managed identity, so the running application holds no secret in configuration either[43]. Each environment has its own vault entries; nothing is shared between staging and production.

How it meets the non-functional requirements

Requirements	How the DevOps approach addresses it
Rapid deployment	A merge is the release trigger. From merge to live is a single pipeline run of roughly eight to twelve minutes, of which only the approval gate is manual.
24/7 availability	Releases are slot swaps, not redeployments over the live site.[31]. The new build is warmed and tested before it takes traffic, and reversing a release is a swap back rather than a rebuild.
Quality and reliability	Automated tests run before anything is deployed; the build fails on a warning; migrations are exercised on every run; smoke tests confirm the deployed build actually works before it is exposed to users.[31]
Auditability and marking	Every production build traces to a merge commit on main, with its pull request, its review, and its pipeline run retained in GitHub. The full source, including DBMS code, is in the repository.
Data protection	Real data exists only in production; secrets live in Key Vault and GitHub secrets rather than in the repository; a database export is taken as a restore point before each production release.

API Research

Internal - Azure Platform Built-In APIs

Because the QULOOBUL MOE'MIENEEN platform is hosted on Microsoft Azure with an ASP.NET Core Web API backend and Azure SQL Database, a large portion of the functionality needed is already provided by Azure's managed services at little or no extra cost. [44]. These should always be used before reaching for a third-party service.

Azure SQL Database API (via Entity Framework Core)

Purpose: The primary data store for the QULOOBUL MOE'MIENEEN platform. Entity Framework Core provides a type-safe C# API over Azure SQL Database - queries, inserts, updates, and deletes are written as LINQ expressions that EF Core translates to T-SQL. The team owns and version-controls the full schema, stored procedures, and migrations.

Pricing: Standard S0 (10 DTU): $14.72/month. Standard S1 (20 DTU): $29/month. Required for 24/7 uptime - the free serverless tier auto-pauses and is unsuitable for continuous operation.

Advantages	**Limitations**
+ Full T-SQL ownership - schema, procedures, and migrations are team-written source code, markable by the lecturer + EF Core scaffolding maps UML class diagrams directly to C# entity classes + Built-in automated backups: full weekly, differential daily, transaction log every 5-12 minutes + Row-Level Security available natively in SQL Server for multi-tenant data isolation + Deployed to South Africa North region for POPIA data residency compliance	- Not free for 24/7 use - requires provisioned S0/S1 tier ($14.72-$29/month) [44] - DTU-based tiers can be limiting under heavy concurrent load - monitor and upgrade to S2/S3 if needed

Key Edge Case: Never use the free serverless tier for a 24/7 system. It auto-pauses after idle and exhausts its 100,000 vCore-second monthly allowance within roughly 28 hours of continuous use, after which it bills at a rate higher than the provisioned S0 tier.

ASP.NET Core Web API

Purpose: The team-owned REST API backend. All business logic, data validation, and endpoint controllers are written in C# and version-controlled in GitHub. This is the layer that the web frontend, .NET MAUI mobile app, and any third-party integrations (invoicing, email, AI) communicate through.

Pricing: Free (.NET 10 SDK is open source under MIT). Hosting on Azure App Service Basic B1: Linux ~$13/month, Windows ~$55/month. Linux B1 is recommended for cost. [45]

Advantages	**Limitations**
+ All source code is C# - consistent with the team's GLMS/PROG7311 experience and markable by the lecturer + UML class diagrams scaffold directly into controller, service, and entity classes in Visual Studio + Handles role-based access control (RBAC) via [Authorize(Roles=...)] attributes on each endpoint + Single API serves both the web frontend and the .NET MAUI mobile client - no duplication + Middleware pipeline allows adding authentication, logging, rate limiting, and CORS in one place	- The team must write and maintain all endpoint logic - no auto-generated CRUD like Supabase PostgREST - Cold starts can occur on Basic B1 if the App Service idles - keep-alive ping recommended for 24/7 use

Key Edge Case: Keep the API stateless - store all session state in the JWT token or the database, never in memory. This ensures the API can be restarted or scaled without losing user sessions.

ASP.NET Core Identity / Azure Entra External ID (AD B2C)

Purpose: Authentication and role-based access control for Admin, Staff, and Imaamat user roles. ASP.NET Core Identity handles login, registration, password hashing (PBKDF2), and role assignment internally. Azure Entra External ID (formerly Azure AD B2C) is available if external identity providers (Google, Microsoft accounts) are needed.

Pricing: ASP.NET Core Identity: completely free, bundled with .NET 10. Azure Entra External ID: free up to 50,000 Monthly Active Users - more than sufficient for a QULOOBUL MOE'MIENEEN [46].

Advantages	Limitations
+ Password hashing (PBKDF2 with salt) is handled automatically - never stored in plain text + JWT bearer tokens issued for the .NET MAUI mobile client - short expiry limits damage from a stolen token + Role-based access enforced at the API layer via [Authorize] attributes + No third-party auth service required - everything lives in the team's own codebase	- Building a custom login/registration UI requires more frontend work than a drop-in auth service like Clerk - No enterprise SSO (SAML) without upgrading to Azure Entra ID Premium (~$6/user/month)

Key Edge Case: Store the JWT signing key in Azure Key Vault - never hard-code it in appsettings.json. If the signing key is exposed in the GitHub repo, all issued tokens can be forged.

Azure Blob Storage

Purpose: Object storage for task attachments, meeting files, and profile photos (UC-07). Files are uploaded from the ASP.NET Core API using the Azure.Storage.Blobs NuGet package and served to clients via time-limited SAS (Shared Access Signature) tokens.

Pricing: Free: 5 GB for the first 12 months. After: ~$0.02/GB/month (hot tier). Negligible at QULOOBUL MOE'MIENEEN scale. The .NET MAUI mobile client uses the same SAS-token upload flow as the web frontend. [47].

Advantages	Limitations
+ SAS tokens grant time-limited, scoped access to specific files - no public buckets needed + Same storage backend serves both the web app and the .NET MAUI mobile client + Azure.Storage.Blobs NuGet package provides a clean C# SDK + CDN integration available via Azure CDN or Azure Front Door for faster file delivery	- SAS token generation logic must be written in the ASP.NET Core API - adds code overhead vs. Supabase's built-in storage policies - Not suitable for video streaming at scale without Azure Media Services

Key Edge Case: Generate SAS tokens server-side in the ASP.NET Core API with the shortest acceptable expiry (e.g., 15 minutes for uploads). Never generate SAS tokens client-side - the storage account key must never leave the server.

Azure SignalR Service

Purpose: Real-time WebSocket communication for live kanban board updates (UC-02) and in-app notifications (UC-08). The ASP.NET Core backend uses the Microsoft.AspNetCore.SignalR NuGet package; the .NET MAUI mobile app uses the Microsoft.AspNetCore.SignalR.Client package - the same C# API on both platforms.

Pricing: Free tier: 20 concurrent connections, 20,000 messages/day - sufficient for the QULOOBUL MOE'MIENEEN's 24 staff users. Standard tier: ~$1.61/unit/month if the free tier is outgrown. [48]

Advantages	**Limitations**
+ Native C# SDK on both server (ASP.NET Core) and mobile (.NET MAUI) - no extra language or toolchain + Hub-based architecture allows broadcasting to specific user groups or roles + Automatic reconnection built into the client SDK + Free tier is adequate for the QULOOBUL MOE'MIENEEN's expected concurrent user count	- Free tier limited to 20 concurrent connections - upgrade needed if usage grows significantly - Requires Azure SignalR Service for production scale; local SignalR works for development only

Key Edge Case: Use SignalR for UI updates only - never trigger server-side operations (emails, database writes) from a SignalR message. Server-side workflows belong in the ASP.NET Core service layer, triggered by API calls or background jobs.

Azure Key Vault

Purpose: Secure centralised storage for all application secrets: database connection strings, JWT signing keys, email API keys, Blob Storage keys, and any invoicing API credentials. The ASP.NET Core API reads secrets at runtime via the Azure.Extensions.AspNetCore.Configuration.Secrets NuGet package.

Pricing: Standard tier: effectively free at under 10,000 operations/month - the QULOOBUL MOE'MIENEEN platform will use far fewer operations than this limit [49].

Advantages	Limitations
+ Secrets never appear in source code or configuration files - eliminates credential leaks via GitHub + Azure Managed Identity allows the App Service to authenticate to Key Vault without any stored credentials + Full audit log of every secret access - useful for POPIA compliance and security reviews	- Adds a step to local development setup - developers must configure local credentials to access Key Vault during development

Key Edge Case: Use a local .env file (excluded from git via .gitignore) during development, and Azure Key Vault in staging and production. Never commit any .env file or appsettings.Development.json containing real credentials to the repository.

Azure Notification Hubs

Purpose: Push notifications to the .NET MAUI mobile app when the app is in the background or closed - for example, when a task is assigned to a staff member or a request is approved. The ASP.NET Core API sends a push via the Azure Notification Hubs SDK; the mobile app registers its device token on startup.

Pricing: Free tier: up to 1 million push notifications/month. More than sufficient for any QULOOBUL MOE'MIENEEN use case [50].

Advantages	**Limitations**
+ Works for both Android (FCM) and iOS (APNs) from a single API call in the ASP.NET Core backend + No direct dependency on Firebase or Apple's push servers from the mobile app - routing handled by Azure + Free tier is generous for QULOOBUL MOE'MIENEEN scale	- iOS push notifications require an active Apple Developer Program subscription ($99/year) [51] - Device token management (registration, expiry, refresh) must be implemented in the mobile app

Key Edge Case: iOS push notifications require the App Service to have the APNs certificate or key stored in Azure Key Vault. Never store the APNs private key in the source repository.

External - Email and Notifications

Required for task assignment notifications, password resets, approval confirmations, and any invoicing or payment communications. The ASP.NET Core API sends emails by calling the chosen provider's REST API or SMTP endpoint from a background service or controller action. Azure Communication Services is the Azure-native option; Resend is the recommended third-party alternative for simplicity and cost.

Resend

Purpose: Developer-first transactional email API. Called from the ASP.NET Core backend using the Resend NuGet package or plain HttpClient. Email templates can be written as Razor templates (C# HTML) consistent with the rest of the .NET stack.

Pricing: Free: 3,000 emails/month (100/day cap). Paid: $20/month for 50,000 emails. $0.90 per 1,000 overage emails [52].

Advantages	**Limitations**
+ Simple REST API - easy to call from ASP.NET Core using HttpClient or the official NuGet package + Permanent free tier (not a time-limited trial) - suitable for QULOOBUL MOE'MIENEEN scale indefinitely + Clean webhook events for bounces, opens, and clicks - log these back to Azure SQL + Domain verification with SPF/DKIM improves deliverability for auth and notification emails	- Transactional only - no marketing campaign or bulk broadcast features - 100 emails/day cap on free tier - could be reached if the system sends many task notifications - Not an Azure-native product - one more external dependency vs. Azure Communication Services

Key Edge Case: Implement idempotency in the email-sending service: log every sent email (recipient, event type, timestamp) to an EmailLog table in Azure SQL and check before sending to prevent duplicate emails if the background job retries.

Azure Communication Services (ACS)

Purpose: Microsoft's Azure-native email, SMS, and push communication platform. Integrates directly with the Azure ecosystem - uses the same Managed Identity and Key Vault authentication as the rest of the stack. Called from the ASP.NET Core API using the Azure.Communication.Email NuGet package.

Pricing: Email: first 100 emails/day free permanently. Then $0.00025 per email (~$0.25 per 1,000). No monthly fee - pure pay-per-use [53].

Advantages	Limitations
+ Native Azure integration - same authentication model (Managed Identity) as Blob Storage and Key Vault + Keeps the entire stack within the Microsoft ecosystem - single vendor for support + Also supports SMS and WhatsApp messaging if the QULOOBUL MOE'MIENEEN needs multi-channel communications + 100 free emails/day is adequate for most QULOOBUL MOE'MIENEEN notification volumes	- Developer experience and documentation are not as polished as Resend [52] - Email template tooling is more basic than Resend [52] Has better tooling for HTML email rendering - SMS and WhatsApp features add cost if used

Key Edge Case: ACS email requires a custom domain and DNS verification (SPF, DKIM, DMARC). Allow 24-48 hours for DNS propagation when setting up; do not leave this for the day before a demo.

SendGrid (Twilio)

Purpose: Long-established transactional and marketing email platform, now owned by Twilio. Suited to high-volume sending. Called from ASP.NET Core via the SendGrid NuGet package (Twilio.SendGrid).

Pricing: No permanent free tier (removed May 2025). 60-day trial: 100 emails/day. Essentials: $19.95/month (50,000 emails). Pro: $89.95/month (dedicated IP) [54].

Advantages	Limitations
+ Battle-tested at very high volume - reliable infrastructure with strong SLA + Inbound email parse feature allows users to reply to notification emails to create task comments + Detailed analytics and event webhooks + HIPAA-compliant with Business Associate Agreement on Pro plan	- No permanent free tier - costs begin after the 60-day trial - Shared IP on Essentials - deliverability affected by neighbouring senders - Support quality has declined since the Twilio acquisition

Key Edge Case: If the invoicing module sends financial emails (invoices, receipts), deliverability is critical. Consider Postmark or the dedicated IP Pro tier if invoice emails must not land in spam.

External - AI & Intelligence Features

Optional but increasingly expected. AI can power task summarisation, natural language task creation, smart priority suggestions, and - relevant to the invoicing module - automated expense categorisation and invoice description generation. All AI calls are made from the ASP.NET Core backend, never directly from the client, to protect API keys.

Anthropic Claude API

Purpose: State-of-the-art language model for task summarisation, natural language commands, and document analysis. Called from the ASP.NET Core API via HttpClient. For the invoicing module, Claude can auto-generate invoice line item descriptions or categorise expenses from free-text input.

Pricing: Claude Sonnet 4: $3.00 per 1M input tokens, $15.00 per 1M output tokens. No monthly fee. A typical task summary or invoice description costs approximately $0.001 [55].

Advantages	Limitations
+ Best reasoning and writing quality for summarisation and natural language tasks + 200K context window - can analyse an entire project history or a set of invoices in one call + API data not used for model training by default - important for client confidentiality + Consistent and predictable output quality	- No free tier - charged from the first API call - 1-3 second latency must be handled with loading indicators in the UI - Rate limits on lower tiers may require a request queue for high-volume use

Key Edge Case: Cache AI-generated summaries and descriptions in Azure SQL against the source record's last-modified timestamp. Only regenerate when the underlying data changes - avoids unnecessary API cost.

OpenAI API

Purpose: GPT-4o and GPT-4o mini for AI-powered features. GPT-4o mini is fast and cheap enough for auto-tagging tasks and generating short invoice descriptions. Azure OpenAI Service (the Azure-hosted version) keeps data within Azure's compliance boundary, which may be preferable for financial data.

Pricing: GPT-4o: $2.50/1M input, $10.00/1M output. GPT-4o mini: $0.15/1M input, $0.60/1M output. Azure OpenAI Service: same models, same pricing, but data stays within Azure [56].

Advantages	**Limitations**
+ GPT-4o mini is extremely cost-effective for short, repetitive AI tasks + Azure OpenAI Service keeps all data within the Azure compliance boundary - good for financial use cases + Structured JSON output mode makes it easy to extract machine-readable data from AI responses + Function calling support for tool-use workflows in the ASP.NET Core backend	- No free tier - Azure OpenAI Service requires a separate application/approval process via the Azure portal - Output consistency can vary for long-form or nuanced writing tasks compared to Claude

Key Edge Case: Use Azure OpenAI Service (not api.openai.com) for any AI features that process client financial data. This keeps the data within the Azure South Africa North region and under the same POPIA-compliant data residency boundary as the rest of the system.

Google Gemini API

Purpose: Google's multimodal AI. Gemini 2.0 Flash has a generous free tier - ideal for prototyping AI features before committing to a paid provider. The free tier is rate-limited but sufficient for development and early demonstrations.

Pricing: Gemini 2.0 Flash-Lite: Free tier available (rate-limited). Paid: $0.075/1M input tokens, $0.30/1M output tokens [57].

Advantages	**Limitations**
+ Genuinely useful free tier for prototyping and development - no credit card needed initially + Very fast Flash model response times + Multimodal - can process images, useful for reading scanned invoices or receipts	- Free tier data may be used by Google for model improvement - not suitable for financial or personal client data - Less consistent output quality for writing and reasoning compared to Claude or GPT-4o - Moves the AI processing outside the Azure ecosystem

Key Edge Case: Use the free tier for internal development and prototyping only. Switch to Claude or Azure OpenAI Service before handling any real client or financial data in production.

External - Authentication Extensions

ASP.NET Core Identity handles the QULOOBUL MOE'MIENEEN's login requirements out of the box. External auth extensions are only needed if the client requests enterprise SSO (e.g., logging in with Microsoft 365 or Google Workspace accounts) or an pre-built auth UI. Do not add these unless specifically required - they introduce extra dependencies and potential cost.

Azure Entra ID (Active Directory) SSO

Purpose: If the QULOOBUL MOE'MIENEEN's staff already use Microsoft 365 accounts, Azure Entra ID allows them to log into the platform with their existing organisational credentials - no separate password needed. Integrates directly with ASP.NET Core Identity via the Microsoft.Identity.Web NuGet package.

Pricing: Azure Entra ID Free: supports SSO for up to 500,000 objects. Azure Entra ID P1: $7/user/month - required for Conditional Access and MFA enforcement. For an QULOOBUL MOE'MIENEEN with Microsoft for Nonprofits, Entra ID P1 may be available at a discounted rate [58].

Advantages	**Limitations**
+ Staff use their existing Microsoft 365 credentials - no new password to remember + Microsoft.Identity.Web NuGet package integrates cleanly with ASP.NET Core + Multi-factor authentication (MFA) enforceable at the directory level + Consistent with the Azure-native stack	- Only useful if the QULOOBUL MOE'MIENEEN already uses Microsoft 365; otherwise adds unnecessary complexity - P1 licence required for Conditional Access policies

Key Edge Case: If implementing Entra ID SSO, ensure the [Authorize] policies in ASP.NET Core still enforce the application roles (Admin, Staff, Imaamat) after a successful Entra login. SSO handles who the user is; the application layer still decides what they can do.

Clerk

Purpose: Drop-in authentication service with pre-built UI components for sign-in, sign-up, and user profile pages. An alternative to ASP.NET Core Identity rather than an extension. Provides organisation and team management out of the box.

Pricing: Free tier. Pro: $25/month (unlimited MAUs, advanced features) [59].

Advantages	Limitations
+ Pre-built, accessible auth UI - saves significant frontend development time + Organisation/team management built in - useful if the platform later serves multiple QULOOBUL MOE'MIENEEN chapters + Webhooks for all auth lifecycle events	- Replaces ASP.NET Core Identity entirely - the team loses ownership of the auth code, which may affect marking - Adds a third-party dependency and another monthly cost as the user base grows - Moves authentication outside the Azure ecosystem

Key Edge Case: Only consider Clerk if the client specifically requests a polished auth UI and the team cannot allocate time to build one. For the university project phase, ASP.NET Core Identity is sufficient and keeps all code within the team-owned, markable stack.

External - Invoicing & Accounting APIs (Not Confirmed)

If the QULOOBUL MOE'MIENEEN platform is extended to include an invoicing or basic accounting module - for example, to issue invoices to donors, track project expenses, or manage supplier payments - the following APIs provide the accounting engine. The ASP.NET Core backend would call these APIs to create invoices, sync contacts, and retrieve financial reports, while the QULOOBUL MOE'MIENEEN's finance team uses the accounting platform's own interface for bookkeeping. Note: full accounting software is complex - consider whether a lightweight invoice-only approach (Section 5.4) is more appropriate for Phase 1 than a full integration.

Xero API

Purpose: Cloud accounting platform with a comprehensive REST API. The ASP.NET Core backend can create invoices, manage contacts (donors, suppliers), reconcile payments, and pull financial reports directly from Xero's API using the Xero-NetStandard-OAuth2-App NuGet library. The QULOOBUL MOE'MIENEEN's accountant uses the Xero web interface for daily bookkeeping while the platform pushes and pulls data via the API.

Pricing: Xero Starter: R90 for the first 3 months, then R450/month (20 invoices, 5 bills). Xero Standard: R159 for the first 3 months, then R795/month (unlimited invoices). Xero Premium: R219 for the first 3 months, then R1095/month (multi-currency). Non-profit discount: Xero offers up to an 80% discount for registered QULOOBUL MOE'MIENEENs - verify eligibility at xero.com/ngo [60].

Advantages	**Limitations**
+ Official .NET SDK (Xero.NetStandard.OAuth2App) for clean ASP.NET Core integration + Full double-entry bookkeeping engine - handles VAT, income statements, balance sheets + Bank feed reconciliation - connects directly to South African bank accounts (FNB, ABSA, Standard Bank, Nedbank) + Registered QULOOBUL MOE'MIENEEN discount of up to 80% available through the Xero for Good programme + Widely used by South African accountants and bookkeepers - QULOOBUL MOE'MIENEEN's existing accountant likely already familiar + Comprehensive webhook support for payment events - trigger notifications	- Starter plan limited to 20 invoices and 5 bills/month - may be too low for active QULOOBUL MOE'MIENEENs - OAuth 2.0 authentication flow requires the QULOOBUL MOE'MIENEEN's Xero account owner to authorise the connection initially - API rate limits: 60 requests/minute, 5,000/day - sufficient for most QULOOBUL MOE'MIENEEN use but requires request queuing at higher volumes

from ASP.NET Core when an invoice is paid	

Key Edge Case: Xero's OAuth 2.0 access tokens expire every 30 minutes. Store the refresh token in Azure Key Vault (never in Azure SQL in plaintext) and implement automatic token refresh in the ASP.NET Core Xero service before every API call. A failed refresh means the integration goes offline until re-authorised by the Xero account owner [60].

Sage Business Cloud Accounting API

Purpose: Cloud accounting platform widely used in South Africa, with local VAT and tax compliance built in. The REST API allows the ASP.NET Core backend to create invoices, manage customers, and retrieve financial summaries. Sage has a strong presence in the South African SME and QULOOBUL MOE'MIENEEN market.

Pricing: Sage Business Cloud Accounting: from R240/month (Sage Accounting Start). QULOOBUL MOE'MIENEEN/charity pricing available - contact Sage directly. No permanent free tier [61].

Advantages	**Limitations**
+ Built specifically for South African tax compliance - VAT calculations, SARS-compliant invoicing, and ZAR as the base currency + Strong South African market presence - likely familiar to local accountants and bookkeepers + REST API with OAuth 2.0 - integrates with ASP.NET Core using standard HttpClient + Supports multi-entity and multi-currency for larger QULOOBUL MOE'MIENEENs	- API documentation is less comprehensive and developer-friendly than Xero's [60] - No official .NET SDK - requires manual HttpClient implementation in ASP.NET Core - Fewer third-party integrations and community resources compared to Xero - QULOOBUL MOE'MIENEEN pricing requires direct negotiation - not as transparent as Xero's [60] published QULOOBUL MOE'MIENEEN discount

Key Edge Case: Sage's API versioning changes periodically. Pin the API version in the ASP.NET Core HttpClient base URL (e.g. /v3.1/) and set up an alert or review when Sage announces version deprecations, to avoid the integration breaking silently.

QuickBooks Online API (Intuit)

Purpose: The world's most widely used small business accounting platform. Full REST API (QuickBooks Online API v3) for invoices, expenses, customers, suppliers, and financial reports. Intuit provides a .NET SDK (Intuit.Ipp.OAuth2PlatformClient) for ASP.NET Core integration.

Pricing: QuickBooks Online Simple Start: ~R96.60 for 6 months, then R322/month. Plus: ~R212,39 for 6 months, then R708/month. No permanent free tier - 30-day free trial only [62].

Advantages	**Limitations**
+ Official .NET SDK for ASP.NET Core integration + Extremely comprehensive API - invoices, expenses, payroll, tax, bank feeds + 50% QULOOBUL MOE'MIENEEN discount available via TechSoup (same channel as the Microsoft for Nonprofits grant) + Widely used globally - the QULOOBUL MOE'MIENEEN's auditor or accountant is likely familiar with QuickBooks	- Pricing is in USD - more expensive than Xero [60] at equivalent tiers - QuickBooks Online is less popular in South Africa compared to Xero and Sage - local accountant support may be limited - API OAuth flow is complex, and sandboxed testing requires a separate developer account - Less suited to South African VAT compliance out of the box compared to Sage

Key Edge Case: QuickBooks [62] Online's Intuit Developer Sandbox environment uses different OAuth credentials from production. Ensure the ASP.NET Core API uses environment-specific configuration (from Azure Key Vault) for sandbox vs. production credentials - never use production keys during development.

Invoice Ninja API

Purpose: Open-source invoicing platform with a full REST API. Can be self-hosted on Azure (a Docker container on Azure Container Instances) or used as a hosted SaaS. A good option if the QULOOBUL MOE'MIENEEN wants invoice generation without a full accounting package. The ASP.NET Core backend calls the Invoice Ninja API to create and send invoices.

Pricing: Self-hosted: free (open source). Hosted SaaS (invoiceninja.com): Free tier, Ninja Pro: ~$14/month, Enterprise: ~$18/month, Premium Business: from $280/year [63].

Advantages	**Limitations**

+ Full REST API for invoices, clients, payments, and recurring invoices + Self-hosted option keeps all financial data within the Azure South Africa North region - strong POPIA alignment + No per-invoice or per-user fees - flat cost regardless of volume + Supports ZAR currency and South African VAT rates	- Not a full accounting platform - no double-entry bookkeeping, no bank reconciliation, no SARS-compliant reports - Self-hosted version requires the team to manage Docker container updates and database backups - Smaller community and fewer South African integrations than Xero or Sage - If an accountant needs to review books, they cannot log into Invoice Ninja the way they would Xero

Key Edge Case: If self-hosting Invoice Ninja [63] on Azure, store the invoice database on Azure SQL (not the default SQLite bundled with the Docker image) so it benefits from the same automated backup and high-availability setup as the rest of the platform's data.

Stripe Invoicing / Billing API

Purpose: If the QULOOBUL MOE'MIENEEN needs to collect payments online alongside issuing invoices - for example, donor contributions, event fees, or service charges - Stripe combines invoice generation and payment collection in one API. The ASP.NET Core backend uses the Stripe.net NuGet package. Not a full accounting system, but it handles the invoice-to-payment-received workflow cleanly.

Pricing: Stripe Invoicing: free to create invoices. 0.4% fee on paid invoices (capped at $2 per invoice) on Starter and 0.5% fee on paid invoices on Plus. Payment processing: 2.9% + $0.30 per card transaction. Note: Stripe is not yet available as a direct merchant account in South Africa - ZAR payouts require a Stripe Atlas company or integration with a local payment gateway [64].

Advantages	**Limitations**
+ Combines invoice creation and payment collection in one workflow + Stripe.net NuGet package provides a clean C# SDK for ASP.NET Core + Automatic payment reminders, PDF invoice generation, and payment receipts + Webhook events for invoice.paid, invoice.payment_failed - trigger status updates in Azure SQL	- Not a full accounting system - no bookkeeping, expense tracking, or SARS-compliant reports - Stripe is not directly available for South African merchant accounts - ZAR payouts require workarounds - For donor or event payments, PayFast or Peach Payments (South African payment gateways) may be more practical - Per-invoice and per-transaction fees add up at higher volumes

Key Edge Case: For South African payment collection, consider PayFast (payfast.co.za) or Peach Payments (peachpayments.com) as local alternatives to Stripe [64]. Both have ASP.NET Core-compatible REST APIs and support ZAR payments without requiring offshore merchant account setup.

Consolidated Cost Summary

All services covered in this report, updated for the Azure / ASP.NET Core / .NET MAUI stack. USD figures should be verified in the Azure Pricing Calculator before submission [65]. Exchange rate used: approximately R16.50 to the dollar.

Service	Category	Free Tier?	Entry Paid Cost	Notes
Azure SQL Database	Internal	No (24/7 requires paid)	$14.72/month (S0) [44]	Provisioned tier required for 24/7 uptime
Azure App Service B1	Internal	No (F1 too limited)	-$13/month (Linux B1) [45]	Hosts ASP .NET Core Web API
Azure Static Web Apps	Internal	Yes - Fully free	$0 [66]	Hosts frontend with CI/CD
Azure Blob Storage	Internal	Yes - 5GB/12 months	- $0.02/GB/month after [47]	File and media storage
Azure SignalR Service	Internal	Yes - 20 connections	- $1.61/unit/month [48]	Real-time UI updates
ASP .NET Core Identity	Internal	Yes - Free	$0	Auth and RBAC built into .NET 10
Azure Key Vault	Internal	Yes - effectively free	Negligible [49]	Secrets management
Azure Notifications Hub	Internal	Yes - 1M pushes/month	$0 for QULOOBUL MOE'MIENEEN scale [50]	Mobile push notifications
Resend (Email)	Email	Yes - 3000/month	$20/month (50k emails) [52]	Recommended email provider
Azure Comm. Services	Email	Yes - 100/day	$0.00025/email after [53]	Azure-native email alternative
SendGrid	Email	No - 60-day trial only	$19.95/month (Essentials) [54]	Enterprise scale; no permanent free tier
Claude API (Anthropic)	AI	No	-$0.001 per task/invoice [55]	Best quality; pay per use
Azure OpenAI Service	AI	No	Same as OpenAI pricing [56]	Azure-native; data stays in Azure region
Google Gemini	AI	Yes - rate-limited	$0.075/1M tokens [57]	Dev/prototyping only

Azure Entra ID SSO	Auth	Yes (Free tier)	$7/user/month (P1) [58] [46]	Only if QULOOBUL MOE'MIENEEN uses Microsoft 365
Xero API	Invoicing	Yes - 25 invoices/month	R90 for first 3 months then R450/month (Starter) [60]	QULOOBUL MOE'MIENEEN discount up to 80% available
Sage Business Cloud	Invoicing	No	From R240/month [61]	South Africa-native; ZAR & VAT compliant
QuickBooks Online	Invoicing	No (30-day trial)	From R96.60 to R708/month [62]	50% QULOOBUL MOE'MIENEEN discount via TechSoup
Invoice Ninja	Invoicing	Yes - self-hosted free	$0-20/month hosted [63]	Open source; self-host on Azure
Stripe Invoicing	Invoicing	Free to create invoices	0.4% on paid invoices [64]	Not directly available in SA - use PayFast instead

Recommendation Summary

The following stack is recommended for the QULOOBUL MOE'MIENEEN platform, updated to reflect the Azure / ASP.NET Core / .NET MAUI architecture. The invoicing recommendation assumes the QULOOBUL MOE'MIENEEN proceeds with an invoice module; if not, the invoicing row can be excluded entirely.

Database	Azure SQL Database (S0/S1) + EF Core	Team-owned T-SQL schema. 24/7 always-on. POPIA-compliant via South Africa North region.
Backend API	ASP .NET Core Web API (.NET 10)	All business logic in C#. Markable source code. Serves web and .NET MAUI mobile clients.
Auth	ASP .NET Core Identity	Built-in to .NET 10. Password hashing, JWT tokens, RBAC. Add Azure Entra SSO only if QULOOBUL MOE'MIENEEN uses Microsoft 365.
File Storage	Azure Blob Storage	SAS-token access control. Same storage for web and mobile. Free for 12 months.
Real-time	Azure SignalR Service	C# SDK on both server and mobile. Free tier sufficient for QULOOBUL MOE'MIENEEN scale.
Email	Resend	Permanent free tier. Simple REST API. Implement an idempotency log in Azure SQL.
AI (Optional)	Anthropic Claude API	Best quality for task summaries and invoice descriptions. Use Gemini free tier for prototyping.
Invoicing (if needed)	Xero API	Best QULOOBUL MOE'MIENEEN discount (up to 80%). Official .NET SDK. South African bank feeds. Familiar to SA accountants.
Invoicing (SA-native alt.)	Sage Business Cloud	South African VAT compliance built in.

		Consider if QULOOBUL MOE'MIENEEN's accountant already uses Sage.
Invoicing (lightweight)	Invoice Ninja (self-hosted)	Free open source. Self-host on Azure to keep financial data in South Africa North region.
Mobile	.NET MAUI	Single C# codebase for Android + iOS. Reuses ASP.NET Core DTOs. No second language needed.
Secrets	Azure Key Vault	Store all API keys, DB connection strings, OAuth tokens. Never commit to GitHub.

All pricing data accurate as of July 2026. Verify current pricing on each provider's official website and the Azure Pricing Calculator [65] before finalising budget estimates. QULOOBUL MOE'MIENEEN discounts (Xero for Good [60], QuickBooks [62] via TechSoup, Microsoft for Nonprofits [67]) require separate applications and verification - allow lead time before project go-live. This document is for internal project use only.

System Security

Network and Infrastructure Security

Because the platform runs entirely on Microsoft Azure, much of the network-level security comes from Azure's built-in infrastructure. However, the team still needs to configure these correctly - having a managed cloud service does not automatically mean the application is secure.

Transport Layer Security (TLS/HTTPS)

All communication between clients (browser, mobile app) and the server must be encrypted. Without this, anyone on the same network - for example, on a shared Wi-Fi connection at a community centre - could read the data being sent, including login credentials and personal information.

- Azure App Service is configured to enforce HTTPS redirection. Any request made over plain HTTP is automatically redirected to HTTPS, so it is not possible to accidentally use the API without encryption [45].
- Azure SQL Database [44] rejects all unencrypted connection attempts by default. The connection string used by the API enforces TLS 1.2 as the minimum version.
- The .NET MAUI mobile app communicates with the API over HTTPS only. There are no plain HTTP endpoints exposed by the backend.

Firewall and Network Access Control

The database is the most sensitive part of the system - it holds all the QULOOBUL MOE'MIENEEN's records. Restricting who can reach it at the network level is the first line of defence against attacks.

- Azure SQL Database [44] Firewall rules are configured to only accept connections from the App Service's known outbound IP address range. No other IP address - including the developers' machines can connect to the production database directly.
- During development, individual developer IPs are temporarily whitelisted in the firewall rules and removed before the project goes live.
- The API itself is the only way for any user to interact with data. The frontend never talks to the database directly.

Secrets and Credential Management

A common mistake in student projects is committing connection strings, API keys, or passwords directly into the GitHub repository. This is a serious vulnerability - GitHub repos are often public, and even private repos can be compromised.

- All secrets (database connection string, JWT signing key, email API key, Azure Blob Storage key) are stored in Azure Key Vault.
- The application reads these values at runtime via environment variables injected by the App Service configuration. Nothing sensitive exists in the source code or in any configuration file committed to the repo.
- GitHub secret scanning is enabled on the repository to alert the team if a credential is accidentally committed.

Security Threat Analysis and Mitigation

The table below identifies the main security threats relevant to this system, explains what each one means in practice, and sets out the specific mitigation strategy the team is applying. The threats are drawn from recognised frameworks - primarily the OWASP Top 10, which is the industry-standard reference for web application security vulnerabilities, adapted to the specifics of this project's stack and context [68].

Threat/Vulnerability	Description	Mitigation Strategy
Unauthorised access/broken access control	An attacker or unauthorised staff member accesses data or functions they should not have permission to use.	Role-based access control (RBAC) enforced in ASP.NET Core Identity. Three roles defined: Admin, Staff, Imaamat. Each API endpoint is decorated with [Authorize(Roles=...)] so the server rejects requests that do not match the required role - even if someone manually crafts an HTTP request.
Insecure data transmission (man-in-the-middle)	Network traffic between the client and server is intercepted and read or modified by a third party.	HTTPS/TLS 1.2 enforced on all connections. Azure App Service [45] automatically redirects HTTP to HTTPS. Azure SQL Database [44] rejects unencrypted connections. All API keys and secrets are transmitted via secure headers only.

Weak or compromised authentication (credential theft)	A user's password is guessed, stolen via phishing, or cracked if stored insecurely.	Passwords are never stored in plain text. ASP.NET Core Identity hashes passwords using PBKDF2 with a salt before storing them in the database. JWT tokens are used for the mobile client with short expiry times to limit the damage of a stolen token.
SQL injection	Malicious SQL is embedded in user input and executed by the database, allowing data theft or destruction.	Entity Framework Core uses parameterised queries by default, which prevents raw SQL from being injected through user input. Direct string concatenation into SQL queries is not used anywhere in the codebase.
Insecure direct object references (IDOR)	A user modifies a URL or request to access another user's data by changing an ID value.	All data access goes through server-side authorisation checks. A staff member cannot retrieve records belonging to another user - the API verifies the requesting user's identity against the resource owner on every request.
Sensitive data exposure	Personal information (names, contact details, task records) is stored or transmitted without adequate protection.	Encryption at rest via Azure SQL Transparent Data Encryption (TDE) - enabled by default. Encryption in transit via TLS on all connections. Only the minimum required data is collected in line with the POPIA data minimisation principle.
Cross-Site Scripting (XSS)	An attacker injects malicious scripts into the web frontend that execute in other users' browsers.	The frontend framework (Blazor/React) escapes user-supplied content before rendering it to the DOM. Content Security Policy (CSP) headers are configured on the App Service to restrict which scripts can execute in the browser.
Misconfigured cloud permissions/exposed secrets	Azure credentials, connection strings, or API keys are accidentally committed to GitHub or misconfigured in the portal.	All secrets stored in Azure Key Vault and injected at runtime via environment variables. The GitHub repository has branch protection and secret scanning enabled. No credentials are hard-coded in any source file.

POPIA noncompliance (data privacy)	The system collects, stores, or shares personal information in ways that violate South African data protection law.	Data minimisation - only the fields genuinely needed are collected. Personal data stored in the South Africa North Azure region (local jurisdiction). Consent is documented. Users can request data deletion.
Denial of Service (DoS)/resource exhaustion	An attacker floods the API with requests, making the system unavailable to legitimate users.	Azure App Service [45] has built-in rate limiting and can be scaled under load. Azure Front Door or API Management can be added as a layer with request throttling if needed. Application-level rate limiting middleware can be added to the ASP.NET Core pipeline.

Security Design Principles

The mitigations listed above are not a random collection of fixes - they are guided by a set of established security design principles. Applying these principles consistently means that even threats that were not specifically anticipated are less likely to cause damage, because the system is designed to resist them by default [69].

Principle	How it is applied in this project
Defence in depth	Security is applied at multiple layers - the network (TLS, Azure firewall), the application (RBAC, input validation), and the database (RLS, TDE). An attacker who bypasses one layer still faces the others.
Least privilege	Each user role only has access to the functions and data it genuinely needs. API endpoints are locked to specific roles. The database service account used by the API has only read/write permissions - not schema owner or admin rights.
Fail securely	If an authentication check fails, the API returns a 401 Unauthorized response and does not expose any data. Error messages shown to the user never reveal internal stack traces or database details.
Secure by default	TLS is enforced by default on App Service. TDE is enabled by default on Azure SQL. ASP.NET Core Identity hashes passwords without any extra configuration needed. The developer must actively turn security off, not on.

Data minimisation (POPIA)	Only data that is needed for the system to function is collected. Unnecessary personal fields are not stored. This reduces the risk and legal exposure if a breach occurs.
Separation of concerns	The frontend, API, and database are separate layers. The frontend never talks to the database directly - it only calls the API, which validates requests before touching any data. This limits the blast radius of any single compromise.

POPIA Compliance

The Protection of Personal Information Act (Act 4 of 2013) applies to this system because it stores and processes personal information about identifiable South African individuals - specifically, the QULOOBUL MOE'MIENEEN's staff members and volunteers. The following measures are in place to ensure compliance [70].

Data Minimisation

The system only collects personal information that is genuinely needed for it to function. For example, the system records a staff member's name, email address, and assigned role. It does not collect ID numbers, physical addresses, or any other information beyond what is operationally necessary [71].

Data Residency

All personal data is stored in the South Africa North Azure region. This keeps the data within South African borders and under South African legal jurisdiction, which is a POPIA requirement for personal information processed by South African organisations [72].

Encryption

Personal data is encrypted both at rest (Azure SQL Transparent Data Encryption, enabled by default) and in transit (TLS 1.2 enforced on all connections). This means that even if an attacker were to gain access to the physical storage media or intercept network traffic, the data would be unreadable without the encryption key [73].

Consent and Documentation

Users are informed about what data is collected and why before the system captures their information. The QULOOBUL MOE'MIENEEN is responsible for maintaining a record of consent in line with POPIA's accountability requirements. The system is designed to support data deletion requests - a user's records can be removed from the database if they withdraw consent or leave the organisation.

Breach Response

Azure Monitor and App Service health alerts are configured to notify the team of unusual activity. In the event of a suspected data breach, POPIA requires the Information Regulator and affected parties to be notified within a reasonable time. The QULOOBUL MOE'MIENEEN should have a documented incident response procedure in place before the system goes live.

Risk Matrix

The risk matrix below covers all 10 identified project risks, including the security and compliance risks discussed in this document. Each risk is assigned an impact score (15), a probability score (1-5), and a combined risk rating (impact × probability). Risk IDs R04 and R-07 are the primary security risks.

GREEN: Low Risk (1-6) -> Accept or Monitor

ORANGE: Medium Risk (7-14) -> Monitor and Mitigate

RED: High Risk (15-25) -> Immediate mitigation required

ID	Category	Risk	Description	Impact (1-5)	Probability (1-5)	Rating	Mitigation
R-01	Usability	Transition from previous system	Transition from previous system causes user friction, delays, and miscommu nication	4	3	**12**	A User Manual will be created and distributed before go-live to guide staff and volunteers through the new system.
R-02	Data	Data corruption or loss	Data corruption or loss due to server failure or system error	4	2	**8**	Automated cloud backups via Azure SQL built-in backup (full weekly, differential daily, transaction log every 5-12 minutes). No manual

							interventio n needed.
R-03	Commun ication	Staff miss notificati ons	Staff miss notifications or updates compared to previous system	4	4	**16**	Automated email notification system via Resend API and Azure Notificatio n Hubs for push alerts on the mobile app.
R-04	Security	Unautho rised access to data	Unauthorised access to data by non-authorised staff or external parties	5	2	**10**	Role-based access control (RBAC), HTTPS/TLS enforceme nt, password hashing (ASP.NET Core Identity), and Azure SQL firewall rules.
R-05	Availability	System downtime e	System downtime during critical events or operations	4	2	**8**	Hosting on Azure App Service with health monitoring and uptime alerts. Planned maintenance windows communic ated in advance.

R-06	Functional	Double booking or scheduling	Double booking or scheduling conflicts not caught by the system	3	3	**9**	Server-side validation on all bookings with unique constraint enforcement at the database level.
R-07	Compliance	Violation of POPIA	Violation of the Protection of Personal Information Act (POPIA)	4	2	**8**	Data minimisation principle applied. Consent documented. Data stored in the South Africa North Azure region. Encryption at rest and in transit enforced.
R-08	Quality	Insufficient testing	Insufficient testing leading to bugs reaching production	3	3	**9**	Unit testing, integration testing, and automated CI/CD testing pipelines via GitHub Actions on every pull request.

R-09	Financial	Cost overruns	Cost overruns or	3	2	**6**	Azure cost monitoring and budget alerts
			mismanaged resources				configured. Microsoft for Nonprofits grant in place. Infrastructure costs tracked monthly.
R-10	Technical	Scope creep	Scope creep - requirements expanding beyond what the system can deliver	4	4	**16**	Full documentation, sprint planning, and a fixed requirements baseline maintained from the start. Change requests go through a formal review process.

Note: Impact and probability values are scored 1-5. Risk Rating = Impact × Probability. Ratings of 15 or above are considered high priority and require immediate mitigation action.

Summary

The QULOOBUL MOE'MIENEEN platform has been designed with security as a core concern rather than an afterthought. The main threats to the system - unauthorised access, data interception, weak authentication, SQL injection, sensitive data exposure, and POPIA non-compliance - each have a specific, implemented mitigation strategy built into the stack.

The security approach is layered: network-level controls (TLS, firewall rules) reduce the attack surface before a request even reaches the application, application-level controls (RBAC, input validation, parameterised queries) protect the logic and data layer, and infrastructure-level controls (Azure Key Vault, TDE, Key Vault-backed secrets) protect credentials and stored data. No single point of failure exposes the whole system.

From a compliance standpoint, the system meets the core POPIA requirements - data minimisation, local data residency, encryption, and support for consent management and deletion requests. The team is aware that POPIA compliance is an ongoing responsibility of the QULOOBUL MOE'MIENEEN after handover, not just a one-time technical setup.

Technical & Financial Feasibility

This document addresses Item 4 of the XBCAD7315 MO2 Deliverable - the Technical and Financial Feasibility of the QULOOBUL MOE'MIENEEN platform. It evaluates all hardware, software, network, and database resources needed to build and operate the system and sets out all additional and ongoing costs the QULOOBUL MOE'MIENEEN/NGO client must be aware of, with each cost properly explained, motivated, and validated against the recommended technology stack.

The recommended stack - Azure SQL Database [44], ASP.NET Core Web API, and .NET MAUI for the mobile client - was selected on three grounds: the client's requirement for a cloud-based database, the module's requirement for team-owned, markable DBMS and backend source code, and alignment with the team's existing C#/.NET experience from the GLMS/PROG7311 module.

The client has stipulated that the platform must remain online 24 hours a day. This requirement materially affects cost because Azure's free and serverless tiers are designed for intermittent use - a resource that never idles receives no benefit from them. All cost figures in this document reflect the provisioned, always-on tiers required for 24/7 continuous operation.

2. Technical Feasibility

This section looks at whether the team has access to everything needed from a technical standpoint - hardware, software, network infrastructure, a suitable database setup, and the mobile-specific requirements introduced by the .NET MAUI client.

2.1 Hardware

Because the whole backend runs on Azure as a managed service, the team does not need to buy or set up any physical servers. The only hardware requirements are developer machines (which the team already has) and a mobile device for testing the MAUI app. The table below breaks this down.

Component	Requirement	Specification	Justification
Development Machines	Per developer	Modern PC/laptop - minimum 8 GB RAM, 256 GB SSD, dual-core CPU (2015 or newer)	Visual Studio 2022 + .NET 10 SDK; SQL Server Management Studio (SSMS) are resource-intensive IDEs that perform poorly below this threshold.
CI/CD Server	GitHub Actions (cloud-hosted)	Provided by GitHub - no on-premise hardware required	Automated build, test, and deploy pipelines run on GitHub-hosted runners; no team hardware needed.
Application Server	Azure App Service Basic B1 (cloud) [45]	1 vCore, 1.75 GB RAM, shared NVMe SSD - PaaS managed by Microsoft	Hosts the ASP.NET Core Web API 24/7 without the team managing OS patches, networking, or physical hardware.
Database Server	Azure SQL Database Standard S1 (cloud)	20 DTUs, 250 GB storage - fully managed by Microsoft [44]	Managed relational cloud database; no on-premise SQL Server instance required.
File Storage	Azure Blob Storage (cloud)	Hot-access tier; pay-as-you-go (first 5 GB free for 12 months)	Stores task attachments, meeting files, and profile photos (UC-07). No local NAS or file server required.
Mobile Devices (testing)	Android and/or iOS test devices	Android 10+ or iOS 15+ smartphone	Required to test the .NET MAUI mobile app in a real device environment before distribution.

2.2 Software

The full stack is either free, open source, or covered by the Microsoft for Nonprofits [67] grant. The table below lists everything needed for development, deployment, and production operation.

Software	Version / Tier	License Cost	Purpose
.NET 10 SDK	10.x (LTS)	Free (MIT)	Runtime and SDK for ASP.NET Core Web API and .NET MAUI mobile app.
Visual Studio 2022	Community / Professional	Free (Community)	Primary IDE; UML class diagram forward-engineering, EF Core scaffolding, MAUI workload.
Azure SQL Database	Standard S0/S1 DTU tier	$14.72-$29/month [44]	Cloud-managed relational DBMS. Full T-SQL schema, stored procedures, and EF Core migrations owned by the team.
Azure App Service	Basic B1 (Windows or Linux)	$13-$55/month [45]	Hosts the ASP.NET Core Web API with 24/7 uptime; auto-scaling, HTTPS, custom domain support.
Azure Blob Storage	Hot tier	Free (5 GB, 12 months) then ~$0.02/GB/month [47]	File and media storage for UC-07 (task attachments, meeting files, profile photos).
Azure SignalR Service	Free tier	Free (≤ 20 concurrent connections) [48]	Real-time kanban board updates and in-app notifications (UC-02, UC-08).
Azure Static Web Apps	Free tier	Free [66]	Hosts the Blazor or React frontend with GitHub CI/CD

			and HTTPS out of the box.
ASP.NET Core Identity / Azure AD B2C	Bundled with .NET / Free tier	Free [46]	Role-based authentication (Admin, Staff, Imaamat). JWT bearer tokens for the mobile client.
Entity Framework Core	10.x (bundled with .NET)	Free (MIT)	ORM layer; UML entities scaffold directly into C# model classes and EF migrations.
Azure Key Vault	Standard tier	Free (≤ 10,000 operations/month) [49]	Secure storage for connection strings and API keys; never hard-coded in source.
Resend / Azure Communication Services	Free tier / Pay-as-you-go	Free tier (3,000 emails/month - Resend) [52]	Transactional email for task assignments, request approvals, and password resets.
.NET MAUI (mobile)	Bundled with .NET 10 SDK	Free	Cross-platform mobile client (Android + iOS) sharing C# DTOs with the existing ASP.NET Core API.
SQLite (mobile offline cache)	sqlite-net-pcl	Free (MIT)	Local read cache and write queue on the mobile device for offline operation.
Azure Notification Hubs	Free tier	Free (≤ 1M push notifications/month) [50]	Background push alerts when the mobile app is closed (task assigned, request approved).
GitHub (source control & CI/CD)	Free / Team plan	Free (public/student repos)	Version control, pull-request

			workflow, and automated build/test/deploy pipelines via GitHub Actions.
SQL Server Management Studio (SSMS)	19.x	Free	Local SQL query tool used during development; not deployed to production.
Azure Communication Services (ACS)	Free Tier	Free (100 emails/day permanently) [53]	Azure-native email alternative to Resend [52]. Uses the same Managed Identity authentication as Blob Storage and Key Vault. Option if the team prefers to keep all services within the Microsoft ecosystem.
Cloudflare Pages	Free Tier	Free	Alternative to Azure Static Web Apps for frontend hosting. Supports GitHub CI/CD and HTTPS. Use if the frontend is fully decoupled from the Azure backend.

2.3 Network Requirements

No specialised network equipment is needed. Standard internet access is sufficient for development and for end users of the system. All security and encryption are handled by Azure's infrastructure. The key network requirements are listed below.

Requirement	Specification	Detail
Internet Connectivity	Stable broadband (≥ 10 Mbps)	Required by all developers for Azure portal access, cloud deployments, GitHub pushes, and real-time testing via Azure SignalR Service [48].
HTTPS / TLS	TLS 1.2 minimum	Enforced end-to-end: Azure App Service [45] enforces HTTPS redirection; Azure SQL Database enforces encrypted connections; Azure Blob Storage uses HTTPS-only SAS tokens.
Azure Virtual Network (optional)	VNet integration - Basic tier	Can isolate the App Service and Azure SQL within a private VNet for POPIA data residency compliance; not required for Phase 1 unless QULOOBUL MOE'MIENEEN policy demands it.
API Firewall / CORS	App Service built-in CORS policy	The ASP.NET Core Web API restricts cross-origin requests to the known frontend origin (Azure Static Web Apps URL), preventing unauthorised browser clients.
Database Firewall	Azure SQL firewall rules	Connections to Azure SQL Database are restricted to the App Service outbound IP address range; direct internet access is blocked.
DNS & Custom Domain	Azure-provided subdomain or custom domain	Azure Static Web Apps and App Service both provide managed SSL certificates for custom domains at no extra cost on the Basic tier.

2.4 Database Resources

The project uses Azure SQL Database [44] - a fully managed cloud relational database running on Microsoft's infrastructure. The team owns and writes all the T-SQL code (schema, stored procedures, views, EF Core migrations), which satisfies the DBMS marking requirement. The table below covers the key decisions around the database setup.

Aspect	Choice	Justification
DBMS	Azure SQL Database (T-SQL)	Fully managed PaaS relational database on Microsoft Azure. Meets the client's cloud-based database requirement. Full T-SQL - the schema, stored procedures, indexes, and triggers are all team-written source code handed to the lecturer/moderator for DBMS marking.
Pricing tier (24/7 requirement)	Standard S0 (10 DTU) or S1 (20 DTU)	The free serverless tier is unsuitable for 24/7 uptime - it auto-pauses after idle and exhausts its free compute allowance within ~28 hours of continuous use. S0/S1 provisioned DTU tiers are cheaper under constant load: $14.72/month (S0) or $29/month (S1).
ORM	Entity Framework Core 8	Scaffold UML class-diagram entities directly into C# model classes and generate schema migrations from code - satisfying the module's forward-engineering hint. The same EF Core toolchain is used in the GLMS/PROG7311 module.
Data residency	Azure South Africa North region	Data stored in the South Africa North Azure region for POPIA compliance -

		personal data remains within South African jurisdiction.
Access control	SQL Server Row-Level Security + App roles	Row-Level Security can restrict data visibility per-user at the database layer. Application roles (Admin, Staff, Imaamat) enforced in ASP.NET Core Identity service layer.
Backups	Azure SQL automated backups	Full backups weekly, differential daily, transaction log backups every 5-12 minutes. Included in all Azure SQL service tiers at no extra cost.

2.5 Compliance & Security

POPIA (Protection of Personal Information Act, South Africa): The platform stores personal data - names, email addresses, task content, and meeting records. This data must be processed lawfully and stored securely from the date of first use. Azure SQL Database and Blob Storage both support encryption at rest by default [70].

Data Residency: Azure SQL Database [44] is deployed to the South Africa North region, keeping all personal data within South African jurisdiction in compliance with POPIA [72].

Transport Security: TLS 1.2 is enforced on all connections - App Service enforces HTTPS redirection, and Azure SQL Database rejects unencrypted connections.

Secrets Management: All connection strings, API keys, and credentials are stored in Azure Key Vault and injected at runtime. No secrets are hard-coded in source code.

Data Security: Azure SQL Database [44] Encrypts data at rest by default using Transparent Data Encryption (TDE).

2.6 Mobile-Specific Technical Requirements

The mobile client is built with .NET MAUI, which produces a single C# codebase that compiles to both Android and iOS. Because the mobile app is just another REST client of the same ASP.NET Core Web API, it does not change the backend, database, or hosting setup. The following are the additional technical considerations specific to the mobile layer, drawn from the project's mobile addendum.

- Auth on device: JWT bearer tokens are issued by the ASP.NET Core Identity service and stored on the device using .NET MAUI SecureStorage - an encrypted key-value store that uses the platform's native secure storage (Android Keystore, iOS Keychain). Tokens are never written to plain text files or shared preferences.
- Offline support: a local SQLite cache (via the sqlite-net-pcl NuGet package) allows staff to view tasks and records without a network connection. Any write operations attempted offline are queued and synced automatically when connectivity is restored. This is the one piece of the mobile layer with no direct web equivalent.
- Real-time on mobile: Azure SignalR Service already has a first-party .NET MAUI-compatible client SDK. Live kanban updates and in-app notifications work identically on mobile as on the web frontend - no separate real-time service is needed.
- File capture: .NET MAUI's MediaPicker and Essentials APIs give the app camera and gallery access for task attachments (UC-07). Captured files are uploaded to the same Azure Blob Storage account used by the web app, via the same SAS token flow.
- Distribution - Android: requires a Google Play Developer account ($25 one-time registration fee) [74]. For the university project phase, the app can be side-loaded via APK without a Play Store listing.
- Distribution - iOS: requires an active Apple Developer Program [51] subscription ($99/year). This is also required for TestFlight distribution during testing. iOS side-loading without a developer account is not supported on stock devices.
- Mac required for iOS builds: building and signing the iOS version of a .NET MAUI app requires a Mac machine running Xcode. This is a hardware requirement the team should confirm before committing to iOS delivery.

3. Financial Feasibility

Financial feasibility addresses all additional and ongoing costs the QULOOBUL MOE'MIENEEN/NGO client will need to be aware of - including hardware, software, licensing, hosting, support, and upgrades. All costs are explained, justified, and validated against the chosen stack and the client's 24/7 uptime requirement.

3.1 Phase 1 Infrastructure Costs (24/7 Uptime)

The client's requirement for continuous 24/7 availability means both compute-bound components - the database and the API host - must run on provisioned, always-on pricing tiers. The free and serverless tiers offered by Azure are not suitable for

continuous operation because they auto-pause during idle periods and incur higher per-second costs when running continuously.

The table below sets out every cost component for Phase 1 (the university project period), with monthly and annual estimates validated against the Azure Pricing Calculator for the South Africa North region.

Component	Monthly (USD)	Annual (USD)	Notes
Azure SQL Database - Standard S0 (10 DTU) [44]	$14.72	$176.64	Cheapest always-on tier. Upgrade to S1 if performance is slow under load.
Azure App Service - Linux [45]	~$13.14	~$157.68	Linux B1 is the most cost-effective 24/7 hosting option. Windows B1 costs ~$54.75/month if required.
Azure Static Web Apps [66]	$0	$0	Frontend hosting is completely free - includes CI/CD and SSL.
Azure Blob Storage [47]	$0 (first 12 months)	$0 (year 1)	5 GB included free. After that roughly $0.02/GB/month - minimal at QULOOBUL MOE'MIENEEN scale.
Azure SignalR Service [48]	$0	$0	Free tier supports up to 20 concurrent connections, which is sufficient for this project.
Azure AD B2C / Entra External ID [46]	$0	$0	Free up to 50 000 monthly active users.
Azure Key Vault [49]	$0	$0	Effectively free under 10 000 operations/month.
Resend (email notifications) [52]	$0	$0	3 000 emails/month on the permanent free tier.

Azure Notification Hubs [50]	$0	$0	1 million push notifications/month on the free tier.
GitHub	$0	$0	Free for student/academic accounts and public repos.
Google Play Developer [74]	$0	$25 (one-time)	One-time registration fee to publish the Android app.
Apple Developer Program [51]	$0/month	$99/year	Required to publish to the iOS App Store or use TestFlight.
Estimated total (Linux B1 + SQL S0)	**~$45/month**	**~$540/year**	Covered by the Microsoft for Nonprofits Azure credit ($2 000/year ≈ $166/month) [67].

Note: USD figures above are based on the Azure Pricing Calculator [65] for the South Africa North region at the time of writing. The exchange rate used is approximately R16.50 to the dollar. Confirm the latest figures on the Azure calculator before finalising any budget with the QULOOBUL MOE'MIENEEN.

3.2 Microsoft for Nonprofits Azure Credit

The client already holds the Microsoft for Nonprofits [67] Azure credit, which provides up to $2 000 per year in Azure credits (roughly $166 per month, or about R2 739/month at R16.50 to the dollar). At the estimated cost of $45-$90/month for Phase 1, the grant more than covers all Azure infrastructure, leaving between $76 and $121 unused each month.

There are a few things the QULOOBUL MOE'MIENEEN needs to keep in mind regarding the grant:

- The grant must be renewed every year. If renewal is missed or the QULOOBUL MOE'MIENEEN loses its nonprofit verification, the Azure subscription immediately reverts to standard commercial billing.
- Unused credits do not carry over to the next year - any remaining balance at the end of the grant period is forfeited.
- The $2 000 credit is per organisation, not per project. If the QULOOBUL MOE'MIENEEN runs other Azure services alongside this platform, those costs will also come out of the same credit.

3.3 Ongoing and Additional Costs

The table below covers recurring costs, upgrade paths, and anything else the QULOOBUL MOE'MIENEEN should budget for after the initial project handover.

Item	Cost / Value	Detail & Risk
Microsoft for Nonprofits Azure Credit [67]	**Up to $2,000/year (≈ R33,000/year)**	The client already holds this grant. Must be renewed annually - if renewal lapses, the subscription immediately reverts to standard pay-as-you-go billing.
Azure infrastructure (24/7, Phase 1)	**~$28-$90/month depending on tier**	Linux B1 + SQL S0 = ~$28/month (lower bound). Windows B1 + SQL S1 = ~$84/month (upper bound). Both scenarios are comfortably within the $166/month grant equivalent.
Apple Developer Program renewal [51]	**$99/year (~R1,850/year)**	Annual fee. Required to maintain App Store distribution and TestFlight access for the iOS mobile client. One team member or the QULOOBUL MOE'MIENEEN must own this account.
Google Play one-time fee [74]	**$25 once (~R465)**	Paid once at app submission. No annual renewal. The QULOOBUL MOE'MIENEEN retains the developer account after the project ends.
Post-graduation scale-up (Phase 2)	**Azure SQL S2/S3 or vCore tier**	If the QULOOBUL MOE'MIENEEN onboards additional departments or a second client, the SQL tier can be scaled up with no architectural change. vCore pricing ($0.0281/vCore-hour, 2 vCores minimum)

		becomes cost-effective above ~50 DTUs.
Licensing	**$0 ongoing**	All core software (.NET 10, EF Core, ASP.NET Core, .NET MAUI, Visual Studio Community, SSMS, SQLite) is free and open-source. No proprietary runtime licences are required.
Support & Upgrades	**$0 (LTS support by Microsoft)**	.NET 10 is Long-Term Support (LTS) - free security patches and runtime updates until November 2026. Azure managed services receive patches automatically.

3.4 Cost Scenarios

The table below summarises costs across two scenarios - one with minimal infrastructure (lower bound) and one with slightly more headroom (upper bound) - alongside the available grant credit.

Scenario	Monthly Estimate (USD)	Annual Estimate (USD)
24/7 - Linux B1 + SQL S0 (lower bound)	~$45/month	~$540/year
24/7 - Windows B1 + SQL S1 (upper bound)	~$84/month	~$1 008/year
Microsoft for Nonprofits credit available [67]	**$166/month**	**$2 000/year**

3.5 Optional: Invoicing Module Costs

The API research document identifies a possible future extension to add an invoicing or basic accounting module alongside the task management system. This is not part of the current scope, but the QULOOBUL MOE'MIENEEN should be aware of the potential costs if this is pursued later.

- Xero API [60] (recommended): Xero Starter costs R90 for first 3 months then R450/month (20 invoices/month), Xero Standard R159 for first 3 months then R795/month (unlimited invoices). Xero offers a registered QULOOBUL MOE'MIENEEN discount of up to 80% through the Xero for Good programme - at the Standard tier with the full discount, this comes to approximately

R159/month. Xero has an official .NET SDK and supports South African bank feed reconciliation (FNB, ABSA, Standard Bank, Nedbank).

- Sage Business Cloud [61] (South Africa-native alternative): from R240/month. Built specifically for South African VAT and SARS compliance. No official .NET SDK - requires manual HttpClient calls from ASP.NET Core. QULOOBUL MOE'MIENEEN pricing is available by negotiating directly with Sage.
- Invoice Ninja [63] (lightweight open-source option): Free if self-hosted on Azure Container Instances (~$10-$20/month for the container). Supports ZAR and South African VAT. Not a full accounting platform - suitable if the QULOOBUL MOE'MIENEEN only needs invoice generation without double-entry bookkeeping.

All three invoicing options are independent of the core platform's Azure infrastructure costs and would be added on top of the $45-$90/month estimate in Section 3.1. The QULOOBUL MOE'MIENEEN's accountant or bookkeeper should be consulted before choosing an accounting platform, as they are likely already familiar with one of these tools.

4. Feasibility Summary

The table below gives an overview of the feasibility assessment across all areas covered in this document.

Feasibility Area	Assessment	Reasoning
Hardware	**FEASIBLE**	All server-side hardware is Azure-managed PaaS - the team requires only standard developer laptops and test devices. No on-premises server procurement needed.
Software	**FEASIBLE**	The full stack (.NET 8, Visual Studio, EF Core, Azure services) is free or covered by the Microsoft for Nonprofits [67] grant. No proprietary runtime licences are required.
Network	**FEASIBLE**	Standard broadband internet is sufficient for development and deployment. Azure enforces TLS and provides built-in firewall rules; no specialised on-premise network infrastructure is required.

Database [44]	**FEASIBLE**	Azure SQL Database (Standard S0/S1) meets the client's cloud-based database requirement at $14.72-$29/month, well within the grant. Full T-SQL ownership is maintained for marking purposes.
Financial (Phase 1 - 24/7)	**FEASIBLE**	Estimated $28-$90/month for continuous operation. The Microsoft for Nonprofits [67] Azure credit provides $166/month equivalent; cost is fully covered with margin. Grant must be applied for and renewed annually.
Ongoing / Post-graduation	**FEASIBLE (conditional)**	Infrastructure scales up by adjusting Azure service tiers. Annual Apple Developer fee ($99) is the only non-Azure recurring cost. Continued feasibility depends on grant renewal or the QULOOBUL MOE'MIENEEN absorbing ~$28-$90/month directly.

Overall: The project is both technically and financially feasible. The entire backend stack is either free or open source, all server infrastructure runs on Azure as managed PaaS (no physical hardware needed), and the estimated running cost of $45-$90/month is comfortably within the existing Microsoft for Nonprofits [67] grant. The only real risk is that the grant needs to be renewed every year - if it lapses, the QULOOBUL MOE'MIENEEN would need to cover the Azure costs directly, which is still manageable at this scale.

Members CV Documents

Patrick Michael Neale Shutte

Computer Science Student – Software Development
Number: 066 268 8019 | Email: patshutte@gmail.com | GitHub:
https://github.com/ST10440987-Patrick

Projects:

Kanban board – Java	May 2024
Student Management System – Java	August 2024
Motorcycle Dealership Management System – Java	August 2024
Movie Ticket Sales Report System – Java	September 2024
Cybersecurity Awareness Chatbot – C# WPF	June 2025
Bookings Management System – C# .NET Core MVC	June 2025
Retail Management System – C# .NET Core MVC	November 2025
Contract Monthly Claim System – C# ASP .NET Core MVC	November 2025
Global Logistics Management System – C#, ASP .NET Core, Blazor, Docker	June 2026
Lucky Budget Mobile App – Android, Kotlin	June 2026

EDUCATION:

BCAD – Bachelor of Computer and Information Sciences in Application Development

Emeris University (Formerly Varsity College) **2024 – 2026** (Expected)

High School

Wynberg Boys' High School Matriculated 2023

TECHNICAL SKILLS:

Programming Languages: Java, C#, HTML, CSS, Kotlin, TypeScript, SQL, YAML

Tools: WPF, Azure, GitHub, Apache Netbeans, Visual Studio Code, Visual Studio, Android Studio, Oracle SQL Developer, Docker, Node.js

Frameworks: Entity Framework Core, MySQL, SQL Server, ASP .NET Core MVC, SignalR, Blazor WebAssembly

COURSES/CERTIFICATIONS (COMPLETED):

- Java Programming for Complete Beginners (Alison)
- Java Programming Masterclass (Alison)
- Introduction to C# Programming (Alison)
- Diploma in C# Programming (Alison)
- Cisco Introduction to Packet Tracer (Cisco Networking Academy)
- Networking Devices and Initial Configurations (Cisco Networking Academy – In-Progress)

Liam Swile

Computer Science Graduate Student • Software Developer

+27 60 528 2245 dswile7@gmail.com Kuilsriver, Cape Town

PROFILE

Motivated Computer & Information Sciences student with hands-on problem-solving experience and a passion for software development, systems thinking, and technology-driven solutions. Combining a strong academic foundation in programming and IT with practical communication and organizational skills. Seeking an entry-level software development or IT role where I can apply and expand my technical abilities.

EDUCATION

Bachelor of Computer & Information Sciences *2019 – Present*

Emeris (formerly IIE Varsity College)

- Studying core areas including programming, data structures, software engineering, and information systems

Project & Event Management Platform | Quloobul Moe'mieneen NPO *Jul 2026 – Dec 2026*

Frontend Developer — Team of 4

- Built the front end of a full-stack web app (Blazor, Visual Studio) and a companion Android mobile app (Android Studio) to help a non-profit organization manage projects, events, and staff
- Engaged directly with NPO stakeholders to gather requirements and translate real organizational needs into working features
- Collaborated within a 4-person development team, combining research, structured coding practices, and stakeholder feedback throughout the build

National Senior Certificate (Bachelor's Admission) *2018*

Bellville Hoër Tegniese Skool

TECHNICAL SKILLS

Programming: Java, Python, HTML/CSS, JavaScript (academic coursework)

Concepts: Data structures & algorithms, OOP, basic database management, systems analysis

Tools: MS Office Suite, POS systems, basic network troubleshooting

Languages: English (fluent), Afrikaans (basic)

KEY STRENGTHS

- Strong analytical and logical problem-solving mindset
- Ability to work independently and as part of a team
- Proven track record of reliability and punctuality
- Fast learner with a drive to continuously develop technical skills
- Effective communicator across technical and non-technical audiences

Available for immediate start • Open to graduate roles, internships & learnerships

Dillon Matthew Rinkwest
(+27)82 773 0354| dillonrinkwest@outlook.com | linkedin.com/in/dillon-rinkwest-1005702a0 | GitHub: https://github.com/ST10439147

EDUCATION

Emeris Newlands (Varsity College) — **Cape Town, South Africa**

Bachelor of Computer and Information Sciences in Application Development — April 2027

PROJECTS

Kanban board | *Java* — May 2024

- Built a task management system in Java with auto-generated task IDs, configurable time estimates, and multi-developer task assignment.
- Implemented user registration and authentication, restricting access to the task management dashboard.
- Designed a menu-driven CLI enabling developers to create, assign, and navigate tasks through a structured interface.
- Supported CRUD operations (create, search, delete, report) on tasks filtered by developer name, enabling team-level workflow visibility.
- Applied structured reporting to generate task summaries across multiple developer assignments and task states.
- Reinforced software development lifecycle practices by managing multi-developer workflows from creation to completion.

Student Management System| *Java* — August 2024

- Data management application using structured arrays to capture, store, and manage student records with full CRUD operations.
- Implemented input validation across all student fields (ID, name, age, email, course) to ensure data integrity and consistency.
- Designed a lookup and reporting module enabling record search, deletion, and full dataset display using unique student identifiers.

Bookings Website | C#.NET Core MVC — May 2025

- Full-stack web application using ASP.NET Core Razor Pages on .NET 8, applying scalable MVC architecture and DevOps best practices.
- Implemented RESTful CRUD endpoints for bookings, events, and venues, supporting full entity lifecycle management through a user-facing interface.
- Leveraged Entity Framework Core for relational database management, implementing code-first migrations and structured data models.
- Applied secure coding standards and a custom storage abstraction layer, ensuring a maintainable, scalable, and production-ready application design.

Global Logistics Management System (GLMS) | *C# / ASP.NET Core 10 / Blazor Server / Docker* — June 2026

- Built a full-stack logistics platform with a Blazor Server frontend and a dedicated ASP.NET Core 10 REST API backend, containerised with Docker Compose (three services: frontend, backend, SQL Server 2022).
- Implemented JWT authentication with role-based access control (Admin, Manager, Viewer) and real-time push notifications via SignalR.
- Integrated live currency conversion via OpenExchangeRates API with 10-minute caching and applied Strategy, Observer, Factory, and Repository/Service Layer design patterns.
- Features include contract lifecycle management (Draft → Active → Expired), PDF upload with magic-byte validation, and automated contract status resolution.
- Configured a GitHub Actions CI/CD pipeline with two parallel jobs: one running dotnet test on every push and pull request to master, and one building both Docker images using docker/build-push-action to validate containerisation on each commit.

Lucky Budget | *Kotlin / Android* — June 2026

- Developed a personal finance Android application in Kotlin with account management, subscriptions, savings goals, transaction tracking, analytics, and a dashboard overview.
- Used Room Database for local data persistence with a multi-screen architecture and custom views, and later migrated to Firebase, including a DonutChart for visual spend analytics.
- Set up a GitHub Actions CI/CD pipeline that builds the debug APK, runs JUnit unit tests and Android Lint on every push, and uploads the APK, lint report, and test results as downloadable artefacts; Gradle dependency caching reduces build time across runs.

CERTIFICATIONS

- Cisco Introduction to Cybersecurity
- Cisco Introduction to Packet Tracer

SKILLS

- **Languages**: CSS, Java, Bash, YAML, SQL, C#, MongoDB, PL/SQL, Kotlin.
- **Tools**: Git, GitHub, Docker, Visual Studio Code, Apache NetBeans, Visual Studio, MongoDB Shell, Oracle SQL Developer, MS Word, MS Excel, Android Studio, Docker.
- **Frameworks**: .NET, ASP.NET Core MVC, Entity Framework Core, MySQL, SQL Server, Oracle SQL developer, Firebase, SignalR, Blazor Server
- **Platforms**: Linux. GitHub Actions (CI/CD). Windows Server. Cisco Networking

Uzair Essa

Age: 21
Driver's License: Valid Driver's License
Email: uzairessa2005@gmail.com
Phone: 0646229344
Location: Cape Town, Western Province, South Africa
GitHub: https://github.com/Uzair-Essa

Professional Profile

Motivated and detail-oriented final-year Bachelor of Computer and Information Sciences student specializing in Application Development at Emeris University. Possesses strong programming, problem-solving, and analytical skills developed through academic projects and real-world software development experience. Currently working on an industry-based assignment involving software development for a real company, applying modern programming principles, software engineering practices, and client-focused solutions.

In addition to technical expertise, brings two years of mathematics tutoring experience for Grade 11 and Grade 12 students, demonstrating excellent communication, leadership, and mentoring abilities. Seeking opportunities to launch a successful career in software development, application development, or related IT fields.

Education

Bachelor of Computer and Information Sciences in Application Development

Emeris University
Expected Graduation: 2026

Current Status: Final Year Student

Key Areas of Study

- Software Development
- Application Development
- Database Design and Management
- Object-Oriented Programming
- Systems Analysis and Design
- Web Development
- Information Systems
- Software Engineering
- Project Management
- Data Structures and Algorithms

Industry Project Experience

Software Development Project for Industry Client

2024 – Present

As part of final-year studies, currently participating in the design and development of a software solution for an actual company.

Responsibilities include:

- Requirements gathering and analysis
- Application design and development
- Database implementation
- Testing and debugging software solutions
- Working within project deadlines
- Collaborating with team members and stakeholders
- Applying software development best practices

Work Experience

Mathematics Tutor

Self-Employed / Private Tutor
2025 – Present (2 Years)

Provided mathematics tutoring for Grade 11 and Grade 12 students.

Key Responsibilities:

- Conducting one-on-one and small-group tutoring sessions
- Explaining complex mathematical concepts in a simple manner
- Assisting students with exam preparation
- Developing problem-solving techniques
- Tracking learner progress and performance
- Building student confidence and academic success

Achievements:

- Assisted students in improving examination performance
- Developed strong communication and teaching skills
- Built effective mentoring and leadership abilities

Technical Skills

Programming Languages

- Java
- C#
- Python
- SQL
- HTML
- CSS

Software Development

- Object-Oriented Programming (OOP)
- Software Testing and Debugging
- Agile Development Principles
- Application Design
- Database Development

Database Technologies

- SQL Server
- MySQL
- Firebase
- Database Design and Normalization

Tools & Technologies

- Visual Studio
- GitHub
- Microsoft Office Suite

Academic Highlights (2024–2025)

Based on submitted assessment results, demonstrated competency in:

- Mathematical problem solving
- Geometry and trigonometry
- Algebra
- Fractions and ratios
- Measurement and calculations
- Critical thinking
- Logical reasoning
- Numerical analysis

Core Competencies

- Problem Solving
- Analytical Thinking
- Creative Thinking
- Software Development
- Team Collaboration
- Communication Skills
- Time Management
- Attention to Detail
- Adaptability
- Leadership and Mentoring
- Client-Focused Development

Certifications & Licenses

- Valid Driver's License

References

Available upon request.

Professional Summary for Applications

"Final-year Bachelor of Computer and Information Sciences (Application Development) student with practical software development experience gained through an industry-based project for a real company. Strong foundation in programming, database development, and software engineering, complemented by two years of mathematics tutoring experience. Passionate about developing innovative technology solutions while continuously expanding technical and professional skills."

Prototypes

Web application

GitHub

https://github.com/DRPS-Solutions/INSY7315_Prototype_v1.0/tree/dev

Video demonstration

https://youtu.be/iajl1yTe4pc

Mobile application

GitHub

https://github.com/DRPS-Solutions/INSY7315_POE_Mobile_Prototype_v1.0

Video demonstration

https://youtu.be/QPmFQNbdpw8

References

[1] “9 Non-functional Software Quality Characteristics.” Accessed: Aug. 16, 2026. [Online]. Available: https://www.itconvergence.com/blog/9-non-functional-software-quality-characteristics/

[2] “Purchasing Models - Azure SQL Database | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-sql/database/purchasing-models?view=azuresql

[3] “Implementing the infrastructure persistence layer with Entity Framework Core - .NET | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-implementation-entity-framework-core

[4] “Grant limited access to data with shared access signatures (SAS) - Azure Storage | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/storage/common/storage-sas-overview

[5] E. F. Codd, “A relational model of data for large shared data banks,” *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.

[6] T. M. . Connolly and C. E. . Begg, *Database systems : a practical approach to design, implementation, and management*. Addison-Wesley, 2010.

[7] “Nonprofit Offerings and Products - Microsoft for Nonprofits | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/industry/nonprofit/microsoft-for-nonprofits/nonprofit-offerings-products

[8] “Security Overview - Azure SQL Database & Azure SQL Managed Instance & Azure Synapse Analytics | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-sql/database/security-overview?view=azuresql

[9] “DTU-Based Purchasing Model - Azure SQL Database | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-sql/database/service-tiers-dtu?view=azuresql

[10] “Serverless compute tier - Azure SQL Database | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-sql/database/serverless-tier-overview?view=azuresql

[11] “Active Geo-Replication - Azure SQL Database | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-sql/database/active-geo-replication-overview?view=azuresql&tabs=tsql

[12] “Read Queries on Replicas - Azure SQL Database & Azure SQL Managed Instance | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-sql/database/read-scale-out?view=azuresql

[13] “Overview of ASP.NET Core SignalR | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction?view=aspnetcore-10.0

[14] P. P.-S. Chen, “The entity-relationship model—toward a unified view of data,” *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, Mar. 1976, doi: 10.1145/320434.320440.

[15] “OWASP Top 10:2025.” Accessed: Aug. 08, 2026. [Online]. Available: https://owasp.org/Top10/2025/

[16] “Government Gazette REPUBLIC OF SOUTH AFRICA P ARLIAMENT of the Republic of South Africa therefore enacts, as follows:-CONTENTS OF ACT,” Cape Town, Nov. 2013.

[17] Martin. Fowler, *Patterns of enterprise application architecture*. Addison-Wesley, 2003.

[18] “Migrations Overview - EF Core | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/?tabs=dotnet-core-cli

[19] Eric. Evans, *Domain-driven design : tackling complexity in the heart of software*. Addison-Wesley, 2007.

[20] Erich. Gamma, Richard. Helm, R. E. . Johnson, and John. Vlissides, *Design patterns : elements of reusable object-oriented software*. Addison-Wesley, 2016.

[21] “Use hubs in ASP.NET Core SignalR | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/signalr/hubs?view=aspnetcore-10.0

[22] J. W. . Satzinger, R. B. . Jackson, and S. D. . Burd, *Systems analysis and design : in a changing world*, 7th ed. Boston: Cengage Learning, 2016.

[23] “Dependency injection in ASP.NET Core | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection?view=aspnetcore-10.0

[24] “Azure Key Vault configuration provider | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/security/key-vault-configuration?view=aspnetcore-10.0

[25] “ASP.NET Core Blazor hosting models | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/blazor/hosting-models?view=aspnetcore-10.0

[26]. “NET Multi-platform App UI documentation - .NET MAUI | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/maui/?view=net-maui-10.0

[27] “ASP.NET Core middleware | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/?view=aspnetcore-10.0

[28] R. Thomas. Fielding, *Architectural styles and the design of network-based software architectures*. [University of California, Irvine], 2000.

[29] Len. Bass, Paul. Clements, and Rick. Kazman, *Software architecture in practice*. Addison-Wesley, 2021.

[30] M. Jone, J. Bradley, and N. Sakimura, “RFC 7519: JSON Web Token (JWT),” Internet Engineering Task Force (IETF). Accessed: Aug. 08, 2026. [Online]. Available: https://www.rfc-editor.org/info/rfc7519/

[31] Dave. Farley and Jez. Humble, *Continuous delivery : a handbook for building, deploying, testing and releasing software*. [Addison-Wesley Professional], 2010.

[32] Mark. Richards and Neal. Ford, *Fundamentals of software architecture : an engineering approach*. O’Reilly, 2020.

[33] “What are Azure regions? | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/reliability/regions-overview

[34] “Azure App Service Quotas and Metrics - Azure App Service | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/app-service/web-sites-monitor

[35] “Set Up Staging Environments - Azure App Service | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/app-service/deploy-staging-slots?tabs=portal

[36] “What is Azure Static Web Apps? | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/static-web-apps/overview

[37] “About protected branches - GitHub Docs.” Accessed: Aug. 08, 2026. [Online]. Available: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

[38] “GitHub Actions documentation - GitHub Docs.” Accessed: Aug. 08, 2026. [Online]. Available: https://docs.github.com/en/actions

[39] “Fluent Assertions - Fluent Assertions.” Accessed: Aug. 08, 2026. [Online]. Available: https://fluentassertions.com/

[40] “GitHub - devlooped/moq: The most popular and friendly mocking framework for .NET · GitHub.” Accessed: Aug. 08, 2026. [Online]. Available: https://github.com/devlooped/moq

[41] “Integration tests in ASP.NET Core | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/test/integration-tests?view=aspnetcore-10.0&pivots=xunit

[42] “GitHub - coverlet-coverage/coverlet: Cross platform code coverage for .NET · GitHub.” Accessed: Aug. 08, 2026. [Online]. Available: https://github.com/coverlet-coverage/coverlet

[43] “Authenticate to Azure Key Vault | Microsoft Learn.” Accessed: Aug. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/key-vault/general/authentication

[44] “Pricing - Azure SQL Database Single Database | Microsoft Azure.” Accessed: Aug. 12, 2026. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/azure-sql-database/single/

[45] “Pricing – App Service for Linux | Microsoft Azure.” Accessed: Aug. 12, 2026. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/app-service/linux/

[46] “Microsoft Entra External ID - Pricing | Microsoft Azure.” Accessed: Aug. 12, 2026. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/microsoft-entra-external-id/

[47] “Azure Blob Storage pricing | Microsoft Azure.” Accessed: Aug. 12, 2026. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/storage/blobs/

[48] “Azure SignalR Service Pricing | Microsoft Azure.” Accessed: Aug. 12, 2026. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/signalr-service/

[49] “Pricing - Key Vault | Microsoft Azure.” Accessed: Aug. 12, 2026. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/key-vault/

[50] “Pricing - Notification Hubs | Microsoft Azure.” Accessed: Aug. 12, 2026. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/notification-hubs/

[51] “Apple Developer Program - Apple Developer.” Accessed: Aug. 12, 2026. [Online]. Available: https://developer.apple.com/programs/

[52] “Pricing · Resend.” Accessed: Aug. 12, 2026. [Online]. Available: https://resend.com/pricing

[53] “Azure Communication Services pricing | Microsoft Azure.” Accessed: Aug. 12, 2026. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/communication-services/

[54] “Email API Pricing | Twilio.” Accessed: Aug. 12, 2026. [Online]. Available: https://www.twilio.com/en-us/products/email-api/pricing

[55] “Pricing - Claude Platform Docs.” Accessed: Aug. 12, 2026. [Online]. Available: https://platform.claude.com/docs/en/about-claude/pricing

[56] “Pricing | OpenAI API.” Accessed: Aug. 12, 2026. [Online]. Available: https://developers.openai.com/api/docs/pricing

[57] “Gemini Developer API pricing | Gemini API | Google AI for Developers.” Accessed: Aug. 13, 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs/pricing

[58] “Microsoft Entra Plans and Pricing | Microsoft Security.” Accessed: Aug. 13, 2026. [Online]. Available: https://www.microsoft.com/en-za/security/business/microsoft-entra-pricing

[59] “Pricing — Free Up to 50K Users | Plans from $0/mo.” Accessed: Aug. 13, 2026. [Online]. Available: https://clerk.com/pricing

[60] “404 | Xero ZA.” Accessed: Aug. 12, 2026. [Online]. Available: https://www.xero.com/za/ngo/

[61] “Management Software & Solutions for South African Businesses | Sage South Africa.” Accessed: Aug. 12, 2026. [Online]. Available: https://www.sage.com/en-za/

[62] “QuickBooks Online Pricing | Try It Free | QuickBooks South Africa.” Accessed: Aug. 12, 2026. [Online]. Available: https://quickbooks.intuit.com/za/pricing/?_gl=1*1vncv42*_up*MQ..*_gs*MQ..&gclid=CjwKCAjws_DTBhB_EiwAXZknGT8RfiBwdKoAMC6fluEJ1PhwR-pV4jBnWot_6EfvswFp1Ydzbe64XBoCUoMQAvD_BwE&gclsrc=aw.ds&gbraid=0AAAADD1w8J_3oZRfuHQTuSAe_jG1aEQZU

[63] “Pricing - Invoice Ninja.” Accessed: Aug. 12, 2026. [Online]. Available: https://invoiceninja.com/pricing-plans/

[64] “Invoicing built for speed and scale.” Accessed: Aug. 12, 2026. [Online]. Available: https://stripe.com/invoicing/pricing

[65] “Pricing Calculator | Microsoft Azure.” Accessed: Aug. 12, 2026. [Online]. Available: https://azure.microsoft.com/en-us/pricing/calculator/

[66] “Pricing – Static Web Apps | Microsoft Azure.” Accessed: Aug. 13, 2026. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/app-service/static/

[67] “Nonprofit Solutions & Technology | Microsoft Nonprofits.” Accessed: Aug. 12, 2026. [Online]. Available: https://www.microsoft.com/en-us/nonprofits

[68] “OWASP Top 10 (2026): The Most Critical Web Application Security Risks Every Developer Should Understand | by Nikoo Asadnejad | Medium.” Accessed: Aug. 13, 2026. [Online]. Available: https://medium.com/@nikoo.asadnejad.work/owasp-top-10-2026-the-most-critical-web-application-security-risks-every-developer-should-d41531188c9a

[69] “What is OWASP? OWASP Top 10 Security By Design Principles - Patchstack.” Accessed: Aug. 13, 2026. [Online]. Available: https://patchstack.com/articles/security-design-principles-owasp/

[70] “Protection of Personal Information Act (POPI Act) - POPIA.” Accessed: Aug. 13, 2026. [Online]. Available: https://popia.co.za/

[71] “What is Data Minimisation? | Experian Business.” Accessed: Aug. 13, 2026. [Online]. Available: https://www.experian.co.uk/business/glossary/data-minimisation/index

[72] T. Fisher, “Data residency: What is it and why is it important? | IBM.” Accessed: Aug. 13, 2026. [Online]. Available: https://www.ibm.com/think/insights/data-residency-why-is-it-important

[73] “What is encryption? | IBM.” Accessed: Aug. 13, 2026. [Online]. Available: https://www.ibm.com/think/topics/encryption

[74] “Creating a developer account.” Accessed: Aug. 12, 2026. [Online]. Available: https://play.google.com/console/signup

AI Declaration of Usage

Sections where AI was used:

- Entire Document

AI Tool Used: Grammarly App

Purpose/Intent:

- I used Grammarly to identify any spelling errors to correct.
- I used it to help lay out my sentences for my answers.

Dates used: Throughout

Link: [Grammarly app is built in, no link]

Action taken:

- I took what Grammarly gave me and used it to format my work to sound more professional and to have better word flow.

Sections where AI was used:

- Database choice

AI Tool Used: Claude Sonnet 4.6

Purpose/Intent:

- I used Claude to decide and do the research on what Database to use, with its reasoning.
- Used to combine the members ' research documents of the members to create a tech stack and see if it is plausible.

Date used: 28 May 2026

Link: https://claude.ai/share/95aeda5a-f6a0-4069-b8e7-1263f1a7a5ea

Action taken:

- I took what Claude gave me and discussed it with the group to choose the best tech stack for this scenario for the NPO.

Sections where AI was used:

- Use case briefs and use case diagrams

AI Tool Used: Claude Sonnet 5

Purpose/Intent:

- I used this tool to help with the fast creation of use case briefs and to use standard templates.

Date used: 22 July 2026

Link: https://claude.ai/share/c7972ce8-e755-46de-b7ac-b74dc10bebbe

Action taken:

- I used the template to help create the diagrams and briefs

Sections where AI was used:

- Risk Matrix

AI Tool Used: Claude Sonnet 5

Purpose/Intent:

- I used this tool to create and understand the risk matrix format.

Date used: 22 July 2026

Link: https://claude.ai/share/049f264b-9710-4975-a557-3d7b6eb69a84

Action taken:

- I used the template to help create the format used for the risk matrix

Sections where AI was used

- Possible APIs to consider

AI Tools used: Claude Sonnet 5

Purpose/Intent:

- I used Claude to give me ideas of the possible APIs to consider for AI, Invoicing, Emails/Notifications, Azure and Authentication Extensions

Dates used: 26 May 2026

Link: https://claude.ai/share/0403027f-af8e-4a97-a54c-6dbfcbaa7943

Action taken:

- I took what Claude gave me as possible API options to use, and I then did more research of and took a collection of the APIs that I found came up more often and then went further and looked at pricing as well as advantages and disadvantages of each API.

Sections where AI was used

- Security Analysis Report

AI Tools used: Gemini 3.6 Flash

Purpose/Intent:

- I used it to get ideas of all possible security threats to any IT systems and what are the more common ones and how to mitigate those security threats

Dates used: 27 July 2026

Link: https://share.gemini.google/5kSXtOi692F6

Action taken:

- I took what Gemini gave me and incorporated it into the document along with additional research such as security principles and what concepts such as Encryption, Data Minimisation, Data Residency are.

Sections where AI was used

- Technical And Financial Report

AI Tools used: Claude Sonnet 5

Purpose/Intent:

- Based on our NPO_Recommended_Stack_Revised document, I got Claude to tell me what is feasible both technically and financially and then did additional research to find out what Azure services cost, what the tech stack would cost based on what we might use.

Dates used: 22 July

Link: https://claude.ai/share/44cb3b1e-4027-46e7-b26b-764f5b3d0ec7

Action taken:

- I took what Claude gave me, and I applied it in the additional research that I needed to do getting prices of the Azure resources and the tech stack that we would be using to get an idea of the total estimated cost for keeping this system running for the client 24/7.

Sections where AI was used

- Non-Functional Requirements

AI Tools used: Gemini 3.6 Flash

Purpose/Intent:

- I used Gemini to give me the typical non-functional requirements, both common as well as less common, but important requirements for IT systems.

Dates used: 16 August 2026

Link: https://share.gemini.google/ympHwZHU3NXn

Action taken:

- I took what Gemini gave me, and I did some more research on non-functional requirements, and I implemented both the research as well as what Gemini gave me into this document. I also used our internal group documents to help get an idea of which of the non-functional requirements align well with our specific application that we are developing.